

Table of content

Chapter 1. Introduction	1
1.1 Project description	1
1.2 Problem Domain and Project as a solution	2
1.3 Aims and Objectives	3
1.3.1 Aims	3
1.3.2 Objectives	3
1.4 Report Structure	4
1.4.1 Background	4
1.4.2 Development	4
1.4.3 Testing and analysis	4
1.4.4 Conclusion	4
Chapter 2: Background	5
2.1 About the End User	5
2.2 Understanding of the Project	5
2.3 Review of Similar Projects	6
2.3.1 Similar Project	6
2.3.2 Comparison of Similar Project	8
Chapter 3: Development	9
3.1 Considered Methodologies	9
3.1.1 Iterative waterfall model	9
3.1.2 Agile Scrum Methodology	10
3.2 Selected Methodology	11
3.3 PHASE OF METHODOLOGY	12
3.4 SURVEY RESULTS	13
3.4.1 POST-SURVEY RESULTS	13
3.5 REQUIREMENT ANALYSIS	13
3.6 DESIGN	13
3.7 Implementation	14
3.7.1 Increment 1 (User Authentication and User Location Tracking)	14
3.7.1.1 Login and Sign-up page	14
3.7.1.2 Email verification code	16
3.7.1.3 Reset password code	17
3.7.1.4 User authentication model.py code	18

3.7.1.5 User authentication model migration.....	19
3.7.1.5 User authentication model test cases code.....	19
3.7.1.5 User location tracking.....	20
3.7.2 Increment 2 OCR Implementation.....	23
3.7.2.1 Document Upload View.py code	23
3.7.2.2 Data annotation using Robo flow (Front side of the document).....	24
3.7.2.3 Data annotation using rob flow (back side of the document)	26
3.7.2.6 Yolo model fine-tuning code for document validation	27
3.7.2.7 Testing the yolo model for document validation	29
3.7.2.8 Extraction of face region from the front side of the document.....	30
3.7.2.9 OCR donut fine tuning.....	31
3.7.3 Increment 3	33
3.7.3.1 Dashboard UI.....	33
3.7.3.1 Dashboard logic code.....	34
3.7.3.3 face detection/verification integration code.....	35
CHAPTER 4: TESTING AND ANALYSIS	37
4.1 Test plan.....	37
4.1.1 UNIT TESTING and URL TESTING PLAN.....	37
4.1.2 Test case plan, system testing	38
4.2 Unit Testing.....	39
4.2.1 Unit testing 1 : Testing if the valid phone number is accepted by the system or not	39
4.2.2 Unit testing 2 : Testing the validation of the email address	40
4.2.3 Unit testing 3: Testing if the password and re-type password are different, then an error message is displayed or not.	41
4.2.4 Unit testing 4 : Testing if the password and re-type password are the same, then the user object is created or not.	42
4.2.5 Unit testing 5: Testing if the invalid email is submitted, then the error message is responded to by the system or not.....	43
4.2.6 Unit testing 6 : Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.	44
4.2.7 Unit testing 7 : Testing if the already existing email in the database is accepted by the system or not.	45
4.2.8 Unit testing 8 : Testing if the already existing phone number in the database is accepted by the system or not.	46
4.2.9 Unit testing 9 Testing if the verification link is sent to the user's email.	47

4.2.10 Unit testing 10 : Testing if the user verifies their email through the verification link	48
4.2.11 Unit testing 11 Testing if the logout function works or not	49
4.2.11 Unit testing 12 Testing if the login functionality works with the correct credentials	50
4.2.13 Unit testing 13: Testing if the login functionality works or not with the unverified email.....	51
4.2.14 Unit testing 14 : Testing if the login functionality works or not with the wrong password	52
4.2.15 Unit test 15: Testing if the login functionality works or not with the email that does not exist in the system.	53
4.2.16 Unit testing 16 Testing if the Reset password functionality	54
4.2.17 Unit testing 17: Testing if the Reset password link is sent to the user's email.....	55
4.3 URL TESTING	56
4.3.1 URL Testing 1: Testing the URL of the register view.....	56
4.3.2 URL Testing 2 Testing the URL of the login view	57
4.3.3 URL Testing 3 : Testing the URL of the logout function.....	58
4.3.4 URL Testing 4: Testing the URL of the email verification	59
4.3.5 URL testing 5: Testing the URL of the reset password.....	60
4.3.6 URL Testing 6 : Testing the URL of the dashboard.....	61
4.4 SYSTEM TESTING	62
4.4.1 System testing 1 : Testing the user Registration process	62
4.4.2 System Testing 2: Testing the user login process.....	65
4.4.3 System Testing 3 : Testing the Reset Password Functionality	67
4.4.4 System Testing 4 : Testing the Document Upload Functionality along with user detail form.....	69
4.4.5 System Testing 5: Testing the User location tracking Functionality.....	72
4.4.6 System Testing 6: Testing the dashboard functionality.....	73
CHAPTER 5: CONCLUSION	76
5.1 LEGAL, SOCIAL and ETHICAL ISSUES	76
5.1.1 LEGAL ISSUES	76
5.1.2 SOCIAL ISSUES	76
5.1.3 ETHICAL ISSUES	76
5.2 ADVANTAGES	76
CHAPTER 6: REFERENCES	77
References.....	77

CHAPTER 8: APPENDIX.....	79
8.1 APPENDIX A: POST SURVEY	79
8.1.1 Sample of filled post survey forms	79
8.2.2 Survey results.....	83
8.1 APPENDIX B: SOFTWARE REQUIREMENTS SPECIFICATIONS.....	87
8.1.1 Introduction.....	88
8.1.2 Overall Description.....	88
8.1.3 Assumption and Dependencies	90
8.1.4 External Interface Requirements.....	90
8.1.5 System Features	92
8.3 APPENDIX C: SAMPLE CODES	96
8.3.1 SAMPLE CODE OF THE UI.....	96
8.3.2 SAMPLE CODE OF THE AUTOMATION SCRIPTS.....	106
8.4 APPENDIX D: DESIGNS	111
8.4.1 GANTT CHART.....	111
8.4.2 WORK BREAK DOWN STRUCTURE	112
8.4.3 MILESTONE CHART.....	113
8.4.4 SYSTEM ARCHITECTURE.....	113
8.4.5 ERD.....	114
8.4.6 USE CASE DIAGRAM.....	115
8.4.7 SEQUENCE DIAGRAM.....	115
8.4.8 COLLABORATION DIAGRAM.....	121
1) Collaboration Diagram of User Registration.....	121
8.4.9 FLOWCHART	126
8.4.10 ACTIVITY DIAGRAM.....	131
8.4.11 WIREFRAME.....	136
8.5 APPENDIX E: SCREENSHOT OF THE SYSTEM	144
8.6 APPENDIX F : LOG SHEET	152
8.6.1 log sheet 1	152
8.6.3 log sheet 3	154
8.6.4 logs sheet 4.....	155
8.6.5 log sheet 5	156
8.6.6 log sheet 6	157
8.6.7 log sheet 7	158
8.6.8 log sheet 8	159

8.6.9 log sheet 9	160
8.6.10 log sheet 10	161
8.6.11 log sheet 11	162

Table of figures

Figure 1 Digital bank account opening (International, 2024).....	1
<i>Figure 2 Finance managed digitally vs offline (Cocheo, 2024)</i>	<i>2</i>
Figure 3 User intention to open a bank account online through e-KYC (Elinzano, 2022).....	5
Figure 4 Orange nxt	6
Figure 5 Nbank	7
Figure 6 NIC asia mobank	7
Figure 7 Iterative waterfall model (geeksforgeeks, 2024)	9
Figure 8 Scrum Process (Bhaskar, 2024).....	10
<i>Figure 9 Incremental Model (More, 2023)</i>	<i>11</i>
Figure 10 Register user function code	14
Figure 11 User login logic code.....	15
Figure 12 Email verification logic code.....	16
Figure 13 Reset password logic code.....	17
Figure 14 Model.py code for user authentication	18
Figure 15 User authentication module model migration	19
Figure 16 User authentication module unit test cases code	19
Figure 17 User location tracking veiew.py	20
Figure 18 User location tracking code	21
Figure 19 User location tracking code 2	21
Figure 20 User location model.py code	22
Figure 21 Document upload view.py code	23
Figure 22 Data annotation using robo flow for front side of document	24
Figure 23 Data YAML file for front side	25
Figure 24 Data annotation result for front side.....	25
Figure 25 Data Annotation for Back side of the document	26
Figure 26 Data YAML file for backside dataset of document	26
Figure 27 Yolo model fine tuning code.....	27
Figure 28 Confusion matrix of front side model	28
Figure 29 Testing Yolo model for document validation.....	29
Figure 30 face detection and crop of the document frontside.....	30
Figure 31 Code of data splitting for orc dount along with Json data formatting	31
Figure 32 Unit Test 1 valid phone number inspection	39

Figure 33 Result of unit test 1	39
Figure 34 Unit test 2: Valid user email inspection	40
Figure 35 Unit test 2 results	40
Figure 36 Unit test 3: Password and retype password validity check.....	41
Figure 37 Results of Unit Test 3	41
Figure 38 Unit test 4 password and retype password with user object creation check.....	42
Figure 39 Unit test 4 results	42
Figure 40 Unit test 5: Invalid email format check	43
Figure 41 Unit test 5 results	43
Figure 42 Unit test 6: user phone number validation.....	44
Figure 43 Unit test 6 results	44
Figure 44 Unit test 7: Email already existed check.	45
Figure 45 Unit test 7 results	45
Figure 46 Unit test 8: User phone number already existed or not.	46
Figure 47 Unit test 8 results	46
Figure 48 Unit test 9: Email Verification link sent	47
Figure 49 Unit test 9 results	47
Figure 50 Unit test 10 Email verification operation	48
Figure 51 Unit test 10 results	48
Figure 52 Unit test 11: Logout function.....	49
Figure 53 Unit test 11 results	49
Figure 54 Unit Test 12 login functionality with correct credentials	50
Figure 55 Unit 12 results.....	50
Figure 56 Unit 13 login functionality with unverified email.....	51
Figure 57 Unit Test 13 Results.....	51
Figure 58 Unit test 14 login functionality with incorrect password	52
Figure 59 Unit test 14 results	52
Figure 60 Unit test 15 login functionality with an email that does not exist	53
Figure 61 Unit test 15 results	53
Figure 62 Unit 16: Reset password functionality	54
Figure 63 Unit test 16 results	54
Figure 64 Unit test 17 Reset password email link	55
Figure 65 Unit test 17 results	55
Figure 66 URL Testing 1 register.....	56

Figure 67 Results of URL Testing 1	56
Figure 68 URL Test 2 login	57
Figure 69 URL test 2 results	57
Figure 70 URL testing 3 logout	58
Figure 71 URL testing 3 results	58
Figure 72 URL test 4 email verification	59
Figure 73 URL test 4 results	59
Figure 74 URL Test 5 reset password	60
Figure 75 URL testing 5 results	60
Figure 76 URL testing 6 code	61
Figure 77 Results of url testing 6.....	61
Figure 78 System Testing 1 User registration detail fill up	62
Figure 79 System Testing 1 Notification after successful registration	63
Figure 80 System Testing 1 Email verification link via mail	63
Figure 81 System Testing 1: Verifying the account	64
Figure 82 System Testing 1 Inspection of recently created used in database of the system....	64
Figure 83 System Testing 2 User credential for authentication to the system.....	65
Figure 84 System Testing 2: Redirection to document upload page after successful login....	66
Figure 85 System Testing 3: Password Reset	67
Figure 86 System Testing 3: Reset password link via email.....	68
Figure 87 System Testing 3: Reset password form.....	68
Figure 88 System Testing 4 User Document upload.....	69
Figure 89 System Testing 4 Success message after the document validation	70
Figure 90 System Testing 4: Document URL saved in the database after successful validation	70
Figure 91 System Testing 4: The document validation status is True after successful validation.....	70
Figure 92 System Testing 4 Redirect to the citizenship detail form	71
Figure 93 System Testing 5 location search.....	72
Figure 94 System Testing 6 user dashboard update user detail	73
Figure 95 System testing 6 updating the user details.....	74
Figure 96 System testing 6 saving the change details.....	74
Figure 97 System testing 6 inspection after updation.....	74
Figure 98 System Testing 6 user deletion dashboard.....	75

Figure 99 System Testing 6 Dashboard table after user deletion.....	75
Figure 100 Survey form How was your overall experience using Sajilo E-bank webapp?	79
Figure 101 Survey form: How simple was it to navigate and use the Sajilo E-bank	79
Figure 102 Survey form: How would you rate the performance and responsiveness of the system?.....	80
Figure 103 Survey form: Which functionality did you liked the most when using Sajilo E-bank web app ?.....	80
Figure 104 Survey form How effective did you find the form automation capabilities that utilize OCR to retrieve information from the submitted citizenship document?.....	81
Figure 105 Survey Form: How effective did you find the Facial Recognition features for KYC Verification ?	81
Figure 106 survey form: How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?	82
Figure 107 Survey form How probable is it that you would suggest this system to your colleagues?.....	82
Figure 108 Survey total responses	82
Figure 109 Survey results How was your overall experience using Sajilo E-bank webapp ?.	83
Figure 110 Survey result 2 : How simple was it to navigate and use the Sajilo E-bank web application?	83
Figure 111 Survey results 3 How would you rate the performance and responsiveness of the system?.....	84
Figure 112 Survey results 4 Which functionality did you liked the most when using Sajilo E-bank web app	84
Figure 113 Survey result 5 : How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?	85
Figure 114 Survey results 6 : How effective did you find the Facial Recognition features for KYC Verification ?	85
Figure 115 Survey form : How probable is it that you would suggest this system to your colleagues?.....	86
Figure 116 Survey form How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?	86
Figure 117 Document upload UI code	96
Figure 118 Email Verification UI Code	97
Figure 119 location tracking UI Code.....	98

Figure 120 Login page UI code	99
Figure 121 Password reset UI code	100
Figure 122 register page UI Code	101
Figure 123 Reset password link UI code	102
Figure 124 Dashboard UI code	103
Figure 125 Citizenship detail form UI code	104
Figure 126 Location UI code	105
Figure 127 Video capture UI code	105
Figure 128 Text extraction from image code	106
Figure 129 Face detection code	107
Figure 130 Ocr donut fine tuning note book.....	108
Figure 131 OCR donut testing code.....	108
Figure 132 Ocr donut data processing code.....	109
Figure 133 detect face and crop the face code	109
Figure 134 Back side and front side validation code	110
Figure 135 yolo model test code	110
Figure 136 Gantt chart	111
Figure 137 Work Breakdown Structure	112
Figure 138 Milestone chart	113
Figure 139 System architecture design	113
Figure 140 Initial ERD diagram	114
Figure 141 Use case diagram.....	115
Figure 142 Sequence Diagram for User Registration	116
Figure 143 Sequence Diagram of User Login	117
Figure 144 Sequence diagram of OCR form Automation	118
Figure 145 Sequence Diagram of User location Tracking.....	119
Figure 146 Sequence diagram of KYC Verification through facial recognition.....	120
Figure 147 Collaboration Diagram of User Registration.....	121
Figure 148 Collaboration Diagram of User Login.....	122
Figure 149 Collaboration Diagram of OCR form automation.....	123
Figure 150 Collaboration Diagram of User location tracking	124
Figure 151 Collaboration diagram of KYC verification through facial recognition	125
Figure 152 Flow Chart of user login.....	126
Figure 153 Flow Chart of User Registration.....	127

Figure 154 Flow Chart for Form Automation Using OCR	128
Figure 155 Flow Chart of User location Tracking	129
Figure 156 Flow chart of KYC verification through face recognition	130
Figure 157 Activity Diagram of user registration	131
Figure 158 Activity Diagram of user login	132
Figure 159 Activity Diagram of user location tracking	133
Figure 160 Activity Diagram of form automation through ocr	134
Figure 161 Activity Diagram of face recognition	135
Figure 162 Wireframe of login page	136
Figure 163 Wireframe of signup page	137
Figure 164 Wireframe of Document upload page	138
Figure 165 Wireframe of User Location Tracking page	139
Figure 166 Wireframe of personal detail form	140
Figure 167 Wireframe of face recognition for kyc verification	141
Figure 168 Django connected with MySQL databas	142
Figure 169 Database of the project	143
Figure 170 Login page	144
Figure 171 Signup page	145
Figure 172 Reset password email sent page	146
Figure 173 Document Upload	147
Figure 174 Location Tracking	148
Figure 175 Email verification	149
Figure 176 Reset password form	150
Figure 177 Citizenship information form	151
Figure 178 User Dashboard	151

Table of tables

Table 1 Similar system comparison	8
Table 2 Test case plan, unit and URL testing	38
Table 3 Test case plan system testing	38
Table 4 Unit testing 1 : Testing if the valid phone number is accepted by the system or not..	39
Table 5 Unit testing 2 Testing the validation of the email address	40
Table 6 Unit testing 3 : Testing if the password and re-type password are different, then an error message is displayed or not.....	41
Table 7 Unit testing 4 : Testing if the password and re-type password are the same, then the user object is created or not.	42
Table 8 Unit testing 5 : Testing if the invalid email is submitted, then the error message is responded to by the system or not.....	43
Table 9 Unit testing 6 : Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.	44
Table 10 Unit testing 7 : Testing if the already existing email in the database is accepted by the system or not.	45
Table 11 Unit testing 8: Testing if the already existing phone number in the database is accepted by the system or not.	46
Table 12 Unit testing 9 Testing if the verification link is sent to the user's email.	47
Table 13 10 Unit testing 10: Testing if the user verifies their email through the verification link	48
Table 14 Unit testing 11 Testing if the logout function works or not	49
Table 15 Unit testing 12 Testing if the login functionality works with the correct credentials	50
Table 16 Unit testing 13 : Testing if the login functionality works or not with the unverified email.....	51
Table 17 Unit testing 14 : Testing if the login functionality works or not with the wrong password	52
Table 18 Unit test 15: Testing if the login functionality works or not with the email that does not exist in the system.....	53
Table 19 Unit testing 16 Testing if the Reset password functionality	54
Table 20 Unit testing 17 : Testing if the Reset password link is sent to the user's email.....	55
Table 21 URL Testing 1: Testing the URL of the register view.....	56

Table 22 URL Testing 2 Testing the URL of the login view.....	57
Table 23 URL Testing 3 : Testing the URL of the logout function.....	58
Table 24 URL Testing 4 : Testing the URL of the email verification	59
Table 25 URL testing 5: Testing the URL of the reset password.....	60
Table 26 URL Testing 6: Testing the URL of the dashboard.	61
Table 27 System testing 1 : Testing the user Registration process	62
Table 28 System Testing 2: Testing the user login process.....	65
Table 29 System Testing 3 : Testing the Reset Password Functionality	67
Table 30 System Testing 4 : Testing the Document Upload Functionality along with user detail form.....	69
Table 31 System Testing 5: Testing the User location tracking Functionality.....	72
Table 32 System Testing 6 : Testing the dashboard functionality.....	73

Chapter 1. Introduction

1.1 Project description

Banking and cooperative sectors play a significant role in improving a country's economic development and financial stability by providing various types of financial services, like savings accounts, investment prospects, and loan and credit facilities, to low-income households to improve the living standards of societies and communities. (Y.A.Christian., 2018). The first thing a customer does when he/ she plan to save money in a bank or cooperative is open an account either by manually filling out the form or through the bank's online website portal. (Y.A.Christian., 2018). Manually filling out the account form in person, whether at a bank or a cooperative, is time-consuming, and prone to human error, long waiting lines, and extensive paperwork, all of which contribute to a bad customer experience.



Figure 1 Digital bank account opening (International, 2024)

Some of the renowned banks and cooperatives in Nepal have implemented an online account opening website portal but for KYC verification customers should be present in person for validation and customers must fill out their long personal details which is time-consuming. Using OCR, the personal information of the customer can be extracted by scanning the citizenship of the customer and for KYC Verification, a Face recognition system can be utilized for a seamless and easy account opening process.

1.2 Problem Domain and Project as a solution

According to Scott Albahary, there has been a 14.5 percent rise in online account openings since the onset of COVID-19, compared to a 9 percent increase in 2019. Only 40 percent of banks offer a complete account opening process through mobile apps, while 72 percent provide the option to open accounts online via their websites. However, nearly 30 percent of customers who attempt to open an account online still must visit a nearby branch for KYC verification, signatures, and other requirements, which disrupts the customer experience. (Albahary, 2020).

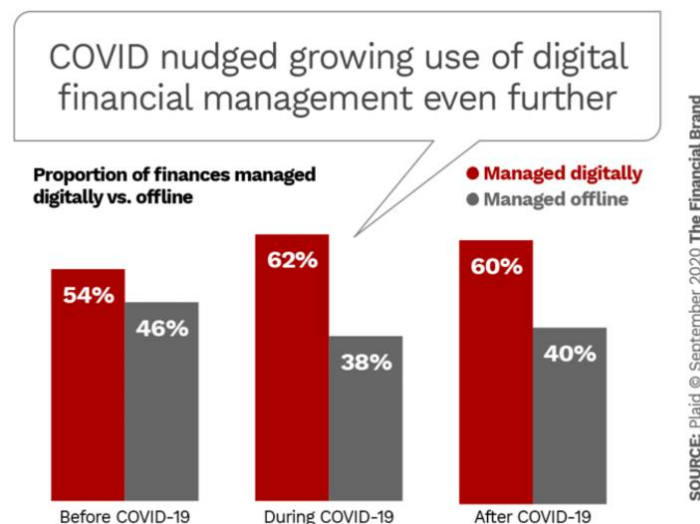


Figure 2 Finance managed digitally vs offline (Cocheo, 2024)

Opening a bank account online still necessitates that customers visit a nearby branch for identity verification, which can hinder the customer experience and waste their precious time; lengthy queues might also arise during the verification process. If customers are required to complete a paper form, they face the inconvenience of having to indicate their residence location on the form, along with spending time filling in personal information such as citizenship number, date of birth, and other details. All these issues can be addressed using OCR for automatic form completion and facial recognition technology for KYC verification. Customers can submit their citizenship documents, and the information is automatically populated, allowing KYC verification to be conducted online, thus conserving customers' valuable time and effort.

1.3 Aims and Objectives

1.3.1 Aims

The aim of this project is to digitise and streamline the bank and cooperative account opening process by applying technology such as OCR and facial recognition for KYC verification.

1.3.2 Objectives

- **Implement OCR for automation:** Obtain the necessary user information from the citizenship document submitted by the user using OCR technology to streamline the form-filling process.
- **Develop a Web application:** Develop an accessible and intuitive web application with an improved user interface design using the Django framework in Python programming language, aimed at streamlining the online account opening process for banks or cooperatives.
- **Integrate Email Notification System:** Implement an email notification system that delivers user account details following successful KYC verification.
- **Introduce facial recognition technology for KYC verification:** Integrate facial recognition capabilities into the system for smooth and precise KYC verification, including liveness detection.
- **Perform comprehensive testing and validation:** Execute extensive testing of web application functionalities such as OCR, facial recognition, location tracking, email notifications, and database to assess performance.

1.4 Report Structure

1.4.1 Background

The background section consists of the details about the end user, understanding of the project and review and comparison of similar projects

1.4.2 Development

The Development section consists of considered methodologies, selected methodology, work breakdown structure and Gantt chart, code implementation as per methodology for the project

1.4.3 Testing and analysis

The testing of the system is conducted with proper testing plan.

1.4.4 Conclusion

The legal, social and ethical issue regarding the project along with the advantage and the limitation of the project is shown.

Chapter 2: Background

2.1 About the End User

This product will be used by customers who are in the process of opening a bank/cooperative account online without their physical presence for identity verification to save time and energy.

2.2 Understanding of the Project

Customers must present their details on a form, followed by an in-person staff verification of their identity to start bank or cooperative accounts. The process damages customer satisfaction while taking up a significant amount of time. A financial institution needs to perform KYC verification of customer identification before establishing any bank account to understand customer history. (Elinzano, 2022).

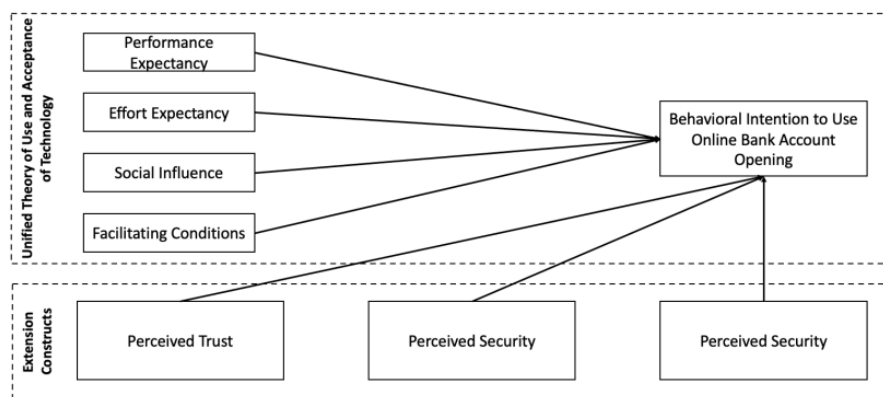


Figure 3 User intention to open a bank account online through e-KYC (Elinzano, 2022)

KYC is also conducted online by video calling, advanced facial recognition, electronic identity numbers, etc., which is called e-KYC (Leon Perlman, 2019). Previously, when the concept of e-KYC was unknown, the policy of physical presence at the bank branch for KYC was challenging, as it was cost-intensive and difficult for the customer (zetsche, 2018). The manual form fill was a hassle where the user had to draw their location on the paper and fill out a lot of details.

The project 'Sajilo e-bank will implement the OCR for automation form fill, which would make user easier to fill their details, and a facial recognition system for KYC verification online. The user will upload their citizenship document, and their detail will be extracted by using OCR for the form automation. The real-time location of the user will be tracked as per their permission for their record.

2.3 Review of Similar Projects

2.3.1 Similar Project

Orange NXT

Orange NXT is a 100 per cent digital banking service which provides services like bank instant account opening, user-friendly design, impressive interest rates in savings accounts, finance management, expense tracker etc. which was operated by Laxmi Sunrise Bank.



Figure 4 Orange NXT

Nbank

The Nbank is the digital banking service provided by the Nabil bank where the customer can perform different types of banking-related tasks fund transfer, account opening, bill payment, card services, etc. without being required to visit the nearby bank branch. It offers a video KYC feature for verification of the customer identity while opening a bank account.

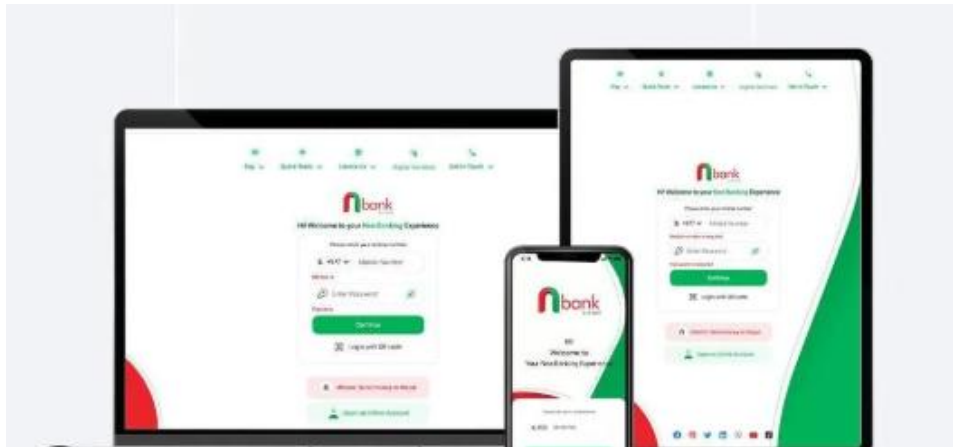


Figure 5 Nbank

NIC ASIA Mobank

It is a mobile banking app of NIC Asia bank developed by fl Soft which provides banking features like account details, fund transfer, and account opening with video kyc. load application.

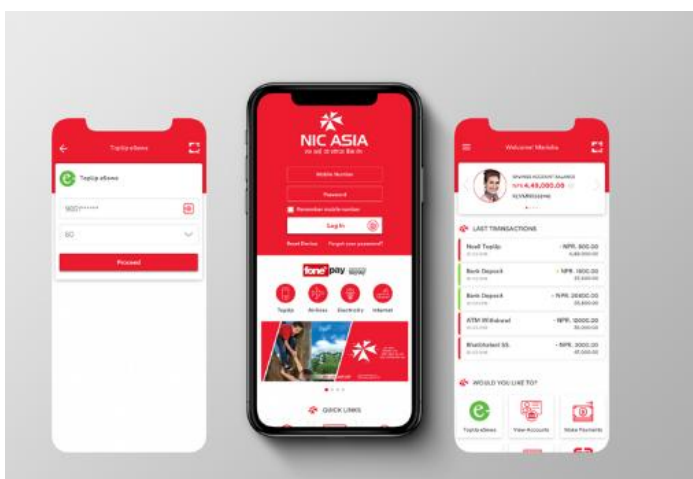


Figure 6 NIC asia mobank

2.3.2 Comparison of Similar Project

Features	Sajilo e-Bank	Orange nxt	Nbank	NIC Asia mobank
User authentication	✓	✓	✓	✓
OCR form automation	✓	✓	✗	✗
User location tracking	✓	✓	✓	✗
Facial Recognition	✓	✓	✗	✗
Liveness detection	✓	✗	✗	✗

Table 1 Similar system comparison

Chapter 3: Development

3.1 Considered Methodologies

3.1.1 Iterative waterfall model

Every stage of the SDLC includes a feedback system in the iterative waterfall model, something the conventional waterfall model failed to provide. (Kaur, 2015). It resolves the waterfall model's flaw, which was that every step was finished. Once the modification is made, we are unable to go back. It combines the flexibility of iterative design with a sequential approach to improve improvements at every stage of the SDLC by offering feedback. (geeksforgeeks, 2024).

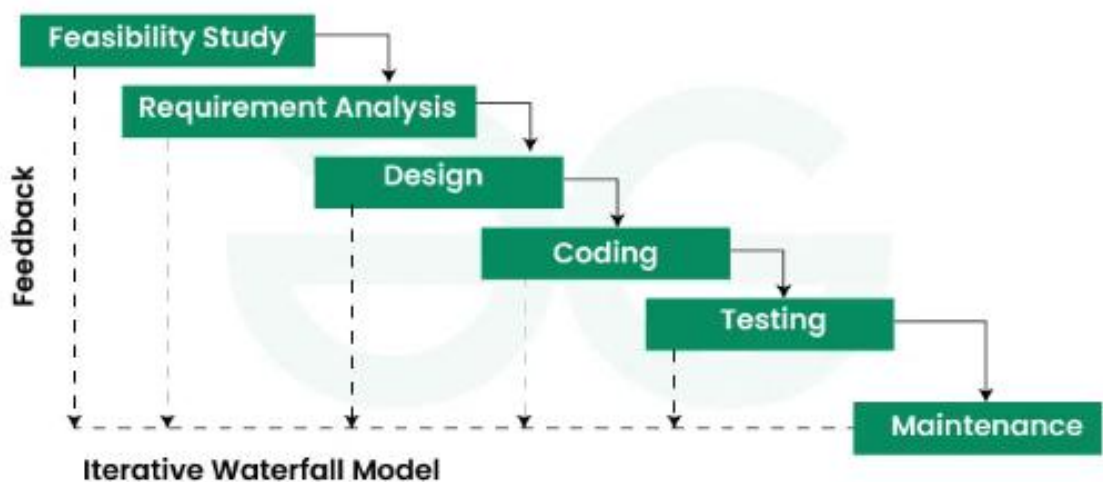


Figure 7 Iterative waterfall model (geeksforgeeks, 2024)

3.1.2 Agile Scrum Methodology

Agile defines the capability that enables quick adaptation to change while tackling difficult situations and understanding uncertain conditions to make effective decisions. (Trivedi, 2021). Project management through agile methodology performs work organization by dividing tasks into various flexible stages known as sprints.

As an agile project management framework, Scrum provides teams with a structured collection of values and practices which enable them to oversee their shared objectives while delivering cost-effective projects precisely and efficiently. The origin of "Scrum" comes from rugby, meaning team players rapidly improve their approaches while every member has assigned responsibilities.(Ozelm Senvar, 2020).



Figure 8 Scrum Process (Bhaskar, 2024)

3.2 Selected Methodology

Incremental Model

The incremental model delivers software through an approach that builds applications by small, manageable parts. The entire product development requires building each part from design through testing and implementation until completion (Deepti Singh, 2015). The increment module functionality will be delivered during each iteration before its integration into the completed product. (Albahary, 2020)

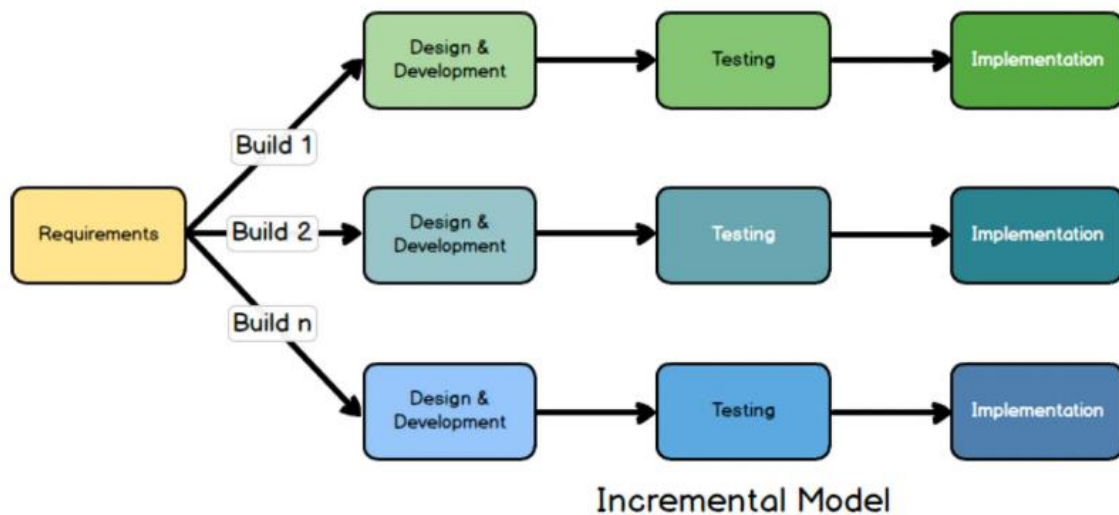


Figure 9 Incremental Model (More, 2023)

3.3 PHASE OF METHODOLOGY

The first step involves identifying main project features and objectives alongside their objectives before starting requirement collection. The system's requirements produce System analysis and use case diagrams as well as various documentation items. Each feature becomes smaller components which complete design and development testing and integration before the end system concludes its work.

Users who wish to use the system must first create their accounts through database authentication, accompanied by webpage validation, before entering personal information. This sequence constitutes the project initiation.

The second increment serves to automate form processing and OCR-based document data reading functions. The system requires advanced testing together with documentation verification for proper image identification before validating the correct document.

Supplementing KYC procedures with face recording capabilities through system accessible cameras forms the major task of the third increment. Face recognition technology performs liveness detection through this system to confirm that a real person is available. The system issues email alerts upon recognition completion.

The fourth increment introduces an admin control panel that presents user information with KYC verification status as well as permission to edit user database entries.

Before deployment the system receives final preparations that involve producing system documentation and manual creation and performing ongoing progress assessment.

3.4 SURVEY RESULTS

3.4.1 POST-SURVEY RESULTS

The result of survey is given at the appendix section [POST SURVEY RESULTS](#)

3.5 REQUIREMENT ANALYSIS

The description on requirements analysis is mentioned in appendix [SRS document](#)

3.6 DESIGN

The design conducted during the project is shown in appendix [design](#)

3.7 Implementation

3.7.1 Increment 1 (User Authentication and User Location Tracking)

3.7.1.1 Login and Sign-up page

```
def register_user(request) :
    if request.method == "POST":
        email = request.POST.get('email')
        password = request.POST.get('password')
        re_type_password = request.POST.get('re-password')
        phone_number = request.POST.get('phone')
        hashed_password = UserAuthentication.hash_password(password)
        try:
            # Validate email format using the provided regex
            email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
            if not re.match(email_regex, email):
                return render(request, "register.html", {"error": "Invalid email format.", "email": email, "phone": phone_number})

            # Check if the email already exists
            if UserAuthentication.objects.filter(email=email).exists():
                return render(request, "register.html", {"error": "Email already exists.", "email": email, "phone": phone_number})

            # Validate phone number format
            if not re.match(r'^\d{10}$', phone_number):
                return render(request, "register.html", {"error": "Phone number must be 10 digits.", "email": email, "phone": phone_number})

            # Check if the phone number already exists
            if UserAuthentication.objects.filter(phone_number=phone_number).exists():
                return render(request, "register.html", {"error": "Phone number already exists.", "email": email, "phone": phone_number})

            # Check if passwords match
            if password != re_type_password:
                return render(request, "register.html", {"error": "Passwords do not match.", "email": email, "phone": phone_number})

        except Exception as e:
            return render(request, "register.html", {"error": str(e), "email": email, "phone": phone_number})

        user = UserAuthentication.objects.create(
            email=email,
            password=hashed_password,
            phone_number=phone_number,
            is_email_verified=False
        )
        send_verification_email(request, user)
        return render(request, "register.html", {"message": "Verification email sent. Please check your inbox."})

    # Creation of the UserAuthentication model object
```

Figure 10 Register user function code

The `register_user` view function is used to render the registration page and to handle form submissions: of the user entered data when accessed via GET, it shows `register.html`, when accessed through POST request the function extract the email, password, confirmation password, and phone number from the submitted data, then runs a series of condition using regular expressions from `re` module to verify the email format and ten-digit phone number, ensuring neither is already registered, and confirming that the two password entries match—returning the form with an error message if any validation fails. After successful validation the system generates a password hash while creating a new `User Authentication` user object entry in the database and sends a verification email which then returns the registration page with a success confirmation and inbox check instruction. After the account is verified, the email verified status is set to true, then user can log into the system via login page using their credentials.

```
def login_user(request):
    if request.method == "POST":
        email = request.POST.get('email')
        password = request.POST.get('password')

        try:
            user = UserAuthentication.objects.get(email=email)
        except UserAuthentication.DoesNotExist:
            return render(request, "login.html", {"error": "Email does not exist!"})

        if user.is_email_verified == 0:
            return render(request, "login.html", {"error": "Please verify your email before logging in."})

        if check_password(password, user.password):
            # Store user ID in session

            request.session['user_id'] = user.user_id
            request.session.modified = True
            request.session.save()
            user.login_count += 1
            print("SESSION DATA:", request.session.items())
            user.save()

            return redirect('document')
        else:
            return render(request, "login.html", {"error": "Password is incorrect!"})

    return render(request, 'login.html')
```

Figure 11 User login logic code

The login_user view function is used to handle the post request which gather the user email and password from the form submission, then check if the credential matches in the database, then, verifies if the email status is verified or not then. If the credential with verified email, then logs into the system while also updating the login count by 1 for tracking the user activity.

3.7.1.2 Email verification code

```
def send_verification_email(request, user):
    token = default_token_generator.make_token(user)
    uid = urlsafe_base64_encode(force_bytes(user.pk))

    verification_url = request.build_absolute_uri(reverse('verify_email', kwargs={'uidb64': uid, 'token': token}))

    subject = "Sajilo Ebank, Please Verify Your Email"
    message = f"Click the link to verify your email: {verification_url}"
    sender_email = settings.EMAIL_HOST_USER
    send_mail(subject, message, sender_email, [user.email], fail_silently=False)

def verify_email(request, uidb64, token):
    try:
        uid = force_str(urlsafe_base64_decode(uidb64))

        user = UserAuthentication.objects.get(pk=uid)
        # Checking if the user email is already verified or not
        if user.is_email_verified:
            return HttpResponse("Your email is already verified.", status=200)

        if default_token_generator.check_token(user, token):
            if request.method == "POST":
                user.is_email_verified = True
                user.save()
                return HttpResponse("Email verified successfully! You can now log in.", status=200)

            return render(request, 'emailverification.html', {'email': user.email})

    except (TypeError, ValueError, OverflowError, UserAuthentication.DoesNotExist):
        return HttpResponse("Invalid verification link.", status=400)

    return HttpResponse("Invalid or expired token.", status=400)
```

Figure 12 Email verification logic code

After the creation of the account through register page, users' needs verify your identity by sending an email link to ensure user proper ownership. protected user ID along with confirmation token is sends it to user email with a notification to verify your email address. After clicking the link system verifies your ID by decoding it before validating both the token and ensuring its originality and status as unused and unexpired token. Upon successful verification the system marks user account as verified then displays a success page.

3.7.1.3 Reset password code

```

signer = TimestampSigner()

def generate_reset_token(user_id):
    return signer.sign(user_id)

# Validate and extract user ID from token
def verify_reset_token(token, max_age=1800): # 30 minutes expiry
    try:
        user_id = signer.unsign(token, max_age=max_age) # Returns user ID if valid
        return user_id
    except (BadSignature, SignatureExpired):
        return None # Invalid or expired token

def send_reset_email(request):
    if request.method == "POST":
        email = request.POST.get('email')

        try:
            user = UserAuthentication.objects.get(email=email)

            if not user.is_email_verified:
                return HttpResponse("Email is not verified.", status=400)

            reset_token = generate_reset_token(user.user_id)
            reset_link = f"http://127.0.0.1:8080/reset-password/{reset_token}/"

            send_mail(
                subject="Password Reset Request",
                message=f"Click the link below to reset your password:\n(reset_link)",
                from_email=settings.EMAIL_HOST_USER,
                recipient_list=[email],
                fail_silently=False,
            )

            return HttpResponse("Please check your email, password reset link has been sent to your email.")

        except UserAuthentication.DoesNotExist:
            return HttpResponse("Email not found.", status=400)

    return render(request, "reset.html")

def reset_password(request, token):
    user_id = verify_reset_token(token)

    if not user_id:
        return HttpResponse("Reset link expired or invalid.", status=400)

    if request.method == "POST":
        new_password = request.POST.get("new_password")
        confirm_password = request.POST.get("confirm_password")

        if new_password != confirm_password:
            return HttpResponse("Passwords do not match.", status=400)

        try:
            user = UserAuthentication.objects.get(user_id=user_id)
            user.password = UserAuthentication.hash_password(new_password)
            user.save()
            return HttpResponse("Password has been reset successfully.")
        except UserAuthentication.DoesNotExist:
            return HttpResponse("User not found.", status=400)

    return render(request, "password_reset_form.html", {"token": token})

```

Figure 13 Reset password logic code

If the user forgot their password, then, through the reset password page, the user email address is entered, the system checks if the email address exists in the database or not then, it sends the reset password link to the user entered email address through send reset email function. The reset password form link is provided to the user to change their password. After the form submission the link is expired.

3.7.1.4 User authentication model.py code

```
1 from django.db import models
2 from django.contrib.auth.hashers import make_password
3 from django.utils.timezone import now
4
5
6
7 # Creation of the model for user authentication
8 class UserAuthentication(models.Model):
9     # Primary key of the model
10     user_id = models.AutoField(primary_key=True)
11     # Email of the user
12     email = models.EmailField(max_length=50, unique=True)
13     # password for authentication
14     password = models.CharField(max_length=128)
15     # phone number of the user
16     phone_number = models.CharField(max_length=15, blank=True, null=True)
17     # Account creation date
18     account_created_date = models.DateTimeField(auto_now_add=True)
19     # Email verified status
20     is_email_verified = models.BooleanField(default=False)
21     is_document_valid = models.BooleanField(default=False)
22     # Login count of the user
23     login_count = models.IntegerField(default=0)
24
25     last_login = models.DateTimeField(default=now)
26
27     #Defining the custom table name in database for UserAuthentication Model
28     class Meta:
29         db_table = 'user_authentication_detail'
30
31     # Hashing the user password for safety
32     def hash_password(password):
33         return make_password(password)
34
35     # returns the email of the user
36     def get_email_field_name(self):
37         return 'email'
```

Figure 14 Model.py code for user authentication

This model contains the required attribute fields for performing user authentication, it tracks user account login count, login time, email and document validation status, and account created date which can be used for further analysis. The credentials are safe using hashing mechanisms for the password. The unique id to define each user is stated in this model.

3.7.1.5 User authentication model migration

The model migration keeps the track of the changes such as adding, deleting, updating the model field in the model.py file. As per the requirements we can track back to the older migration by rolling back to the previous state of the model.

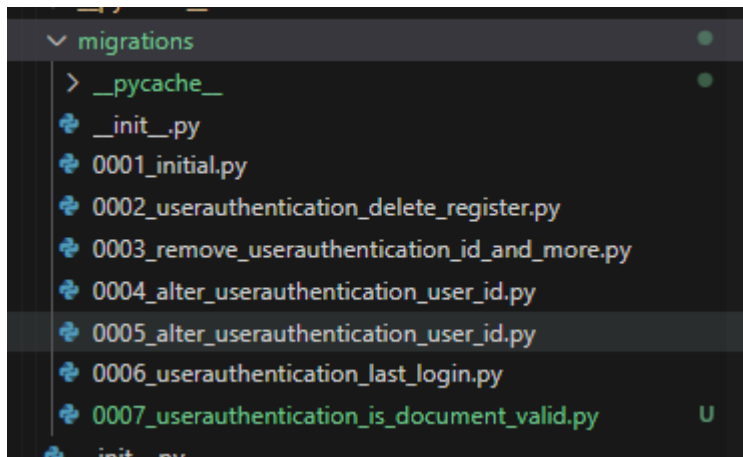


Figure 15 User authentication module model migration

3.7.1.5 User authentication model test cases code

This is the snippet of the unit test case code for different test cases using pytest.

```
def create_test_user(db):
    """Fixture to create a test user."""
    return UserAuthentication.objects.create(
        email="testuser@example.com",
        password=UserAuthentication.hash_password("password123"),
        phone_number="1234567890",
        is_email_verified=False
    )

@pytest.fixture
def register_url():
    """Fixture to provide the register URL."""
    return reverse('register')

#complete
# Testing registration with a valid phone number
@pytest.mark.django_db
def test_register_user_valid_phone(client, register_url):
    """Testing registration with a valid phone number."""
    data = {
        "email": "user@example.com",
        "password": "password123",
        "re-password": "password123",
        "phone": "0987654321" # Valid phone number with 10 digits
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    assert UserAuthentication.objects.filter(phone_number="0987654321").exists()

#complete
# Testing registration with an invalid phone number
@pytest.mark.django_db
def test_register_user_invalid_phone(client, register_url):
    """Test registration with an invalid phone number."""
    data = {
        "email": "user@example.com",
        "password": "password123",
        "re-password": "password123",
        "phone": "12345" # Invalid phone number which is (less than 10 digits)
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    # ensuring the error message is returned
    assert "Phone number must be 10 digits." in response.content.decode()
    # Ensuring the user is not created in the database
    assert not UserAuthentication.objects.filter(phone_number="12345").exists()
```

Figure 16 User authentication module unit test cases code

3.7.1.5 User location tracking

```
from django.shortcuts import render, redirect

from .models import UserLocation

def get_location_details(request):
    if request.method == "POST":
        # Handle saving location details
        latitude = request.POST.get("latitude")
        longitude = request.POST.get("longitude")

        if latitude and longitude:
            UserLocation.objects.create(latitude=latitude, longitude=longitude)
            return redirect("get_location_details")

    # For GET requests, simply show the form with the last saved location
    saved_location = UserLocation.objects.last()
    return render(request, "location.html", {"saved_location": saved_location})
```

Figure 17 User location tracking view.py

This code of view.py functions as both data processor and destination page because it retrieves POST request data latitude and longitude fields to create or save UserLocation entries before redirecting users back to this view's page. With a standard GET request this view retrieves the latest UserLocation record while loading location.html and makes it available as saved_location for the template to display the stored coordinates. The view functions both as an interface for data entry and for displaying data in a concise format. It also renders the location page and displays the user saved location.

```

function updateFormFields(lat, lng) {
    document.getElementById("latitude").value = lat;
    document.getElementById("longitude").value = lng;
}

function initializeMap(lat, lng) {
    map = L.map('map').setView([lat, lng], 15);

    // Add OpenStreetMap tiles
    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    }).addTo(map);

    // Add marker
    marker = L.marker([lat, lng], { draggable: true }).addTo(map);

    // Update form when marker is dragged
    marker.on('dragend', function (e) {
        const newLatLng = e.target.getLatLng();
        updateFormFields(newLatLng.lat, newLatLng.lng);
    });

    // Update marker and form when user clicks on the map
    map.on('click', function (e) {
        const clickedLat = e.latlng.lat;
        const clickedLng = e.latlng.lng;
    });
}

```

Figure 18 User location tracking code

```

<html lang="en">
<body>
<script>
function handleError(error) {
    // Handle error
}

const geoOptions = {
    enableHighAccuracy: true,
    timeout: 10000,
    maximumAge: 0
};

window.onload = function () {
    if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(updateLocation, handleError, geoOptions);
    } else {
        alert("Geolocation is not supported by your browser.");
    }
};

document.getElementById("trackLocationBtn").addEventListener("click", function () {
    const userConsent = confirm("This feature requires access to your location. Do you want to allow location tracking?");
    if (!userConsent) {
        alert("Location tracking was denied. You can manually select your location on the map.");
        return;
    }

    const loadingMessage = document.getElementById("loadingMessage");
    loadingMessage.style.display = "block";

    if (navigator.geolocation) {

```

Figure 19 User location tracking code 2

The using JavaScript geolocation API the user location is tracked based on IP address which sometime show incorrect location. The leaflet.js with OpenStreetMap is used to produce a map interface along with search location functionality. When user selects the location in the map its detail like latitude and longitude are store in the system.

```
webapp > user_location > models.py > ...
1  from django.db import models
2
3  from user_authentication.models import UserAuthentication # Import your custom auth model
4
5  class UserLocation(models.Model):
6      user = models.OneToOneField(UserAuthentication, on_delete=models.CASCADE, primary_key=True)
7      latitude = models.FloatField(blank=True, null=True)
8      longitude = models.FloatField(blank=True, null=True)
9
10     timestamp = models.DateTimeField(auto_now=True)
11
12     class Meta:
13         db_table = 'user_location_detail'
14
```

Figure 20 User location model.py code

3.7.2 Increment 2 OCR Implementation

3.7.2.1 Document Upload View.py code

The below code is used to handle the user uploaded document images of both sides, creating a folder to store the user data and saving the URL path of the document in the database for further processing. The uploaded data is passed to the fine-tuned yolo model for validating if the uploaded data by the user is valid or not. The document validation works by comparing the confidence score generated by the yolo model to the specified criteria.

```
def document_upload(request):
    if request.session.get('user_id') is None:
        return redirect('login')

    try:
        user_id = request.session.get('user_id')
        user = UserAuthentication.objects.get(user_id=user_id)
        user_document = UserDocument.objects.filter(user=user).first()
    except UserAuthentication.DoesNotExist:
        return redirect('login')

    front_side_url = user_document.front_side_url if user_document else None
    back_side_url = user_document.back_side_url if user_document else None

    if request.method == 'POST':
        front_side_document = request.FILES.get('front_side_document')
        back_side_document = request.FILES.get('back_side_document')

        if front_side_document and back_side_document:
            # Save uploaded images in the MEDIA_ROOT directory
            timestamp = datetime.now().strftime('%Y%m%d%H%M%S') # Format: YYYYMMDDHHMMSS
            front_side_filename = f'{user_id}_front_{timestamp}_{front_side_document.name}'
            back_side_filename = f'{user_id}_back_{timestamp}_{back_side_document.name}'

            front_side_path = os.path.join(settings.MEDIA_ROOT, 'uploads', 'front', front_side_filename)
            back_side_path = os.path.join(settings.MEDIA_ROOT, 'uploads', 'back', back_side_filename)

            os.makedirs(os.path.dirname(front_side_path), exist_ok=True)
            os.makedirs(os.path.dirname(back_side_path), exist_ok=True)

            with open(front_side_path, 'wb+') as destination:
                for chunk in front_side_document.chunks():
                    destination.write(chunk)
            with open(back_side_path, 'wb+') as destination:
                for chunk in back_side_document.chunks():
                    destination.write(chunk)

            # Generate URLs for the saved files
            front_side_url = f'{settings.MEDIA_URL}uploads/front/{front_side_filename}'
            back_side_url = f'{settings.MEDIA_URL}uploads/back/{back_side_filename}'

            # Save the URLs in the UserDocument model
            user_document, created = UserDocument.objects.get_or_create(user=user)
            user_document.front_side_url = front_side_url
            user_document.back_side_url = back_side_url
            user_document.save()

            # YOLO Model Integration
            results_dir = os.path.join(settings.MEDIA_ROOT, 'results')
            os.makedirs(results_dir, exist_ok=True)

            # Paths to YOLO models
            front_model_path = 'E:\\Sajilo E-bank\\runs-front\\detect\\train\\weights\\best.pt'
            back_model_path = 'E:\\Sajilo E-bank\\runs-back\\detect\\train\\weights\\best.pt'

            # Run YOLO model on front and back images
            front_result_path, front_confidence, is_front_side = yolo_model_test(front_model_path, front_side_path, results_dir)
            back_result_path, back_confidence, is_back_side = yolo_model_test(back_model_path, back_side_path, results_dir, is_back_side=True)

            # Generate messages based on validity
            if is_front_side and is_back_side:
                request.session['success_message'] = "Both the front-side and back-side documents are valid!"
                # Set is_document_valid to True
```

Figure 21 Document upload view.py code

3.7.2.2 Data annotation using Robo flow (Front side of the document)

Around 170 Front side dataset of the document was available in the robo flow and annotation was carried out to label the classes which is required for the document validation. In total 5 classes were labeled. printed logo, stamped logo, frontside, citizenship title, and silver patch so, that the confidence score of this class can be prediction using pre trained model yolo for document validation process.



Figure 22 Data annotation using robo flow for front side of document

The data yaml file is the configuration file which tells the pretrained yolo model the data path of the images of the dataset to be fine-tuned along with the number of labels, labels name, training, validation and testing folder path. It is used when training the yolo model on our own dataset.

```
front_side_dataset > ! data.yaml
1 path: E:\Sajilo E-bank\front_side_dataset
2 train: ../train/images
3 val: ../valid/images
4 test: ../test/images
5
6 nc: 5
7 names: ['Printed_logo', 'Silver_patch', 'Stamped_logo', 'citizenship title', 'frontside']
8
9 roboflow:
10 workspace: backside-citizenship
11 project: frontdataset-h7jol
12 version: 3
13 license: Public Domain
14 url: https://universe.roboflow.com/backside-citizenship/frontdataset-h7jol/dataset/3
```

Figure 23 Data YAML file for front side

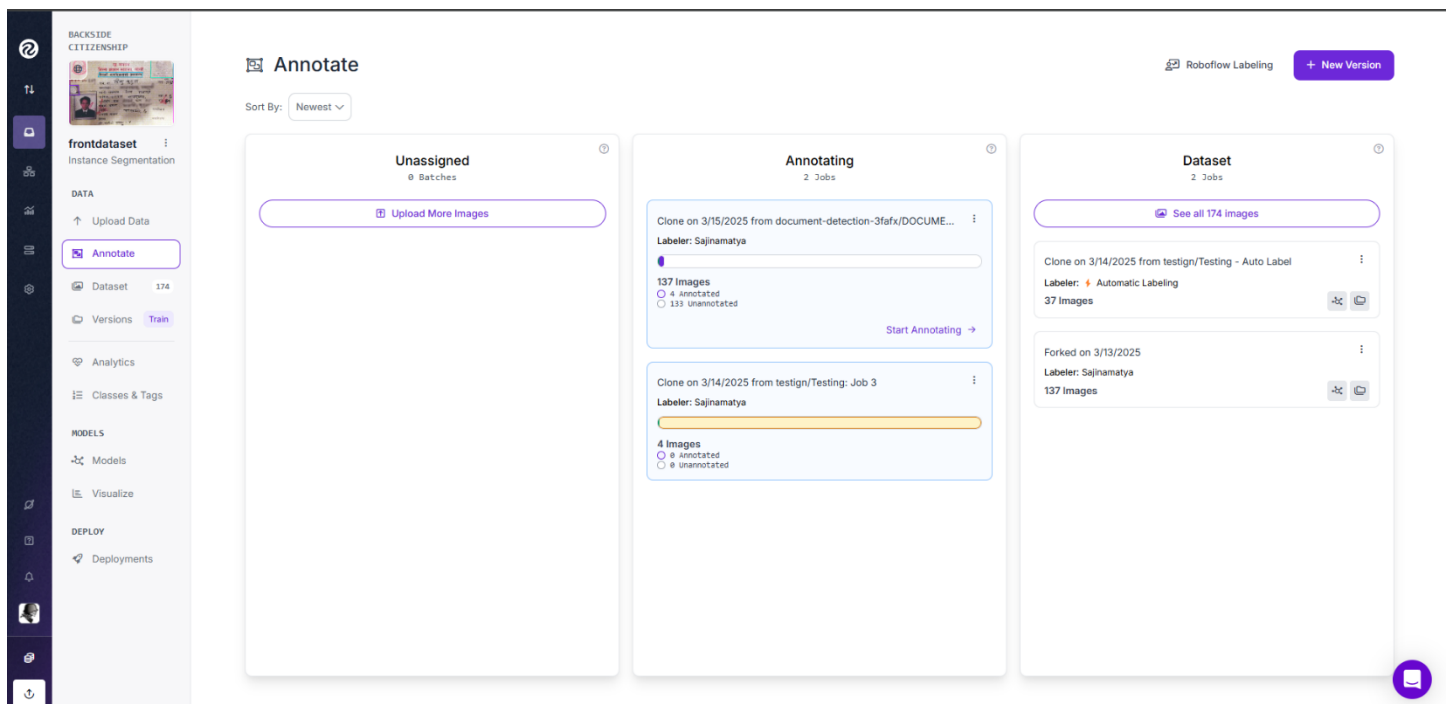


Figure 24 Data annotation result for front side

3.7.2.3 Data annotation using robo flow (back side of the document)

Around 140 back side dataset of the document was available in the robo flow and annotation was carried out to label the classes which is required for the document validation. In total 8 classes 'DOB', 'backside', 'citizenship no', 'fingerprint', 'full name', 'gender', 'permanent address', 'place of birth' so, that the confidence score of class backside and fingerprint can be predicted using pre trained model yolo for document validation process.

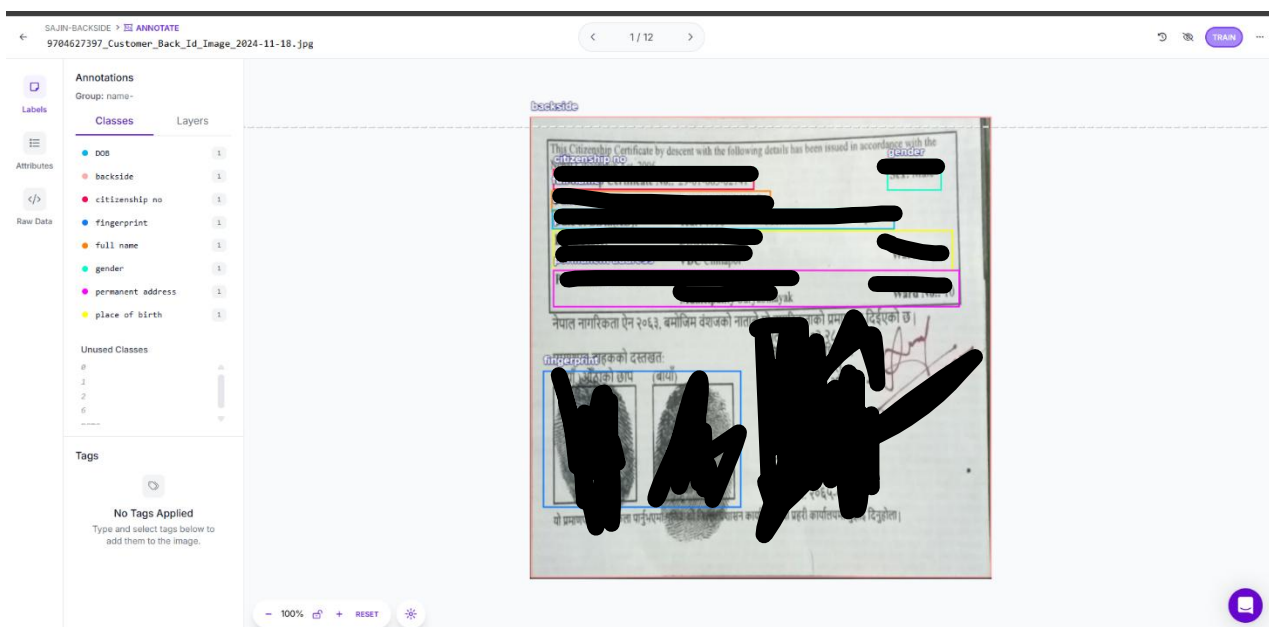


Figure 25 Data Annotation for Back side of the document

```
dataset-backside > ! datayaml
1 path: E:\Sajilo E-bank\dataset-backside
2 train: ../train/images
3 val: ../train/images
4
5 nc: 8
6 names: ['DOB', 'backside', 'citizenship no', 'fingerprint', 'full name', 'gender', 'permanent address', 'place of birth']
7
8 roboflow:
9   workspace: backside-citizenship
10  project: sajin-backside
11  version: 1
12  license: CC BY 4.0
13  url: https://universe.roboflow.com/backside-citizenship/sajin-backside/dataset/1
```

Figure 26 Data YAML file for backside dataset of document

3.7.2.6 Yolo model fine-tuning code for document validation

```

from ultralytics import YOLO
import os

def train_yolo_model(data_path, model_path, epochs, imgsize, batch_size, export_dir="document identification/models"):
    """
    Training of the yolo model on the front images dataset of citizenship to identify if the images is citizenship or not by classifying the images on the basis of classes such as stamped logo, silver patch etc.

    Args:
        data_path (str): Path to the data.yaml file.
        model_path (str): Path to the pre-trained YOLO model
        epochs (int): Number of training epochs.
        imgsize (int): Image size for training.
        batch_size (int): Batch size for training.
        export_dir (str): Directory where the model will be saved after training.
    """
    # Initializing the YOLO model for document detection
    model = YOLO(model_path)

    # Training the model on the dataset with proper labels and bounding boxes
    model.train(data=data_path, epochs=epochs, imgsz=imgsize, batch=batch_size)

    # Model validation
    results = model.val()
    print(results)

    # creating the dir if the path do not exist
    if not os.path.exists(export_dir):
        os.makedirs(export_dir)

    model.export(format='onnx', imgsz=imgsize, half=False, simplify=True, opset=11, save_dir=export_dir)
    export_path = os.path.join(export_dir, "documentdetection.onnx")

    print(f"Model exported successfully to: {export_path}")

if __name__ == "__main__":
    data_path = "E:/Sajilo E-bank/front_side_dataset/data.yaml"
    model_path = "yolov8n.pt"
    epochs = 50 # epochs for training
    imgsz = 640 # Image size for training
    batch_size = 8 # batch size for training

    train_yolo_model(data_path, model_path, epochs, imgsz, batch_size)

```

Figure 27 Yolo model fine tuning code

This code is for fine-tuning to the labeled citizenship documents by loading yolov8n.pt as the pre-trained model to detect stamped logos, silver patches, citizenship front side, etc based on paths and class definitions within data.yaml. The Ultralytics library is used so that it performs input resolution resizing and optimization tasks and backpropagation computations based on the input epochs which is set to 50 along with imgsz value which is the image size and batch size of 8 for each epoch. The system carries out validation step after training completion to present validation result, confusion matrix results before it converts the trained weights into an ONNX file for further testing and deployment of the model. Same code can be used to fine tune for the both front side model and back side model by changing the data path which is data.yaml for the particular dataset to fine tune.

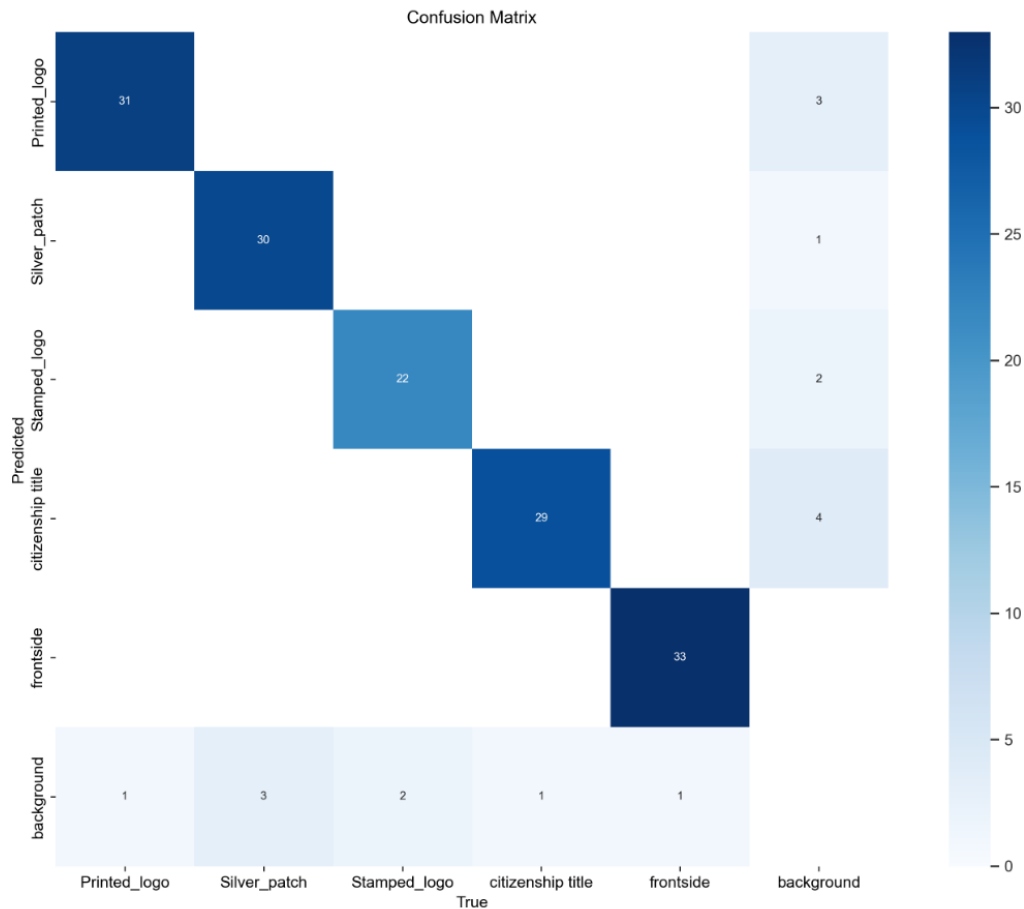


Figure 28 Confusion matrix of front side model

Here, the confusion matrix displays performance data for a multi-class classification system which detects Printed_logo, Silver_patch, Stamped_logo, and citizenship title and frontside and background part of citizenship documents. Among the detected classes the model displayed best accuracy in recognizing Printed_logo together with Silver_patch and frontside because most predictions lie along the diagonal line of the matrix which represent better prediction.

A total of 31 out of 34 examples of Printed_logo received correct classification and only 3 ended up being classified under background. Similarly, Silver_patch and frontside show high accuracy with 30 and 33 correct predictions respectively. The model struggle on the prediction of stamped_logo and citizenship title classes.

3.7.2.7 Testing the yolo model for document validation

```
def detect_front_side(confidence_scores):
    frontside_class_id = 4 # Assuming 'frontside' is the 5th class (index 4)
    frontside_confidence = [conf for cls, conf in confidence_scores if cls == frontside_class_id]
    other_classes_confidence = [conf for cls, conf in confidence_scores if cls != frontside_class_id]

    if frontside_confidence and max(frontside_confidence) > 0.95:
        if all(conf > 0.80 for conf in other_classes_confidence):
            return True
    return False

def detect_back_side(confidence_scores):
    backside_class_id = 1 # 'backside' is the 2nd class (index 1)
    fingerprint_class_id = 3 # 'fingerprint' is the 4th class (index 3)

    backside_confidence = [conf for cls, conf in confidence_scores if cls == backside_class_id]
    fingerprint_confidence = [conf for cls, conf in confidence_scores if cls == fingerprint_class_id]

    if backside_confidence and max(backside_confidence) > 0.90:
        if fingerprint_confidence and max(fingerprint_confidence) > 0.85:
            return True
    return False
```

Figure 29 Testing Yolo model for document validation

For validating whether the uploaded document is citizenship or not, in case of front side if the confidence score of the frontside class is greater than 0.90 and confidence score of stamped logo, printed logo and silver patch is greater than 0.80 then, it is classified as a front side document. In case of back side if the confidence score of the class “backside” is greater than 0.90 and confidence score of the class fingerprint is greater than 0.85 then the document is classified as back side of citizenship documents.

3.7.2.8 Extraction of face region from the front side of the document

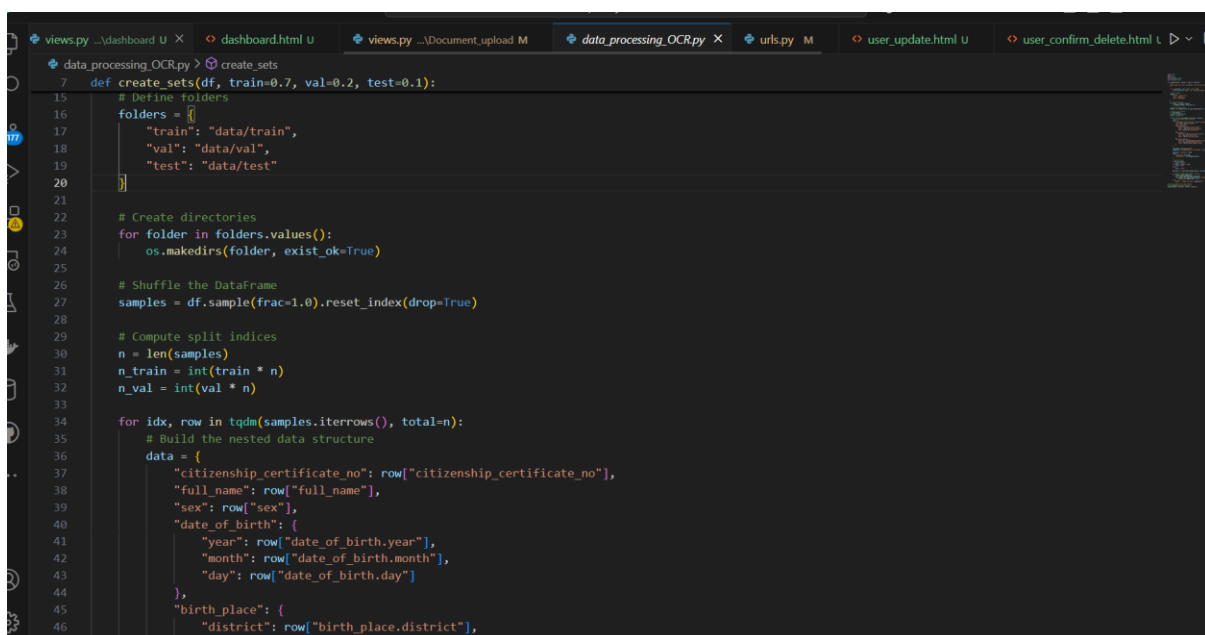
For facial verification purposes from, front side of the document the face region is detected using mtcnn face detection library, along with the region of the face region was cropped using open cv then, save to the path with folder name face in naming order user id with the timestamp and its URLs is stored in the database which can be used to extract the face for facial recognition task.

```
def detect_and_save_faces(image_path, user_id):  
    # Initialize the MTCNN detector  
    detector = MTCNN()  
  
    # Load the image  
    image = cv2.imread(image_path)  
  
    # Detect faces  
    faces = detector.detect_faces(image)  
    face_urls = []  
  
    # Extract and save faces  
    for i, face in enumerate(faces):  
        x, y, w, h = face['box']  
        face_image = image[y:y+h+3, x:x+w+3]  
        timestamp = datetime.now().strftime('%Y%m%d%H%M%S') # Format: YYYYMMDDHHMMSS  
        face_filename = f"{user_id}_face_{timestamp}_{i}.jpg"  
        face_path = os.path.join(settings.MEDIA_ROOT, 'uploads', 'faces', face_filename)  
        os.makedirs(os.path.dirname(face_path), exist_ok=True)  
        cv2.imwrite(face_path, face_image)  
        face_url = f"{settings.MEDIA_URL}uploads/faces/{face_filename}"  
        face_urls.append(face_url)  
  
    return face_urls
```

Figure 30 face detection and crop of the document frontside

3.7.2.9 OCR donut fine tuning

The code does dataset preprocessing tool for OCR jobs that separates citizenship information together with image files into distinct training and validation and test divisions. The code firstly starts by processing a CSV which includes the full name together with date of birth information and address data alongside the associated image file names for each record. The data processing system applies 70% training data and 20% validation data and 10% testing data. The code generates appropriate data subfolders (train and test and Val) upon each split and these directories are made only when they are absent. This code then creates JSON objects with structured data structures containing fields for birth date and addresses for each record. The code verifies image presence in the ocr-back/images directory before it performs both image copying operations and metadata.jsonl file entry in relevant split directories. This file features two parts in each line where image filenames are followed by JSON ground truth data. The script generates a warning message when an image file is absent.

The image shows a screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'views.py .../dashboard U', 'dashboard.html U', 'views.py .../Document_upload M', 'data_processing_OCR.py X', 'urls.py M', 'user_update.html U', and 'user_confirm_delete.html U'. The active tab is 'data_processing_OCR.py'. The code is a Python function named 'create_sets' that takes a DataFrame 'df', 'train=0.7', 'val=0.2', and 'test=0.1' as arguments. It defines a dictionary 'folders' with keys 'train', 'val', and 'test' pointing to 'data/train', 'data/val', and 'data/test' respectively. It then creates these directories using 'os.makedirs'. The code shuffles the DataFrame using 'df.sample(frac=1.0).reset_index(drop=True)'. It calculates the number of samples 'n' and splits them into 'n_train', 'n_val', and 'n_test' based on the provided percentages. It then iterates over the samples using 'tqdm(samples.iterrows(), total=n)'. For each row, it builds a nested JSON object 'data' containing fields like 'citizenship_certificate_no', 'full_name', 'sex', 'date_of_birth' (with year, month, and day), and 'birth_place' (with district). The code is well-commented and uses standard Python syntax for data manipulation and file operations.

```
7 def create_sets(df, train=0.7, val=0.2, test=0.1):
15     # Define folders
16     folders = {
17         "train": "data/train",
18         "val": "data/val",
19         "test": "data/test"
20     }
21
22     # Create directories
23     for folder in folders.values():
24         os.makedirs(folder, exist_ok=True)
25
26     # Shuffle the DataFrame
27     samples = df.sample(frac=1.0).reset_index(drop=True)
28
29     # Compute split indices
30     n = len(samples)
31     n_train = int(train * n)
32     n_val = int(val * n)
33
34     for idx, row in tqdm(samples.iterrows(), total=n):
35         # Build the nested data structure
36         data = {
37             "citizenship_certificate_no": row["citizenship_certificate_no"],
38             "full_name": row["full_name"],
39             "sex": row["sex"],
40             "date_of_birth": {
41                 "year": row["date_of_birth.year"],
42                 "month": row["date_of_birth.month"],
43                 "day": row["date_of_birth.day"]
44             },
45             "birth_place": {
46                 "district": row["birth_place.district"],
```

Figure 31 Code of data splitting for ocr donut along with Json data formatting


```
training_args = Seq2SeqTrainingArguments(  
    output_dir="/kaggle/working/model",  
    evaluation_strategy="steps",  
    eval_steps=200,  
    learning_rate=5e-5,  
    per_device_train_batch_size=1, # Reduced batch size  
    per_device_eval_batch_size=1, # Reduced batch size  
    gradient_accumulation_steps=4, # Reduced gradient accumulation steps  
    weight_decay=0.01,  
    save_total_limit=1,  
    num_train_epochs=3,  
    predict_with_generate=True,  
    fp16=True, # Enable mixed precision training  
    save_strategy="steps",  
    save_steps=200,  
    logging_dir="./logs",  
    logging_steps=50,  
    dataloader_num_workers=0,  
    gradient_checkpointing=True,  
    ddp_find_unused_parameters=False,  
    optim="adamw_torch",  
    max_grad_norm=0.5, # Add gradient clipping  
    report_to="none" # Disable reporting to save memory  
)  
  
# Initialize trainer  
trainer = Seq2SeqTrainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=val_dataset,  
)  
  
# Train model  
print("Starting training...")  
trainer.train()  
trainer.save_model()
```

Due to computational resource limitations, the ocr donut was fine tuned in Kaggle, this code trains the mode by configuring Seq2Seq training through Hugging Face Seq2SeqTrainer before running the actual training process for sequence-to-sequence (Seq2Seq) models. The implementation starts by establishing training conditions through Seq2SeqTrainingArguments which includes multiple improvements to optimize limited computer power usage. The training involves setting each batch size to 1 while performing gradient accumulation across 4 steps in combination with activating half precision training (fp16=True) to improve speed and memory efficiency. To conserve space and prevent the loss due to some hazard the model saves checkpoints upon step 200 but activates a single checkpoint maintenance system. The trainer system needs initialization using the model coupled with training dataset and validation dataset. The trainer.train() function launches the training process before the model gets saved into the designated output directory after full completion.

3.7.3 Increment 3

3.7.3.1 Dashboard UI

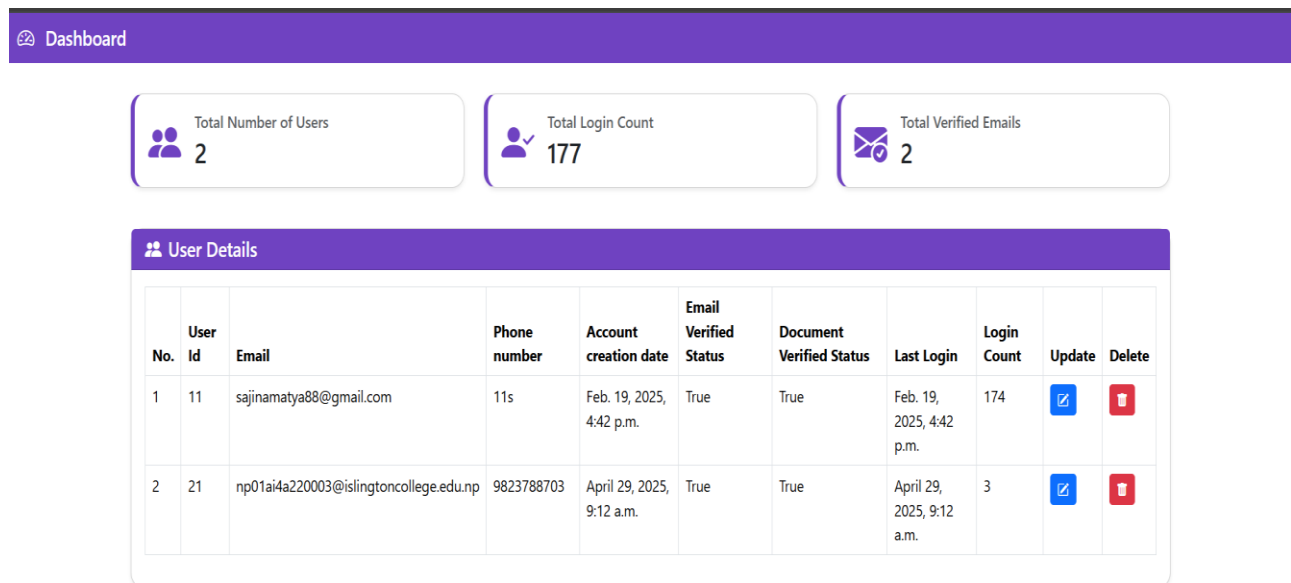


Figure 32 increment 3 dashboards

The screenshot shows a web browser window with the URL "127.0.0.1:8000/user/11/update/". The page title is "Update User 11". The form contains the following fields and controls:

- Email:** sajinamatya88@gmail.com
- Phone Number:** 11s
- Email Verified:** ☒
- Document Valid:** ☒
- Buttons:** "Save Changes" (blue) and "Cancel" (grey)

Figure 33 User update form

3.7.3.1 Dashboard logic code

```
from django.shortcuts import render
from user_authentication.models import UserAuthentication
from django.db.models import Sum
from django.shortcuts import render, get_object_or_404, redirect
from django.urls import reverse
# Create your views here.
def dashboard_view(request):
    users = UserAuthentication.objects.all()
    total_users = users.count()
    total_logins = users.aggregate(Sum('login_count'))['login_count__sum'] or 0
    total_verified_emails = users.filter(is_email_verified=True).count()
    return render(request, 'dashboard.html', {
        'users': users,
        'total_users': total_users,
        'total_logins': total_logins,
        'total_verified_emails': total_verified_emails
    })
```

Figure 34 Dashboard view.py

```
def user_update(request, user_id):
    user = get_object_or_404(UserAuthentication, pk=user_id)
    if request.method == 'POST':
        # manually pull fields from POST
        user.email = request.POST.get('email', user.email)
        user.phone_number = request.POST.get('phone_number', user.phone_number)
        user.is_email_verified = bool(request.POST.get('is_email_verified'))
        user.is_document_valid = bool(request.POST.get('is_document_valid'))
        # add any other fields you wish to update here...
        user.save()
        return redirect(reverse('dashboard'))
    return render(request, 'user_update.html', {'user': user})
```

Figure 35 user updating form view.py

```
def user_delete(request, user_id):
    user = get_object_or_404(UserAuthentication, pk=user_id)
    if request.method == 'POST':
        user.delete()
        return redirect(reverse('dashboard'))
    return render(request, 'user_confirm_delete.html', {'user': user})

def user_detail(request, user_id):
```

Figure 36 User delete function view.py

3.7.3.3 face detection/verification integration code

```
# Load FaceNet model
embedder = FaceNet()

# Mediapipe face detector
mp_face_detection = mp.solutions.face_detection
mp_drawing = mp.solutions.drawing_utils

# Cosine similarity function
def cosine_similarity(a, b):
    return np.dot(a, b) / (norm(a) * norm(b))

# Extract face using Mediapipe (returns cropped face)
def extract_face_mediapipe(frame):
    with mp_face_detection.FaceDetection(model_selection=1, min_detection_confidence=0.5) as detector:
        results = detector.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

        if results.detections:
            detection = results.detections[0]
            bbox = detection.location_data.relative_bounding_box
            h, w, _ = frame.shape
            x, y, w_box, h_box = int(bbox.xmin * w), int(bbox.ymin * h), int(bbox.width * w), int(bbox.height * h)
            x, y = max(0, x), max(0, y)
            face = frame[y:y + h_box, x:x + w_box]
            if face.size == 0:
                return None
            return cv2.resize(face, (160, 160))
    return None
```

Figure 37 Face detection using media pipe

```
# Load stored image embedding from path
def get_stored_embedding_from_path(image_path):
    try:
        img = Image.open(image_path).convert('RGB')
        img = img.resize((160, 160))
        img_np = np.array(img)
        embeddings = embedder.embeddings([img_np])
        return embeddings[0]
    except Exception as e:
        print("Error loading face image from path:", e)
        return None
```

Figure 38 Saving the embedding of the face

```
def generate_frames():
    cap = cv2.VideoCapture(0)
    stored_embedding = get_stored_embedding_from_path("E:/Sajilo E-bank/webapp/mediauploads/faces/11_face_20250430012930_0.jpg")

    try:
        while True:
            success, frame = cap.read()
            if not success:
                break

            frame = cv2.flip(frame, 1)
            face = extract_face_mediapipe(frame)

            if face is not None and stored_embedding is not None:
                live_embedding = embedder.embeddings([cv2.cvtColor(face, cv2.COLOR_BGR2RGB)])[0]
                similarity = cosine_similarity(stored_embedding, live_embedding)

                label = f"Match: {similarity:.2f}" if similarity > 0.8 else f"No Match: {similarity:.2f}"
                color = (0, 255, 0) if similarity > 0.8 else (0, 0, 255)
                cv2.putText(frame, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
            else:
                cv2.putText(frame, "No face detected", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

            _, buffer = cv2.imencode('.jpg', frame)
            frame = buffer.tobytes()

            yield (b'--frame\r\n'
                  b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

    finally:
        cap.release()
```

Figure 39 Video frame streaming code

CHAPTER 4: TESTING AND ANALYSIS

4.1 Test plan

4.1.1 UNIT TESTING and URL TESTING PLAN

Test Number	Test cases
Unit testing 1	Testing if the valid phone number is accepted by the system or not
Unit testing 2	Testing the validation of the email address
Unit testing 3	Testing if the password and re-type password are different, then an error message is displayed or not.
Unit testing 4	Testing if the password and re-type password are the same, then the user object is created or not.
Unit testing 5	Testing if the invalid email is submitted, then the error message is responded to by the system or not.
Unit testing 6	Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.
Unit testing 7	Testing if the already existing email in the database is accepted by the system or not.
Unit testing 8	Testing if the already existing phone number in the database is accepted by the system or not.
Unit testing 9	Testing if the verification link is sent to the user's email.
Unit testing 10	Testing if the user verifies their email through the verification link
Unit testing 11	Testing if the logout function works or not
Unit testing 12	Testing if the login functionality works with the correct credentials
Unit testing 13	Testing if the login functionality works or not with the unverified email
Unit testing 14	Testing if the login functionality works or not with the wrong password
Unit testing 15	Testing if the login functionality works or not with the email that does not exist in the system.
Unit testing 16	Testing if the Reset password functionality

Unit testing 17	Testing if the Reset password link is sent to the user's email
URL testing 1	Testing the URL of the register view
URL testing 2	Testing the URL of the login view
URL testing 3	Testing the URL of the logout function
URL testing 4	Testing the URL of the email verification
URL testing 5	Testing the URL of the reset password
URL testing 6	Testing the URL of the dashboard

Table 2 Test case plan, unit and URL testing

4.1.2 Test case plan, system testing

Test number	Test case
System testing 1	Testing the user Registration process
System testing 2	Testing the user login process
System testing 3	Testing the Reset Password Functionality
System testing 4	Testing the Document Upload Functionality along with user detail form
System testing 5	Testing the User location tracking Functionality
System testing 6	Testing the dashboard functionality

Table 3 Test case plan system testing

4.2 Unit Testing

4.2.1 Unit testing 1 : Testing if the valid phone number is accepted by the system or not

Test	Unit testing 1
Description	Testing if the valid phone number is accepted by the system or not
Action	The user valid phone number with 10 digit is passed and test through unit test code using pytest.
Results	The Valid phone number is accepted by the system

Table 4 Unit testing 1 : Testing if the valid phone number is accepted by the system or not

```
@pytest.mark.django_db

def test_register_user_valid_phone(client, register_url):
    """Testing registration with a valid phone number."""
    data = {
        "email": "user@example.com",
        "password": "password123",
        "re-password": "password123",
        "phone": "0987654321" # Valid phone number with 10 digits
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    assert UserAuthentication.objects.filter(phone_number="0987654321").exists()
```

Figure 40 Unit Test 1 valid phone number inspection

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_valid_phone
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 2 items / 1 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 1 deselected in 4.83s =====
(.venv) PS E:\Sajilo E-bank\webapp>
```

Figure 41 Result of unit test 1

4.2.2 Unit testing 2 : Testing the validation of the email address

Test	Unit testing 2
Description	Testing the validation of the email address
Action	The valid email is passed through the unit test case code
Results	The Valid email is accepted by the system.

Table 5 Unit testing 2 Testing the validation of the email address

```
@pytest.mark.django_db
def test_register_user_valid_email(client, register_url):
    """Test registration with a valid email format."""
    data = {
        "email": "validuser@example.com", # Valid email format
        "password": "123",
        "re-password": "123",
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    # Testing if the user is created in the database
    # Assuming the email is unique, check if the user exists in the database
    assert UserAuthentication.objects.filter(email="validuser@example.com").exists()
```

Figure 42 Unit test 2: Valid user email inspection

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_valid_email
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 3 items / 2 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 2 deselected in 5.06s =====
```

Figure 43 Unit test 2 results

4.2.3 Unit testing 3: Testing if the password and re-type password are different, then an error message is displayed or not.

Test	Unit testing 3
Description	Testing if the password and re-type password are different, then an error message is displayed or not.
Action	The different password and password are passed through the unit test code
Results	The error message is displayed by the system.

Table 6 Unit testing 3 : Testing if the password and re-type password are different, then an error message is displayed or not.

```
@pytest.mark.django_db
def test_register_user_password_mismatch(client, register_url):
    """Test registration with mismatched password and re-password."""
    data = {
        "email": "validuser@example.com",
        "password": "password123",
        "re-password": "differentpassword", # Mismatched password
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    assert "Passwords do not match." in response.context['error']
    # Ensure the user is not created in the database
    assert not UserAuthentication.objects.filter(email="validuser@example.com").exists()
```

Figure 44 Unit test 3: Password and retype password validity check

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_password_mismatch
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 4 items / 3 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 3 deselected in 4.72s =====
```

Figure 45 Results of Unit Test 3

4.2.4 Unit testing 4 : Testing if the password and re-type password are the same, then the user object is created or not.

Test	Unit testing 4
Description	Testing if the password and re-type password are the same, then the user object is created or not.
Action	The same re-type of password and password are passed through the test case function using pytest
Results	The user object was created when the same password and re-type password were entered.

Table 7 Unit testing 4 : Testing if the password and re-type password are the same, then the user object is created or not.

```
@pytest.mark.django_db
def test_register_user_same_passwords(client, register_url):
    """Test registration with matching password and re-password."""
    data = {
        "email": "user@example.com",
        "password": "123",
        "re-password": "123", # Matching passwords
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    # Ensure the user is created in the database
    assert UserAuthentication.objects.filter(email="user@example.com").exists()
```

Figure 46 Unit test 4 password and retype password with user object creation check

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_same_passwords
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 5 items / 4 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 4 deselected in 4.72s =====
```

Figure 47 Unit test 4 results

4.2.5 Unit testing 5: Testing if the invalid email is submitted, then the error message is responded to by the system or not.

Test	Unit testing 5
Description	Testing if the invalid email is submitted, then the error message is responded to by the system or not.
Action	The invalid email is passed through the test case function using pytest
Results	The error message was responded to, and the user object was not created.

Table 8 Unit testing 5 : Testing if the invalid email is submitted, then the error message is responded to by the system or not.

```
@pytest.mark.django_db
def test_register_user_invalid_email(client, register_url):
    """Test registration with an invalid email format."""
    data = {
        "email": "invalid-email", # Invalid email format
        "password": "password123",
        "re-password": "password123",
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200
    assert "Invalid email format." in response.content['error']
    assert not UserAuthentication.objects.filter(email="invalid-email").exists()
```

Figure 48 Unit test 5: Invalid email format check

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_invalid_email
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 5 items / 4 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 4 deselected in 4.91s =====
```

Figure 49 Unit test 5 results

4.2.6 Unit testing 6 : Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.

Test	Unit testing 6
Description	Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.
Action	The invalid phone number was passed to the test case function using pytest
Results	The error message was responded to, and the user object was not created.

Table 9 Unit testing 6: Testing if the invalid phone number is submitted, then the error message is responded to by the system or not.

```
@pytest.mark.django_db
def test_register_user_invalid_phone(client, register_url):
    """Test registration with an invalid phone number."""
    data = {
        "email": "user@example.com",
        "password": "password123",
        "re-password": "password123",
        "phone": "12345" # Invalid phone number which is (less than 10 digits)
    }

    response = client.post(register_url, data)
    assert response.status_code == 200
    # ensuring the error message is returned
    assert "Phone number must be 10 digits." in response.content.decode()
    # Ensuring the user is not created in the database
    assert not UserAuthentication.objects.filter(phone_number="12345").exists()
```

Figure 50 Unit test 6: user phone number validation

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_invalid_phone
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 6 items / 5 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 5 deselected in 4.75s =====
```

Figure 51 Unit test 6 results

4.2.7 Unit testing 7 : Testing if the already existing email in the database is accepted by the system or not.

Test	Unit testing 7
Description	Testing if the already existing email in the database is accepted by the system or not.
Action	The email already existed in the system is passed to the test case using pytest
Results	The error message was responded to, and the user object with an already existing email address was not created.

Table 10 Unit testing 7 : Testing if the already existing email in the database is accepted by the system or not.

```
@pytest.mark.django_db
def test_register_user_email_already_exists(client, register_url):
    """Test registration with an email that already exists."""
    UserAuthentication.objects.create( # creating a user with the same email for verification
        email="testuser123@example.com",
        password="password123",
        phone_number="0987654321"
    )
    data = {
        "email": "testuser123@example.com", # Email already exists in the database
        "password": "password123",
        "re-password": "password123",
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200

# Ensure no duplicate user is created
assert UserAuthentication.objects.filter(email="testuser123@example.com").count() == 1
```

Figure 52 Unit test 7: Email already existed check.

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_email_already_exists
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 7 items / 6 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 6 deselected in 4.82s =====
(.venv) PS E:\Sajilo E-bank\webapp>
```

Figure 53 Unit test 7 results

4.2.8 Unit testing 8 : Testing if the already existing phone number in the database is accepted by the system or not.

Test	Unit testing 8
Description	Testing if the already existing phone number in the database is accepted by the system or not.
Action	The existing phone number in the system was passed to the test case using pytest
Results	The error message was responded to, and the user object with an already existing phone number was not created.

Table 11 Unit testing 8: Testing if the already existing phone number in the database is accepted by the system or not.

```
@pytest.mark.django_db
def test_register_user_phone_already_exists(client, register_url):
    """Test registration with an email that already exists."""
    UserAuthentication.objects.create( # creating a user with the same phone number for verification
        email="testuser123@example.com",
        password="password123",
        phone_number="0987654321"
    )
    data = {
        "email": "testuser123@example.com", # phone number already exists in the database
        "password": "password123",
        "re-password": "password123",
        "phone": "0987654321"
    }
    response = client.post(register_url, data)
    assert response.status_code == 200

    # Ensure no duplicate user is created
    assert UserAuthentication.objects.filter(phone_number="0987654321").count() == 1
```

Figure 54 Unit test 8: User phone number already existed or not.

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_register_user_phone_already_exists
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 8 items / 7 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 7 deselected in 4.70s =====
```

Figure 55 Unit test 8 results

4.2.9 Unit testing 9 Testing if the verification link is sent to the user's email.

Test	Unit testing 9
Description	Testing if the verification link is sent to the user's email.
Action	The verification link was tested using pytest using token and verifying if the token is same or not
Results	The verification link was received by the user.

Table 12 Unit testing 9 Testing if the verification link is sent to the user's email.

```
@pytest.mark.django_db
def test_send_verification_email():
    """Test that the verification email is sent correctly."""
    # Step 1: Create a test user
    user = UserAuthentication.objects.create(
        email="testuser@example.com",
        password=UserAuthentication.hash_password("password123"),
        phone_number="1234567890",
        is_email_verified=False
    )

    # Step 2: Create a mock request using RequestFactory
    factory = RequestFactory()
    request = factory.get("/") # Create a GET request to the root URL

    # Step 3: Generate the token and UID
    token = default_token_generator.make_token(user)
    uid = urlsafe_base64_encode(force_bytes(user.pk))

    # Step 4: Call the send_verification_email function from the view.py
    send_verification_email(request, user)

    # Step 5: Check that an email was sent
    assert len(mail.outbox) == 1 # Ensure one email was sent
    email = mail.outbox[0]
    assert email.subject == "Sajilo Ebank, Please Verify Your Email"
    assert "Click the link to verify your email" in email.body
    assert user.email in email.to

    # Step 6: Verify the verification link in the email
    expected_link = f"http://testserver/verify-email/{uid}/{token}"
    assert expected_link in email.body
```

Figure 56 Unit test 9: Email Verification link sent

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_send_verification_email
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 9 items / 8 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 8 deselected in 4.71s =====
```

Figure 57 Unit test 9 results

4.2.10 Unit testing 10 : Testing if the user verifies their email through the verification link

Test	Unit testing 10
Description	Testing if the user verifies their email through the verification link
Action	The token was verified, using the py test for the test case where the uid and token value was compared. Along with success message was asserted.
Results	The user's email address was verified.

Table 13 10 Unit testing 10: Testing if the user verifies their email through the verification link

```

@pytest.mark.django_db
def test_verify_email_success(client):
    """Test successful email verification."""
    # create a test user
    user = UserAuthentication.objects.create(
        email="testuser@example.com",
        password=UserAuthentication.hash_password("password123"),
        phone_number="1234567890",
        is_email_verified=False
    )

    # generate the token and UID
    uid = urlsafe_base64_encode(force_bytes(user.pk))
    token = default_token_generator.make_token(user)
    # verify the email
    verify_url = reverse('verify_email', kwargs={'uidb64': uid, 'token': token})
    response = client.post(verify_url) # Use POST instead of GET

    # check the response
    assert response.status_code == 200
    assert "Email verified successfully! You can now log in." in response.content.decode()

    # Check that the user's email is now verified
    user.refresh_from_db()
    assert user.is_email_verified is True

```

Figure 58 Unit test 10 Email verification operation

```

(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_verify_email_success
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 10 items / 9 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 9 deselected in 4.99s =====

```

Figure 59 Unit test 10 results

4.2.11 Unit testing 11 Testing if the logout function works or not

Test	Unit testing 11
Description	Testing if the logout function works or not
Action	The session was checked in the test case code to determine if the session exist after the logout function hits using pytes.
Results	The user's email address was verified.

Table 14 Unit testing 11 Testing if the logout function works or not

```
@pytest.mark.django_db
def test_logout_user(client):
    """Test the logout functionality."""
    # Simulating a logged-in user by setting session data
    session = client.session
    session['user_id'] = 3 # Example user ID
    session.save()

    # Ensuring the session contains the user ID before log
    assert 'user_id' in client.session
    # Call the logout view
    logout_url = reverse('logout')
    response = client.get(logout_url)
    # Assert the session is cleared
    assert 'user_id' not in client.session
    # Assert the user is redirected to the login page
    assert response.status_code == 302
    assert response.url == reverse('login')
```

Figure 60 Unit test 11: Logout function

```
===== 10 deselected in 0.10s =====
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_logout_user
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 11 items / 10 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 10 deselected in 4.86s =====
(.venv) PS E:\Sajilo E-bank\webapp>
```

Figure 61 Unit test 11 results

4.2.11 Unit testing 12 Testing if the login functionality works with the correct credentials

Test	Unit testing 12
Description	Testing if the login functionality works with the correct credentials
Action	The correct credential was passed to the test case function using pytest
Results	The login functionality worked, and the user was redirected to main page.

Table 15 Unit testing 12 Testing if the login functionality works with the correct credentials

```
# Testing the login functionality with correct credentials
@pytest.mark.django_db
def test_login_user_success(client):
    """Test successful login."""
    # Create a test user
    user = UserAuthentication.objects.create(
        email="testuser@example.com",
        password=make_password("password123"),
        phone_number="1234567890",
        is_email_verified=True
    )

    # POST request to the login view
    login_url = reverse('login')
    data = {"email": "testuser@example.com", "password": "password123"}
    response = client.post(login_url, data)

    # Asserting the user is redirected to the document page
    assert response.status_code == 302
    assert response.url == reverse('document')

    # Asserting the session contains the user ID
    assert client.session["user_id"] == user.user_id
```

Figure 62 Unit Test 12 login functionality with correct credentials

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_login_user_success
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 12 items / 11 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 11 deselected in 5.22s =====
(.venv) PS E:\Sajilo E-bank\webapp>
```

Figure 63 Unit 12 results

4.2.13 Unit testing 13: Testing if the login functionality works or not with the unverified email

Test	Unit testing 13
Description	Testing if the login functionality works or not with the unverified email
Action	The unverified email account was passed to the test case function using pytest.
Results	The login functionality does not work with unverified email.

Table 16 Unit testing 13 : Testing if the login functionality works or not with the unverified email

```
# Testing the login functionality with unverified email
@pytest.mark.django_db
def test_login_user_unverified_email(client):
    """Test login with an unverified email"""
    # Creating a test user with an unverified email
    UserAuthentication.objects.create(
        email="testuser@example.com",
        password=make_password("password123"),
        phone_number="1234567890",
        is_email_verified=False # verification status is False
    )

    # POST request to the login view
    login_url = reverse('login')
    data = {"email": "testuser@example.com", "password": "password123"}
    response = client.post(login_url, data)

    # Asserting the response contains the error message
    assert response.status_code == 200
    assert "Please verify your email before logging in." in response.content.decode()
```

Figure 64 Unit 13 login functionality with unverified email

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_login_user_unverified_email
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 13 items / 12 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 12 deselected in 5.18s =====
```

Figure 65 Unit Test 13 Results

4.2.14 Unit testing 14 : Testing if the login functionality works or not with the wrong password

Test	Unit testing 14
Description	Testing if the login functionality works or not with the wrong password
Action	The wrong password was passed to the test case using pytest
Results	The login functionality does not work with wrong password.

Table 17 Unit testing 14 : Testing if the login functionality works or not with the wrong password

```
# testing the login functionality with incorrect password
@pytest.mark.django_db
def test_login_user_incorrect_password(client):
    """Test login with an incorrect password."""
    # Creating a test user
    UserAuthentication.objects.create(
        email="testuser@example.com",
        password=make_password("123"),
        phone_number="1234567890",
        is_email_verified=True
    )

    # POST request with an incorrect password
    login_url = reverse('login')
    data = {"email": "testuser@example.com", "password": "password11"}
    response = client.post(login_url, data) #response with incorrect password

    # Asserting the response contains the error message
    assert response.status_code == 200
    assert "Password is incorrect!" in response.content.decode()
```

Figure 66 Unit test 14 login functionality with incorrect password

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_login_user_incorrect_password
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 14 items / 13 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 13 deselected in 5.12s =====
```

Figure 67 Unit test 14 results

4.2.15 Unit test 15: Testing if the login functionality works or not with the email that does not exist in the system.

Test	Unit testing 15
Description	Testing if the login functionality works or not with the email that does not exist in the system.
Action	The email which does not exist in the system was passed to the test case using pytest
Results	The login functionality does not work with an email that does not exist in the system.

Table 18 Unit test 15: Testing if the login functionality works or not with the email that does not exist in the system.

```
# testing the login functionality with an email that does not exist in the database
@pytest.mark.django_db
def test_login_user_email_not_exist(client):
    """Test login with an email that does not exist in the database."""
    # POST request with a non-existent email
    login_url = reverse('login')
    data = {"email": "nonexistent@example.com", "password": "password123"} # non-existent email a
    response = client.post(login_url, data)

    # Asserting the response contains the error message
    assert response.status_code == 200
    assert "Email does not exist!" in response.content.decode()
```

Figure 68 Unit test 15 login functionality with an email that does not exist

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_login_user_email_not_exist
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 15 items / 14 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 14 deselected in 4.69s =====
```

Figure 69 Unit test 15 results

4.2.16 Unit testing 16 Testing if the Reset password functionality

Test	Unit testing 16
Description	Testing if the Reset password functionality
Action	The reset password with the new password was sent and verified using pytest
Results	The password was reset with a new password

Table 19 Unit testing 16 Testing if the Reset password functionality

```
#testing the password reset functionality
@pytest.mark.django_db
def test_reset_password_success(client):
    """Test successful password reset"""
    # Create a test user
    user = UserAuthentication.objects.create(
        email="testuser@example.com",
        password=make_password("oldpassword123"),
        phone_number="1234567890",
        is_email_verified=True
    )
    # Generate a valid reset token
    signer = TimestampSigner()
    reset_token = signer.sign(user.user_id)
    # POST request to reset the password
    reset_url = reverse('reset-password', kwargs={'token': reset_token})
    data = {
        "new_password": "newpassword123",
        "confirm_password": "newpassword123"
    }
    response = client.post(reset_url, data)

    # Asserting the response
    assert response.status_code == 200
    assert "Password has been reset successfully." in response.content.decode()

    # Verifying the password has been updated or not
    user.refresh_from_db()
    assert check_password("newpassword123", user.password)
```

Figure 70 Unit 16: Reset password functionality

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_reset_password_success
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 16 items / 15 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 15 deselected in 5.50s =====
```

Figure 71 Unit test 16 results

4.2.17 Unit testing 17: Testing if the Reset password link is sent to the user's email

Test	Unit testing 17
Description	Testing if the Reset password link is sent to the user's email
Action	The reset password link dummy was created and passed to the test case function using py test
Results	The reset password link was sent to the user email address

Table 20 Unit testing 17 : Testing if the Reset password link is sent to the user's email

```
#Test complete
#Testing the password reset email sent functionality
@pytest.mark.django_db
def test_reset_password_email_link(client):
    """Test that the reset password email contains the correct reset link."""
    # Create a test user
    user = UserAuthentication.objects.create(
        email="testuser@example.com",
        password=UserAuthentication.hash_password("password123"),
        phone_number="1234567890",
        is_email_verified=True
    )
    # Simulate a POST request to send the reset email
    reset_email_url = reverse('send-password-reset')
    data = {"email": user.email}
    response = client.post(reset_email_url, data)
    # Assert the response
    assert response.status_code == 200
    assert "Please check your email" in response.content.decode()
    # Check that an email was sent
    assert len(mail.outbox) == 1 # Ensure one email was sent
    email = mail.outbox[0]
    assert email.subject == "Password Reset Request"
    assert user.email in email.to
```

Figure 72 Unit test 17 Reset password email link

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k test_reset_password_email_link
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 17 items / 16 deselected / 1 selected

user_authentication\tests.py . [100%]

===== 1 passed, 16 deselected in 5.80s =====
```

Figure 73 Unit test 17 results

4.3 URL TESTING

4.3.1 URL Testing 1: Testing the URL of the register view

Test	URL Testing 1
Description	Testing the URL of the register view
Action	The register URL path was checked to determine if it resolved to the view name and the URL valid http response was verified.
Results	The URL of the register view was resolved, and a correct HTTP response was returned.

Table 21 URL Testing 1: Testing the URL of the register view

```
# register url testing
# #complete
def test_register_url():
    """Testing the register URL."""
    path = '/register/'
    assert resolve(path).view_name == 'register'

def test_register_url_status(client):
    url = reverse('register')
    response = client.get(url)
    assert response.status_code in [200, 302]
```

Figure 74 URL Testing 1 register

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k "test_register_url or test_register_url_status"
>> ;
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 19 items / 17 deselected / 2 selected

user_authentication\tests.py .. [100%]

===== 2 passed, 17 deselected in 3.20s =====
```

Figure 75 Results of URL Testing 1

4.3.2 URL Testing 2 Testing the URL of the login view

Test	URL Testing 2
Description	Testing the URL of the login view
Action	The login URL path was checked to determine if it resolved to the view name, and the URL's validity as an HTTP response was verified.
Results	The URL of the login view was resolved, and a correct HTTP response was returned.

Table 22 URL Testing 2 Testing the URL of the login view

```
# login url testing
# complete
def test_login_url():
    """Testing the login URL."""
    path = '/login/'
    assert resolve(path).view_name == 'login'

def test_login_url_status(client):
    """Testing the login URL status."""
    url = reverse('login')
    response = client.get(url)
    assert response.status_code in [200, 302]
```

Figure 76 URL Test 2 login

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k "test_login_url or test_login_url_status"
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 21 items / 19 deselected / 2 selected

user_authentication\tests.py .. [100%]

===== 2 passed, 19 deselected in 3.12s =====
```

Figure 77 URL test 2 results

4.3.3 URL Testing 3 : Testing the URL of the logout function

Test	URL Testing 3
Description	Testing the URL of the logout function
Action	The logout URL path was checked to determine if it resolved to the view name, and the URL's validity as an HTTP response was verified.
Results	The URL of the logout view was resolved, and a correct HTTP response was returned 200 or 302.

Table 23 URL Testing 3 : Testing the URL of the logout function

```
# logout url testing
def test_logout_url():
    """Testing the logout URL."""
    path = '/logout/'
    assert resolve(path).view_name == 'logout'

def test_logout_url_status(client):
    """Testing the logout URL status."""
    url = reverse('logout')
    response = client.get(url)
    assert response.status_code in [200, 302] # logout redirects to the login pag
```

Figure 78 URL testing 3 logout

```
===== 2 passed, 19 deselected in 3.12s =====
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k "test_logout_url or test_logout_url_status"
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 23 items / 21 deselected / 2 selected

user_authentication\tests.py .. [100%]

===== 2 passed, 21 deselected in 3.29s =====
(.venv) PS E:\Sajilo E-bank\webapp>
```

Figure 79 URL testing 3 results

4.3.4 URL Testing 4: Testing the URL of the email verification

Test	URL Testing 4
Description	Testing the URL of the email verification
Action	The email verification URL path was checked to determine if it resolved to the view name, and the URL's validity as an HTTP response was verified.
Results	The URL of the email verification view was resolved, and a correct HTTP response was returned 200 or 400.

Table 24 URL Testing 4 : Testing the URL of the email verification

```
# email verification url testing
#complete
def test_verify_email_url():
    """Testing the verify email URL."""
    path = '/verify-email/test-uid/test-token/'
    assert resolve(path).view_name == 'verify_email'

def test_verify_email_url_status(client):
    """Testing the verify email URL status."""
    url = reverse('verify_email', kwargs={'uidb64': 'test-uid', 'token': 'test-token'})
    response = client.get(url)
    assert response.status_code in [200, 400] # 400 for invalid or expired token
                                           # 200 for valid token or success
```

Figure 80 URL test 4 email verification

```
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k "test_verify_email_url or test_verify_email_url_status"
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 25 items / 23 deselected / 2 selected

user_authentication\tests.py .. [100%]

===== 2 passed, 23 deselected in 2.42s =====
```

Figure 81 URL test 4 results

4.3.5 URL testing 5: Testing the URL of the reset password

Test	URL Testing 5
Description	Testing the URL of the reset password
Action	The reset password URL path was checked to determine if it resolved to the view name, and the URL's validity as an HTTP response was verified.
Results	The URL of the reset password view was resolved, and a correct HTTP response, 200 or 400, was returned.

Table 25 URL testing 5: Testing the URL of the reset password

```
# complete\
# reset password url testing
def test_reset_password_url():
    """Testing the reset password URL."""
    path = '/reset-password/test-token/'
    assert resolve(path).view_name == 'reset-password'

def test_reset_password_url_status(client):
    """Testing the reset password URL status."""
    url = reverse('reset-password', kwargs={'token': 'test-token'})
    response = client.get(url)
    assert response.status_code in [200, 400] # 400 for invalid or expired token
```

Figure 82 URL Test 5 reset password

```
===== 2 passed, 23 deselected in 2.42s =====
(.venv) PS E:\Sajilo E-bank\webapp> pytest -k "test_reset_password_url or test_reset_password_url_status"
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 27 items / 25 deselected / 2 selected

user_authentication\tests.py ..                                     [100%]

===== 2 passed, 25 deselected in 2.48s =====
```

Figure 83 URL testing 5 results

4.3.6 URL Testing 6 : Testing the URL of the dashboard.

Test	URL Testing 6
Description	Testing the URL of the dashboard
Action	The reset password URL path was checked to determine if it resolved to the view name, and the URL's validity as an HTTP response was verified.
Results	The URL of the reset password view was resolved, and a correct HTTP response, 200 or 400, was returned.

Table 26 URL Testing 6: Testing the URL of the dashboard.

```
@pytest.mark.django_db
def test_dashboard_url():
    """Testing the Dashboard URL."""
    path = '/dashboard'
    assert resolve(path).view_name == 'dashbaord'

def test_dashboard_status(client):
    """Testing the Dashboard URL status."""
    url = reverse('dashboard' )
    response = client.get(url)
    assert response.status_code in [200, 400]
```

Figure 84 URL testing 6 code

```
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.5, pluggy-1.5.0
django: version: 5.1.6, settings: webapp.settings (from ini)
rootdir: E:\Sajilo E-bank\webapp
configfile: pytest.ini
plugins: anyio-4.8.0, Faker-37.0.0, django-4.11.1, requests-mock-1.12.1
collected 27 items / 25 deselected / 2 selected

ser_authentication\tests.py ..                                     [100%]

===== 2 passed, 25 deselected in 2.48s =====
```

Figure 85 Results of url testing 6

4.4 SYSTEM TESTING

4.4.1 System testing 1 : Testing the user Registration process

Test	System Testing 1
Description	Testing the user Registration process
Action	The required detail for the registration was filled with valid data, then the Email verification process was conducted.
Results	The user account was successfully created
Conclusion	The test was successful

Table 27 System testing 1 : Testing the user Registration process

Welcome to Sajilo Ebank!
Fast, secure, and easy banking services.

Create an Account

Enter your email
np01ai4a220003@islingtoncollege.edu.np

Enter your phone number
9823788703

Enter your password
.....

Re-enter password
.....

Sign Up

Already have an account? [Login now](#)

Figure 86 System Testing 1 User registration detail fill up

The screenshot shows the 'Create an Account' page for Sajilo Ebank. On the left, a purple banner reads 'Welcome to Sajilo Ebank!' with the tagline 'Fast, secure, and easy banking services.' On the right, a registration form is displayed with the following fields: 'Enter your email', 'Enter your phone number', 'Enter your password', and 'Re-enter password'. A blue 'Sign Up' button is located below the password fields. Below the button, a link says 'Already have an account? [Login now](#)'. A red box highlights a green message box that says 'Verification email sent. Please check your inbox.', with a red arrow pointing from the 'Sign Up' button to it.

Figure 87 System Testing I Notification after successful registration

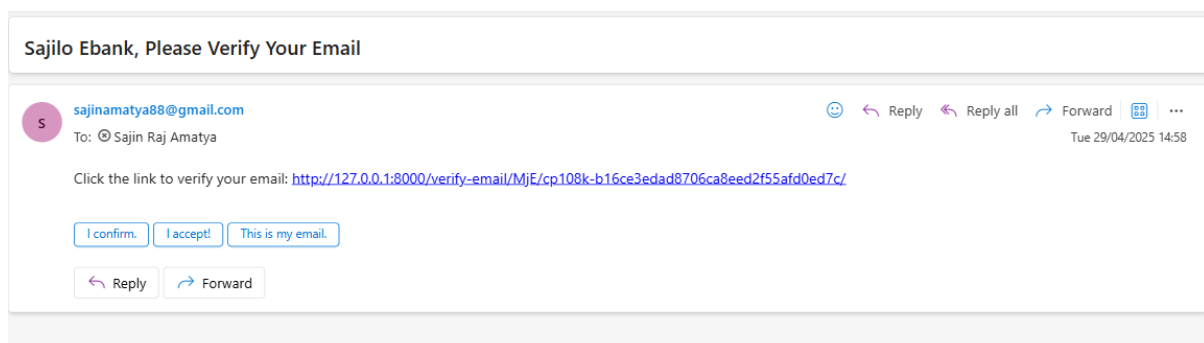


Figure 88 System Testing I Email verification link via mail

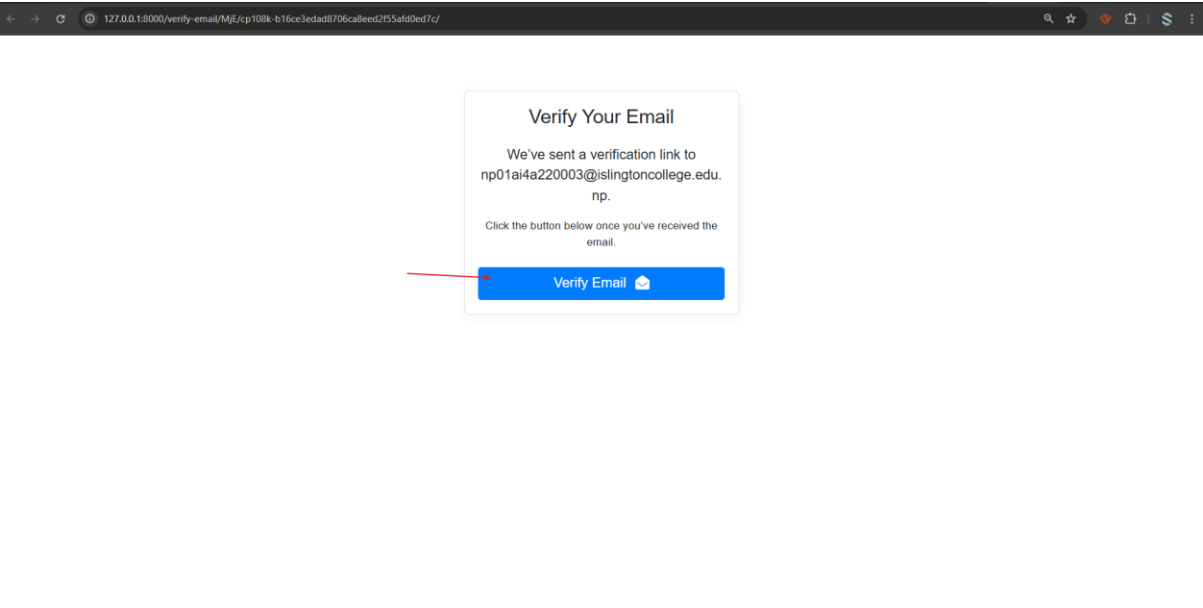


Figure 89 System Testing 1: Verifying the account

Result Grid									
Filter Rows:									
Edit:									
Export/Import:									
Wrap Cell Content:									
email	password	phone_number	account_created_date	is_email_verified	login_count	user_id	last_login	is_document_valid	
sajinamatya88@gmail.com	pbkdf2_sha256\$870000\$WvtfPwfh1miHno5qq...	11	2025-02-19 16:42:58.361382	1	167	11	2025-02-19 16:42:58.361382	1	
ss	pbkdf2_sha256\$870000\$D9DoaVgR1ztC8Y3fbe...	ss	2025-02-22 05:09:14.044625	0	0	12	2025-02-22 05:09:14.044625	0	
np01ai4a220003@islingtoncollege.edu.np	pbkdf2_sha256\$870000\$PTSyFHqmxG46KtMAT...	9823788703	2025-04-29 09:12:56.826314	1	0	21	2025-04-29 09:12:56.826314	0	
total	total	total	total	total	total	total	total	total	

Figure 90 System Testing 1 Inspection of recently created used in database of the system

4.4.2 System Testing 2: Testing the user login process

Test	System Testing 2
Description	Testing the user login process
Action	The required detail for the login was filled with valid data. Email: “ np01ai4a220003@islingtoncollege.edu.np ” Password: “sajin” The submit button was clicked.
Results	The user was successfully authenticated to the system
Conclusion	The test was successful

Table 28 System Testing 2: Testing the user login process

Welcome to Sajilo Ebank!

Login

Email:

Password:

[Submit](#)

Not registered yet ? [Create an account](#)

Forgot your password ? [Reset password](#)

Lets get started!

Figure 91 System Testing 2 User credential for authentication to the system

Sajilo E-Bank

HomeLocationDocument UploadLogout

Upload Your Documents

Front Side

Back Side

Upload

Figure 92 System Testing 2: Redirection to document upload page after successful login

4.4.3 System Testing 3 : Testing the Reset Password Functionality

Test	System Testing 3
Description	Testing the Reset Password Functionality
Action	The required details for resetting the password were filled out with valid data. Email: “np01ai4a220003@islingtoncollege.edu.np” The submit button was clicked. In the reset password form: New password: “hello” Confirm password: “hello” Then, using a new credential, login detail was filled then login button was click
Results	The user password was successfully changed
Conclusion	The test was successful

Table 29 System Testing 3 : Testing the Reset Password Functionality

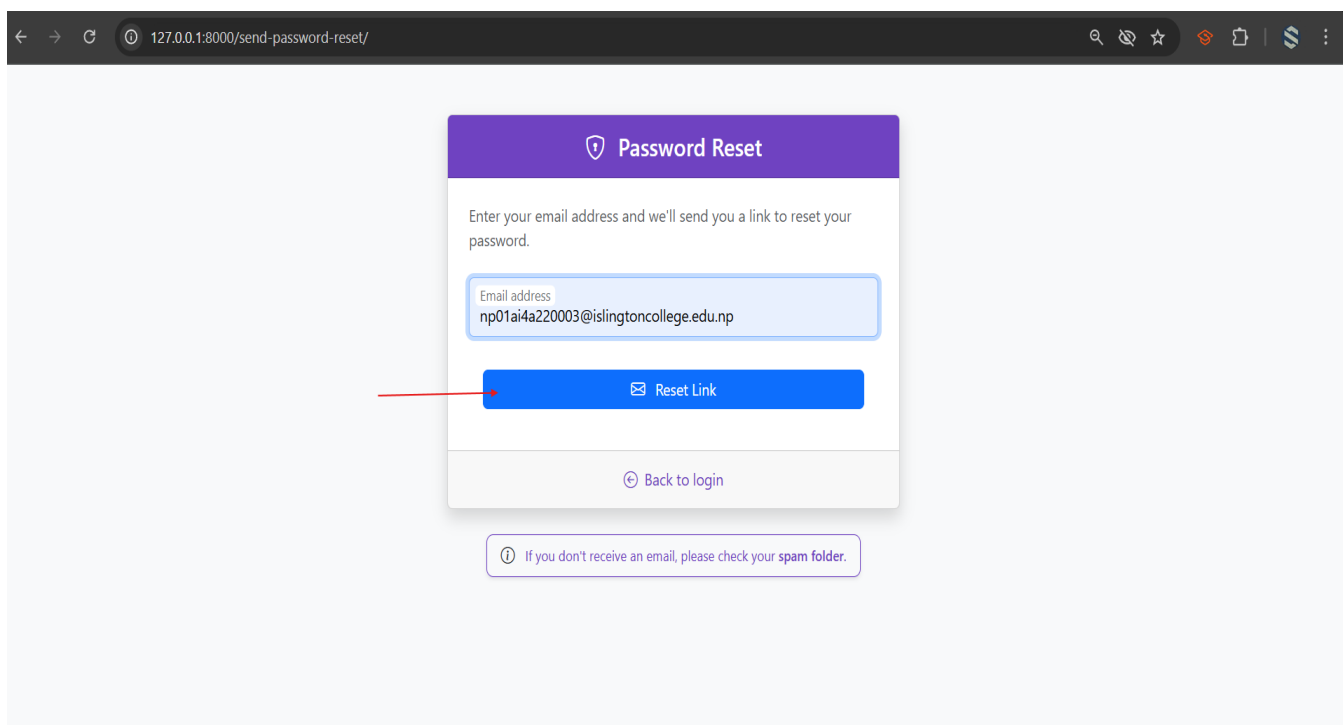


Figure 93 System Testing 3: Password Reset

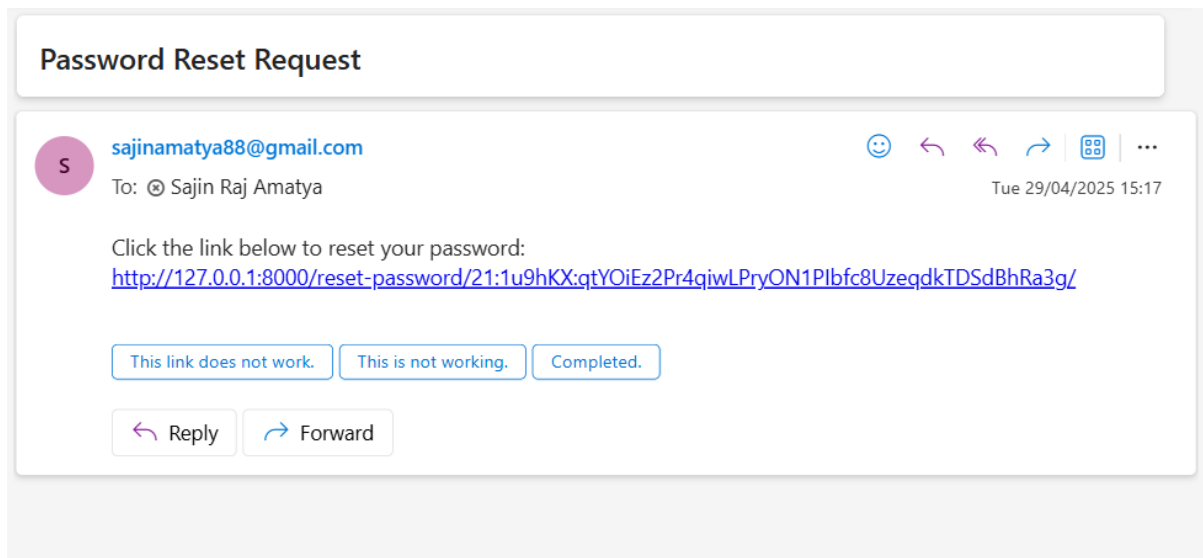


Figure 94 System Testing 3: Reset password link via email

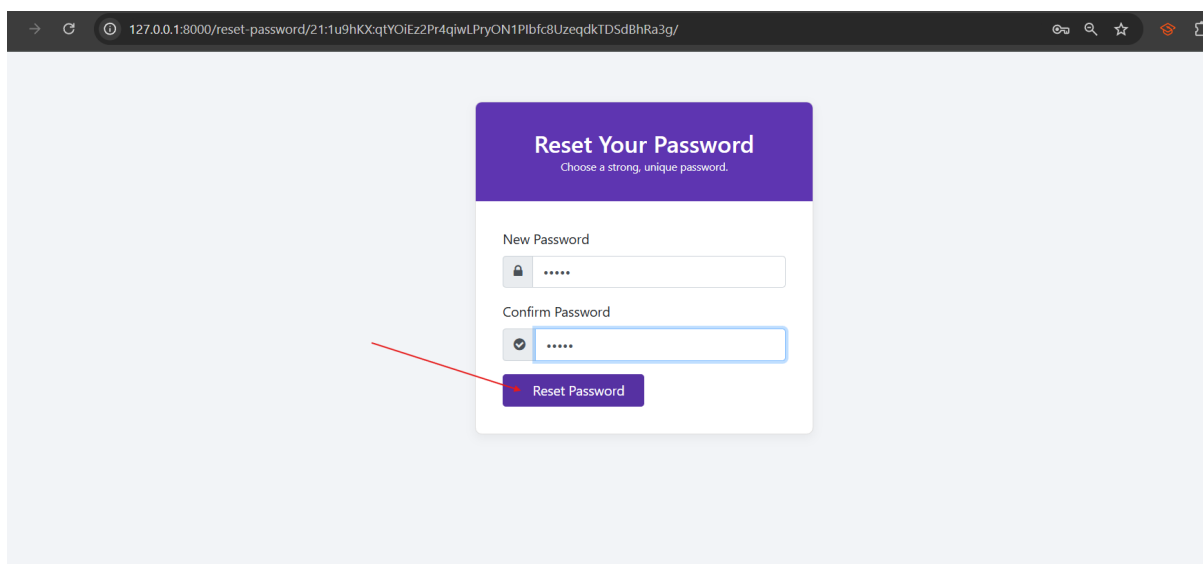


Figure 95 System Testing 3: Reset password form

4.4.4 System Testing 4 : Testing the Document Upload Functionality along with user detail form

Test	System Testing 4
Description	Testing the Document Upload Functionality along with user detail form
Action	The Front side and the back side of the document were uploaded then, submit button was clicked.
Results	The user password was successfully changed
Conclusion	The test was successful

Table 30 System Testing 4 : Testing the Document Upload Functionality along with user detail form

The screenshot shows the 'Sajilo E-Bank' interface. At the top, there's a navigation bar with 'Home', 'Location', 'Document Upload', and 'Logout' links. The main heading is 'Upload Your Documents'. Below this, there are two document upload sections. The 'Front Side' section shows a scanned ID card with Nepali text and a photo. The 'Back Side' section shows a scanned document with personal details in Nepali and English. Below these sections is an 'Upload' button with a red arrow pointing to it.

Figure 96 System Testing 4 User Document upload

Both the front-side and back-side documents are valid!

Upload Your Documents

Front Side

Figure 97 System Testing 4 Success message after the document validation

Result Grid			Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
user_id	back_side_url	front_side_url				
11	/media/uploads/back/11_back_2025042913462...	/media/uploads/front/11_front_202504291346...				
21	/media/uploads/back/21_back_2025042915353...	/media/uploads/front/21_front_202504291535...				

Figure 98 System Testing 4: Document URL saved in the database after successful validation

email	password	phone_number	account_created_date	is_email_verified	is_document_valid	login_count	user_id	last_login
sajinamatya88@gmail.com	pbkdf2_sha256\$870000\$WvtfPwfh1mHno5qq...	11	2025-02-19 16:42:58.361382	1	1	168	11	2025-02-19 16:4...
ss	pbkdf2_sha256\$870000\$D9DoaVgR1ztC8Y3fbc...	ss	2025-02-22 05:09:14.044625	0	0	0	12	2025-02-22 05:0...
np01ai4a220003@islingtoncollege.edu.np	pbkdf2_sha256\$870000\$VRHWIRPL7BhX9Pr6z...	9823788703	2025-04-29 09:12:56.826314	1	1	3	21	2025-04-29 09:1...
HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL

Figure 99 System Testing 4: The document validation status is True after successful validation

Sajilo E-Bank

HomeLocationDocument UploadLogout

Citizenship Information Form

Citizenship Number

Enter your citizenship number

Full Name

Enter your full name

Gender

Select your gender

Date of Birth

mm/dd/yyyy

Place of Birth

Enter place of birth

Permanent Address

Enter your permanent address

Reset

Submit

Figure 100 System Testing 4 Redirect to the citizenship detail form

4.4.5 System Testing 5: Testing the User location tracking Functionality

Test	System Testing 5
Description	Testing the User location tracking Functionality
Action	The location detail was searched, and tracking functionality was triggered.
Results	The location search would appear, along with track location.
Conclusion	The test was successful

Table 31 System Testing 5: Testing the User location tracking Functionality

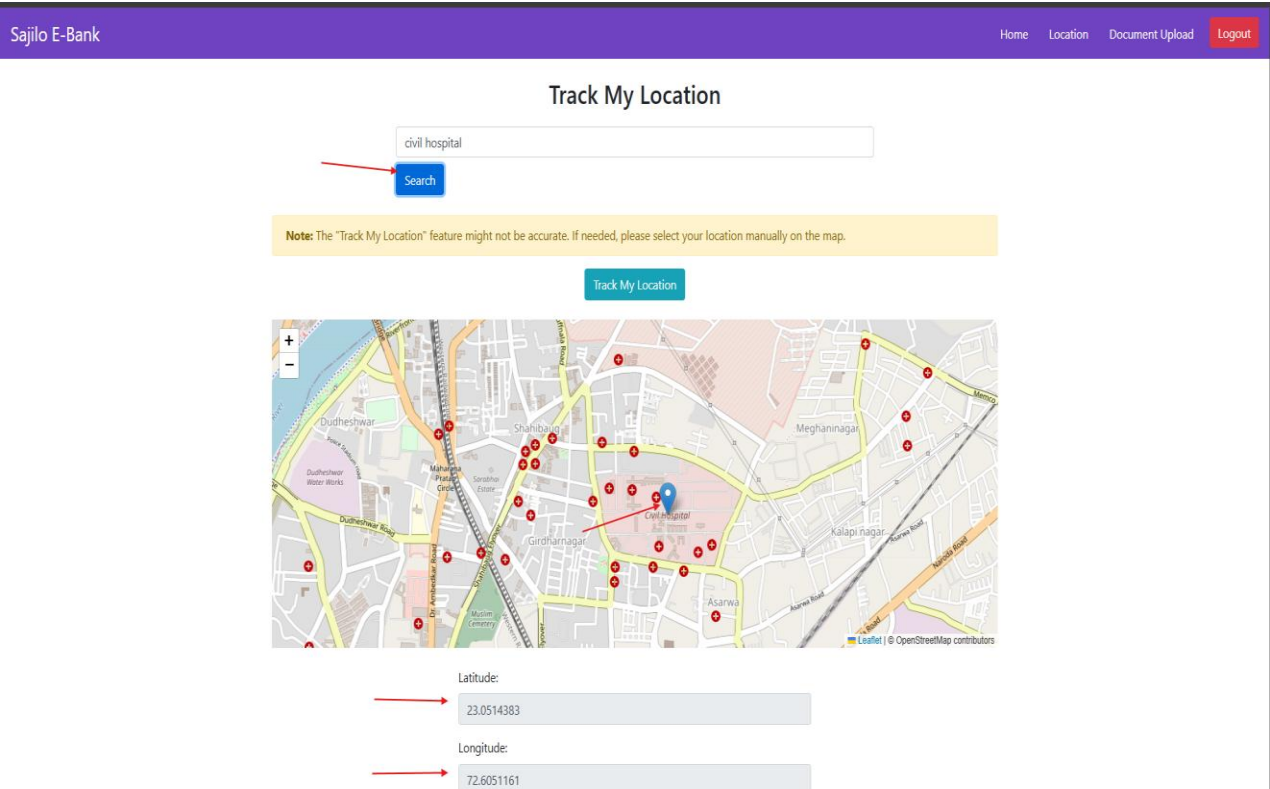


Figure 101 System Testing 5 location search

4.4.6 System Testing 6: Testing the dashboard functionality

Test	System Testing 6
Description	Testing the dashboard functionality
Action	The dashboard functionality like update and delete operation was conduction.
Results	The Dashboard functionality like update and delete feature works properly.
Conclusion	The test was successful

Table 32 System Testing 6 : Testing the dashboard functionality

The screenshot shows a web dashboard with a purple header. Below the header, there are three summary cards: 'Total Number of Users' (3), 'Total Login Count' (177), and 'Total Verified Emails' (2). Below these cards is a 'User Details' section with a table of user information. The table has columns for User No., User Id, Email, Phone number, Account creation date, Email Verified Status, Document Verified Status, Last Login, Login Count, Update, and Delete. A red box highlights the 'Update' button for the second user (User Id 12).

User No.	User Id	Email	Phone number	Account creation date	Email Verified Status	Document Verified Status	Last Login	Login Count	Update	Delete
1	11	sajinamatya88@gmail.com	11s	Feb. 19, 2025, 4:42 p.m.	True	True	Feb. 19, 2025, 4:42 p.m.	174		
2	12	ss	ss	Feb. 22, 2025, 5:09 a.m.	False	False	Feb. 22, 2025, 5:09 a.m.	0		
3	21	np01ai4a220003@islingtoncollege.edu.np	9823788703	April 29, 2025, 9:12 a.m.	True	True	April 29, 2025, 9:12 a.m.	3		

Figure 102 System Testing 6 user dashboard update user detail

Update User 12

Email

ss

Phone Number

ss

☐ Email Verified

☐ Document Valid

Save Changes

Cancel

Figure 103 System testing 6 updating the user details

Update User 12

Email

ss@gmail.com

Phone Number

123421561

☒ Email Verified

☐ Document Valid

Save Changes

Cancel

Figure 104 System testing 6 saving the change details

Dashboard

Total Number of Users

3

Total Login Count

177

Total Verified Emails

3

User Details

No.	User Id	Email	Phone number	Account creation date	Email Verified Status	Document Verified Status	Last Login	Login Count	Update	Delete
1	11	sajinamatya88@gmail.com	11s	Feb. 19, 2025, 4:42 p.m.	True	True	Feb. 19, 2025, 4:42 p.m.	174		
2	12	ss@gmail.com	123421561	Feb. 22, 2025, 5:09 a.m.	True	False	Feb. 22, 2025, 5:09 a.m.	0		
3	21	np01ai4a220003@islingtoncollege.edu.np	9823788703	April 29, 2025, 9:12 a.m.	True	True	April 29, 2025, 9:12 a.m.	3		

Figure 105 System testing 6 inspection after updation

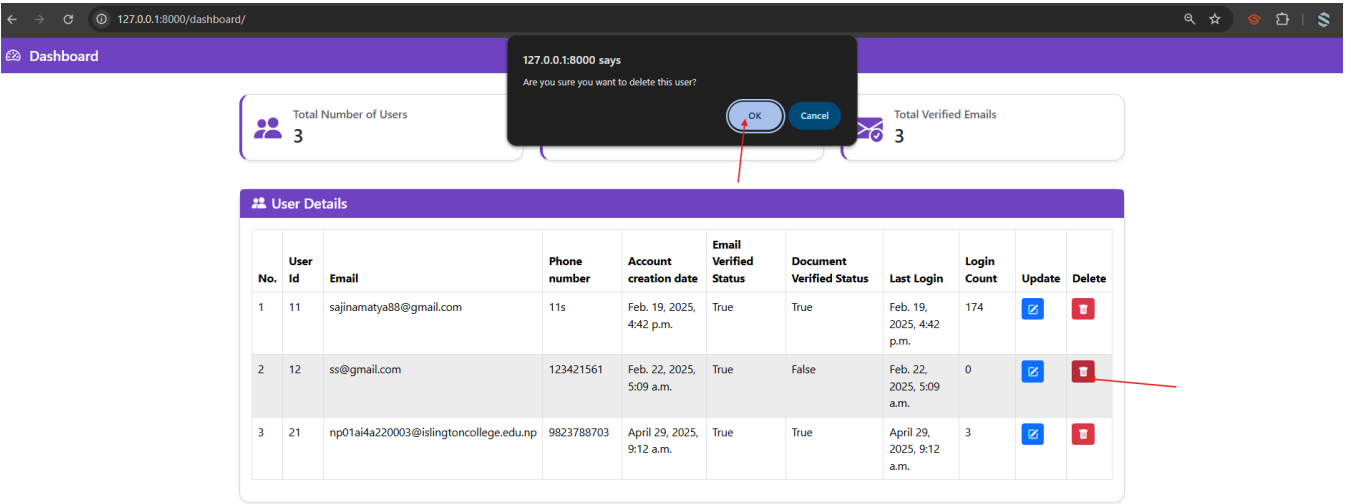


Figure 106 System Testing 6 user deletion dashboard

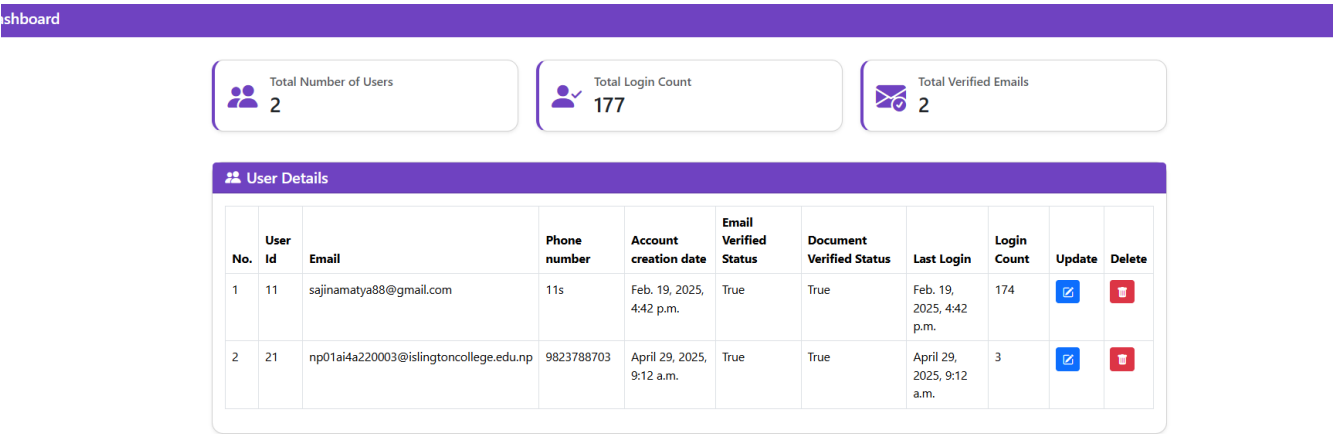


Figure 107 System Testing 6 Dashboard table after user deletion

CHAPTER 5: CONCLUSION

5.1 LEGAL, SOCIAL and ETHICAL ISSUES

5.1.1 LEGAL ISSUES

Collecting the citizenship data along with user confidential information without user consent can cause the legal issue as per the Nepal Data Protection Act (NDPA). Storing the biometric information in the system the law must comply with it to which govern how the data is stored, processed and shared too other.

5.1.2 SOCIAL ISSUES

To access the system, the user needs to have access to the device with internet connection, the user with no access to the device is restricted to using the system. The people might also be on doubt regarding their sensitive data being collected and the transparency might be required.

5.1.3 ETHICAL ISSUES

The user must be aware about the information regarding the consent and transparency of the system, agreement must be check by the user before accepting the system consent. The sensitive information stored in the system must be protected against data breach to minimize ethical obligations.

5.2 ADVANTAGES

1. The user can open their account within a minute without visiting the bank branch.
2. OCR can increase the accuracy of data entry reducing the error from the manual entry.
3. Higher security due to use of biometric which is safer than password and other security process.
4. Decrease in operational cost on staff due to manual work.
5. Integration with other system like fraud detection, wallet etc. to create a fully working banking system

CHAPTER 6: REFERENCES

References

- Albahary, S., 2020. *Digital account opening: the secret to growth amid Covid-19?*. [Online] Available at: <https://www.bizjournals.com/columbus/news/2020/11/02/digital-account-opening-the-secret-to-growth-amid.html> [Accessed 19 September 2024].
- Bhaskar, 2024. *What is Scrum Methodology? An Introduction for Agile Teams*. [Online] Available at: <https://www.nimblework.com/agile/scrum-methodology/> [Accessed 27 December 2024].
- Cocheo, S., 2024. *How COVID-19 Permanently Changed Americans' Financial Habits*. [Online] Available at: <https://thefinancialbrand.com/news/banking-trends-strategies/covid-19-digital-banking-app-fintech-innovation-trend-lockdown-branch-102277/> [Accessed 2 Decemember 2024].
- Deepti Singh, A. T. A. C., 2015. A Comparative Study between Waterfall and Incremental Software. *International Journal of Emerging Trends in Science and Technology*, 02(04), pp. 2202-2208.
- Dhanamma Jagli, S. Y., 2017. CloudSDLC: Cloud Software Development Life Cycle. *International Journal of Computer Applications (0975 – 8887)*, 168(8), p. 6.
- Elinzano, G. B., 2022. *Factors that leads to Adoption and Use of Online Bank Account Opening through e-KYC using UTAUT and its Extensions*. [Online] Available at: https://www.researchgate.net/publication/360126620_Factors_that_leads_to_Adoption_and_Use_of_Online_Bank_Account_Opening_through_e-KYC_using_UTAUT_and_its_Extensions [Accessed 28 December 2024].
- geeksforgeeks, 2024. *Iterative Waterfall Model – Software Engineering*. [Online] Available at: <https://www.geeksforgeeks.org/software-engineering-iterative-waterfall-model/> [Accessed 27 December 2024].
- International, R. B., 2024. *Three best practices for digital account opening in banking*. [Online] Available at: https://www.retailbankerinternational.com/thought_leaders/best-practices-digital-account-opening/ [Accessed 29 December 2024].
- Kaur, C., 2015. Comparative Analysis of Iterative Waterfall. *FP-International Journal of Computer Science Research (IJCSR)*, 2(1), pp. 11-14.
- Leon Perlman, N. G., 2019. *Focus Note: The Use of eKYC for Customer Identity and Verification and AML*. [Online] Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3370665 [Accessed 29 December 2024].

More, J., 2023. *Software Development Model — Incremental Development*. [Online]
Available at: <https://medium.com/design-bootcamp/dlc-software-development-model-incremental-development-7598ae6500e6>
[Accessed 28 November 2024].

Ozelm Senvar, W. Z., 2020. Framework Study for Agile Software Development Via Scrum and Kanban. *International Journal of Innovation and Technology Management*, 12(4), p. 2.

Trivedi, D., 2021. Agile Methodologies. *International Journal of Computer Science & Communication*, 12(2), p. 91.

Y.A.Christian., F., 2018. COMPARATIVE STUDY OF ACCOUNT OPENING PROCESS IN BANK SULUTGO AND BANK MANDIRI : STAKEHOLDERS PERSPERCTIVE. 6(4), p. 2.

zetsche, D., 2018. *Digital ID and AML/CDD/KYC utilities for financial inclusion, integrity and competition*. [Online]
Available at:
https://www.researchgate.net/publication/328092073_Digital_ID_and_AMLCDDKYC_utilities_for_financial_inclusion_integrity_and_competition
[Accessed 29 December 2024].

CHAPTER 8: APPENDIX

8.1 APPENDIX A: POST SURVEY

8.1.1 Sample of filled post survey forms

Responses cannot be edited

Sajilo E-bank Post-Survey

Thank you for your review on the Sajilo E-bank System.
Your response will help us to improve the system quality.
It will only take a few minutes to complete!

* Indicates required question

How was your overall experience using Sajilo E-bank webapp ? *

☐ Poor

☐ Netural

☒ Good

☐ Excellent

Figure 108 Survey form How was your overall experience using Sajilo E-bank webapp?

How simple was it to navigate and use the Sajilo E-bank web application? *

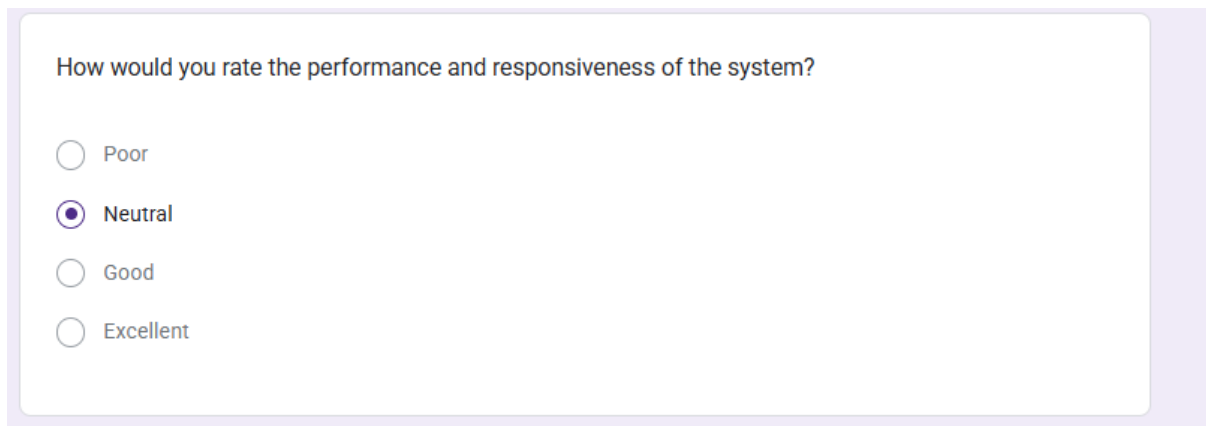
☐ Easy

☒ Neutral

☐ Hard

☐ Very hard

Figure 109 Survey form: How simple was it to navigate and use the Sajilo E-bank



How would you rate the performance and responsiveness of the system?

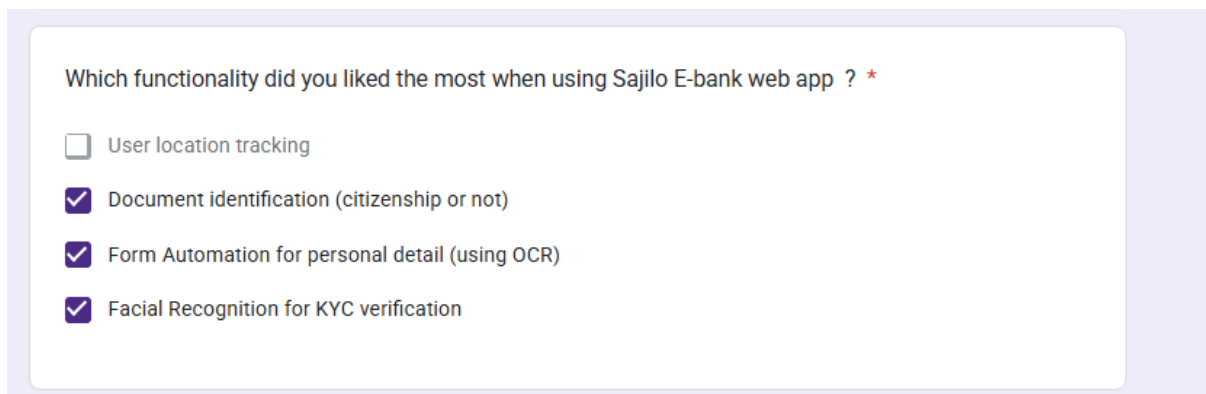
☐ Poor

☒ Neutral

☐ Good

☐ Excellent

Figure 110 Survey form: How would you rate the performance and responsiveness of the system?



Which functionality did you liked the most when using Sajilo E-bank web app ? *

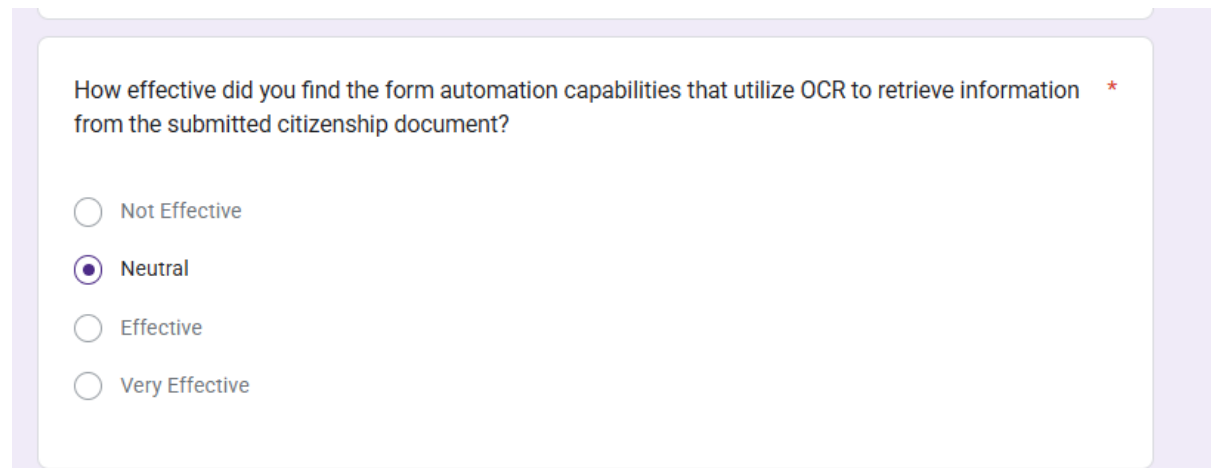
☐ User location tracking

☒ Document identification (citizenship or not)

☒ Form Automation for personal detail (using OCR)

☒ Facial Recognition for KYC verification

Figure 111 Survey form: Which functionality did you liked the most when using Sajilo E-bank web app ?



How effective did you find the form automation capabilities that utilize OCR to retrieve information from the submitted citizenship document? *

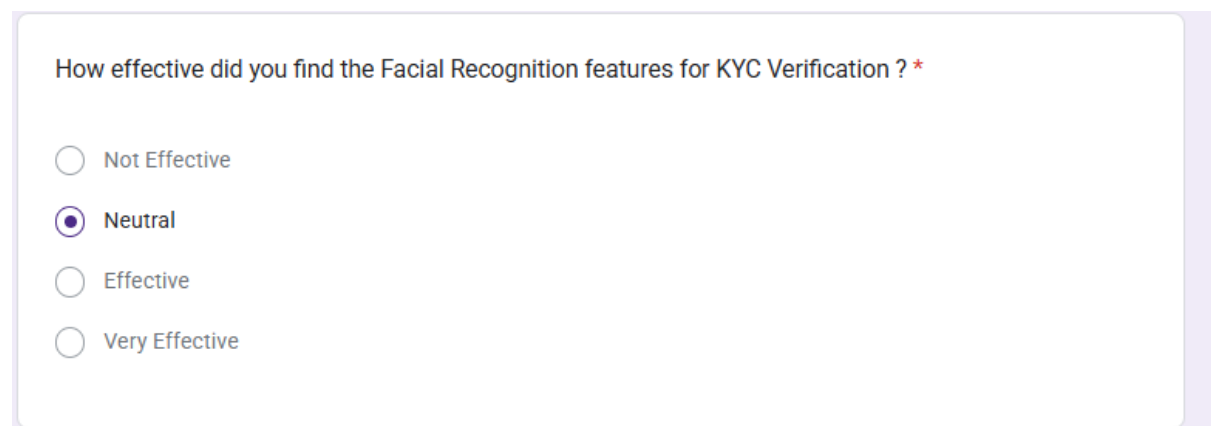
☐ Not Effective

☒ Neutral

☐ Effective

☐ Very Effective

Figure 112 Survey form How effective did you find the form automation capabilities that utilize OCR to retrieve information from the submitted citizenship document?



How effective did you find the Facial Recognition features for KYC Verification ? *

☐ Not Effective

☒ Neutral

☐ Effective

☐ Very Effective

Figure 113 Survey Form: How effective did you find the Facial Recognition features for KYC Verification ?

How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ? *

- ☐ Not Effective
- ☒ Neutral
- ☐ Effective
- ☐ Very Effective

Figure 114 survey form: How effective did you find document identification features which identifies if the user uploaded document is citizenship or not?

How probable is it that you would suggest this system to your colleagues?

- ☐ Improbable
- ☐ Neutral
- ☒ Probable
- ☐ Very Probable

Figure 115 Survey form How probable is it that you would suggest this system to your colleagues?

22 responses



[View in Sheets](#)



Summary

Question

Individual

Figure 116 Survey total responses

8.2.2 Survey results

How was your overall experience using Sajilo E-bank webapp ?

22 responses

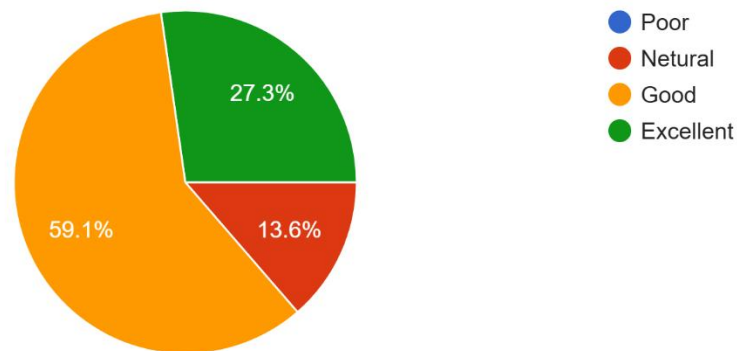


Figure 117 Survey results How was your overall experience using Sajilo E-bank webapp ?

How simple was it to navigate and use the Sajilo E-bank web application?

22 responses

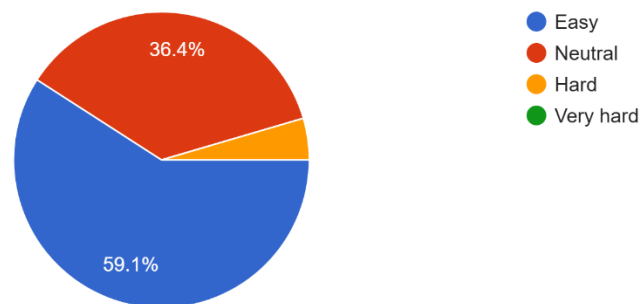


Figure 118 Survey result 2 : How simple was it to navigate and use the Sajilo E-bank web application?

How would you rate the performance and responsiveness of the system?

22 responses

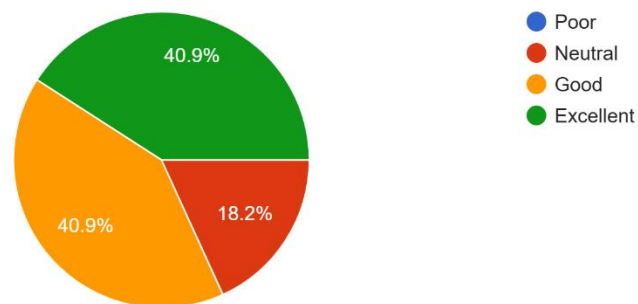


Figure 119 Survey results 3 How would you rate the performance and responsiveness of the system?

Which functionality did you liked the most when using Sajilo E-bank web app ?

22 responses

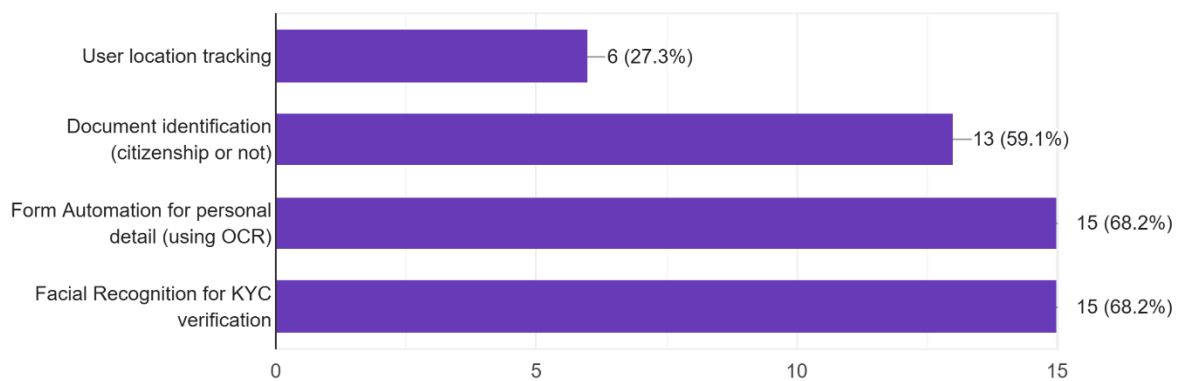


Figure 120 Survey results 4 Which functionality did you liked the most when using Sajilo E-bank web app

How effective did you find the form automation capabilities that utilize OCR to retrieve information from the submitted citizenship document?

22 responses

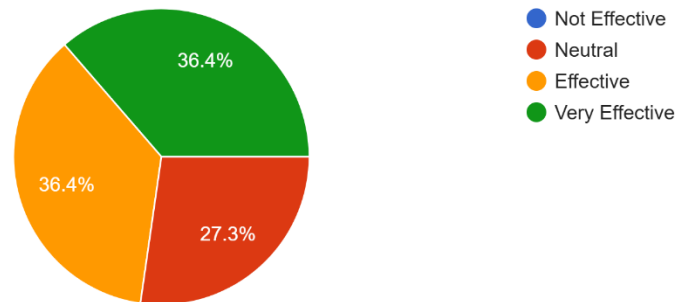


Figure 121 Survey result 5 : How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?

How effective did you find the Facial Recognition features for KYC Verification ?

22 responses

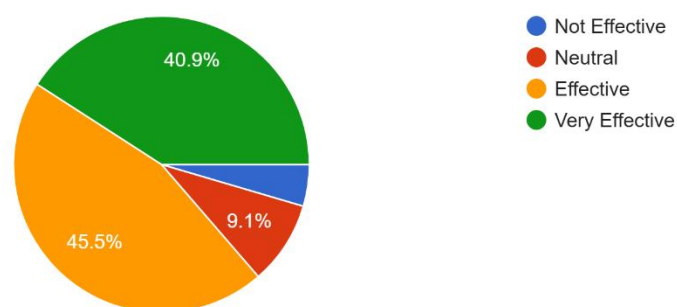


Figure 122 Survey results 6 : How effective did you find the Facial Recognition features for KYC Verification ?

How probable is it that you would suggest this system to your colleagues?

22 responses

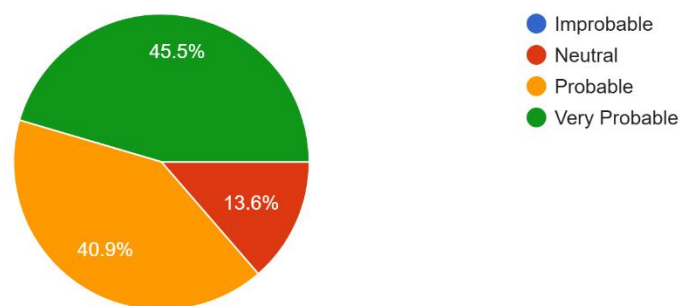


Figure 123 Survey form : How probable is it that you would suggest this system to your colleagues?

How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?

25 responses

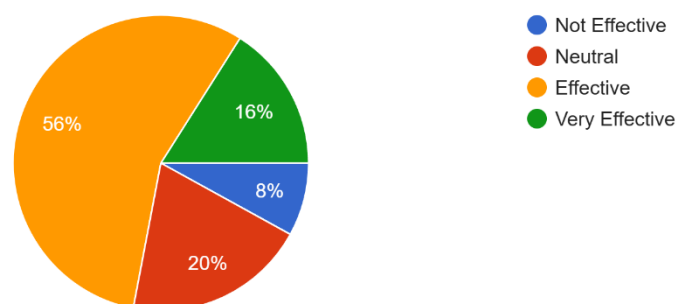


Figure 124 Survey form How effective did you find document identification features which identifies if the user uploaded document is citizenship or not ?

8.1 APPENDIX B: SOFTWARE REQUIREMENTS SPECIFICATIONS

Software Requirements Specification for Sajilo E-bank

Version 1.0 approved

Prepared by: Sajin Raj Amatya

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

8.1.1 Introduction

Purpose

The Software Requirements Specification (SRS) document aims to outline the precise functional and non-functional requirements, system limitations, as well as the requirements for system features and technical specifications for both hardware and software.

Intended Audience

This document offers guidance for the system developer, project manager, tester, and user about the system's features, the issues it aims to address, and its general functionality. The project manager and stakeholders can utilize this document to examine the project's scope, objectives, and vision.

The system developer can refer to it to grasp the technology stack employed in the system's development, while the tester can use it to evaluate the various functionalities of the system according to the requirements.

Project Scope

The aim of this project is to create a web application that digitizes and streamlines the process of opening bank and cooperative accounts by incorporating OCR technology for automated form completion and facial recognition for KYC verification. Users will have the ability to establish their accounts effortlessly and without needing to be physically present for KYC checks, thereby conserving both time and effort.

8.1.2 Overall Description

Product Perspective

The existing method for opening a bank account involves either completing a physical form by hand or filling out an online application that requests excessive information. Manually completing the form can lead to errors, such as incorrectly marking your current location on the paper and waiting in a long line to submit the form at the bank negatively impacts the overall customer experience and wastes their valuable time.

Product Function

The major functions of the system are:

1. Features for user registration and login.
2. Automated form filling of personal details for account creation using OCR technology based on the user's uploaded citizenship document.
3. Tracking of user location in real-time while maintaining user privacy in accordance with the terms and conditions.
4. A facial recognition feature for KYC validation that includes liveness detection, such as eye blinking or head movement.
5. Email notifications for user authentication confirmation and successful completion of the KYC process.
6. An admin dashboard for comprehensive reporting, monitoring account status, user information, KYC progress, and analysis.

User Classes and Characteristics

The system includes two types of users: Admin and customer/regular user. The attributes of each user type are outlined below: Admin

1. Access the comprehensive dashboard displaying summary statistics and details about users.
2. The admin has the authority to add, modify, or remove users.
3. Review the status of email notifications, account status, and KYC compliance for each user.

Customer/Normal user

1. Sign up and log into the system.
2. Submit their documents, such as citizenship, for the purpose of extracting their information to automate form-filling for account creation.
3. Agree to allow tracking of their real-time location if they consent to the terms and conditions.
4. Utilize face recognition technology with liveness detection for KYC verification.
5. Receive email notifications at the registered email address.

Operating Environment

The platform is an online application designed to serve users who have a web browser and a reliable internet connection

8.1.3 Assumption and Dependencies

1. Users must have a laptop or desktop equipped with a camera and internet access.
2. Users should have installed web browsers such as Chrome, Safari, or Firefox to access the web application.
3. Users will need to upload a clear and correct citizenship document for OCR to accurately extract data.
4. Users should be in a well-lit environment for face recognition and must carefully follow the provided instructions.
5. The personal information of the user will be stored within the system.
6. The email address supplied by the user during registration must be valid for receiving email notifications.
7. Users must agree to the privacy terms and conditions, granting permission for access to their live location and device's web camera.

8.1.4 External Interface Requirements

User Interfaces.

The user interface for the web app system is to be developed using the technology HTML, CSS, bootstrap and tailwind CSS. The appropriate color combination and proper design layout of the element should be used for a better customer experience. The system will not meet resolution and screen layout requirements for mobile devices.

Hardware Interfaces

The web application's user interface will be designed utilizing HTML, CSS, Bootstrap, and Tailwind CSS. A suitable color scheme and an effective layout of the elements should be implemented to enhance the user experience. The system will not comply with the resolution and screen layout specifications for mobile devices.

Software Interfaces

- 1. Frontend Technology:** A user-friendly interface for the system will be developed using HTML, CSS, Bootstrap, and Tailwind CSS.
- 2. Backend Technology:** Python Django will handle database connections, API creation, user authentication, and email notifications.
- 3. Integrated Development Environment:** Visual Studio Code will be utilized for coding the necessary components of the project.
- 4. Database:** User data will be stored using a MySQL Relational Database.
- 5. Version Control:** Git and GitHub will be employed to monitor changes made to the project's source code, with git also used for saving the source code in cloud repositories.
- 6. Backup Solution:** OneDrive will serve as a backup for the project to safeguard against data loss due to failures.
- 7. Pre-trained Model:** The Tesseract OCR Engine will be utilized for extracting details from uploaded citizenship documents, while Face Net will be employed for facial recognition.

8.1.5 System Features

User Registration and Login

To utilize the system's features, users need to sign up, confirm their new account, and log in with the verified information. There is no need for admin registration; users can access the system dashboard by inputting the appropriate admin credentials.

Stimulus/Response Sequences for Normal User

Stimulus: Users are granted access to the user registration page.

Response: The system presents the required input fields for registration.

Stimulus: The user fills in all the necessary details and clicks submit.

Response: The system dispatches a confirmation email to the user's email address.

Stimulus: The user confirms their email address through the validation link.

Response: The system redirects the user to the login page.

Stimulus: The user enters their credentials in the fields provided on the login page.

Response: The system checks the entered credentials and redirects the user to the homepage.

Stimulus/Response Sequences for Admin

Stimulus: The administrator goes to the login pages.

Response: The system directs the administrator to the login page.

Stimulus: The administrator enters their credentials in the necessary fields and clicks the submit button.

Response: The system checks the administrator's credentials and takes them to the dashboard.

Functional Requirements

The system must offer registration and sign-up forms for new users, as well as login forms for both existing users and administrators to access the system.

REQ-1: Email, Password and Contact information.

Registration forms must contain fields for email, password, contact number, and a re-entered password, which will be validated by the system. If the user details are accurate and the email

confirmation has been completed by the user, then the registration process is finalized. For administrators, their credentials are pre-established by the system.

REQ-2: Admin Email and password

To access the admin dashboard, the appropriate admin credentials must be input on the login page.

User detail form automation through OCR

Following system login, the user uploads their citizenship for OCR processing, which extracts and automatically fills in the mapping input fields with the necessary information for forming an account. If the data extraction is inaccurate, there is a way to verify and adjust the automated details.

Stimulus/Response Sequences

Stimulus: The user uploads their citizenship document in the system through the upload interface.

Response: The system validates the document whether it is a citizenship or not extracts the required data like permanent address, citizenship number, date of birth etc. from the document and fills it in the required input field.

Stimulus: The user verifies if the inputted personal details by the system are correct or not and presses the next button if the details are correct.

Response: The system verifies if all the required input fields are not empty and redirects the user to the next page.

Functional Requirement

The system shall extract the required details from the user-uploaded citizenship document and automatically fill the mapped input field.

REQ-1: Clear and well-formatted citizenship document

For form automation through the OCR, the user shall upload a clear citizenship document file. Stimulus: The user grants access to the camera.

In response, the system recognizes the user's face and requests actions from them, such as eye blinking and head movements, to determine their level of vitality.

Stimulus: The user follows the system's motion instructions.

Reaction. After confirming the user's existence, the system uses facial recognition.

A stimulus. If the KYC is validated, the user will be notified by email.

Stimulus: The user grants access to the camera.

In response, the system recognizes the user's face and requests actions from them, such as eye blinking and head movements, to determine their level of vitality.

Stimulus: The user follows the system's motion instructions.

Reaction. After confirming the user's existence, the system uses facial recognition.

A stimulus. If the KYC is validated, the user will be notified by email.

User Location Tracking

The real-time location of the user is tracked if permission is allowed by the user, and the data is stored in the system which is needed for account creation. In case the user location is different then, they can manually enter their correct location.

Stimulus/Response Sequences

Stimulus: The user allows the permission for real-time location tracking.

Response: The system tracks the user's real-time location and asks the user if their location is different or not.

Stimulus: The user manually selects their correct location through map interface if their real-time location does not match.

Response: The system stores the user's location into the system and redirects them to the next page.

Functional Requirement

The system shall track the user's real-time location and store it in the system with the option to manually select location through map interface in case of different location data.

REQ – 1: Allow the access to the user location:

For tracking the user's real-time location, the user shall accept the terms and conditions and allow access to their location to the system.

KYC verification through facial recognition

Verification of the user identity is conducted through facial recognition system by comparing the user photo stored in the system database.

Stimulus/Response Sequences

Stimulus: The user allows permission for camera access.

Response: The system detects the user's face and asks the user to perform motions such as blinking an eye and moving the head for liveness detection.

Stimulus: The user performs the motion activities instructed by the system.

Response: The system verifies whether the user exists and then performs facial recognition.

Stimulus: The user will receive an email notification if the KYC is verified.

Functional Requirement

The system shall perform liveness detection for detecting the user's presence along with facial recognition for KYC verification.

REQ – 1: Allow access to the camera:

The user must provide the system access to their webcam to perform facial recognition and liveness detection.

8.3 APPENDIX C: SAMPLE CODES

8.3.1 SAMPLE CODE OF THE UI

```

documentupload.html M X
webapp > template > documentupload.html > HTML > body > nav.navbar.navbar-expand-lg.navbar-dark.bg-purple > button.navbar-toggler
3  <html lang="en">
4  <head>
5    <style>
74    </style>
75  </head>
76  <body>
77    <!-- Navigation Bar -->
78    <nav class="navbar navbar-expand-lg navbar-dark bg-purple">
79      <a class="navbar-brand" href="#"> Sajilo E-Bank</a>
80      <button
81        class="navbar-toggler"
82        type="button"
83        data-toggle="collapse"
84        data-target="#navbarNav"
85        aria-controls="navbarNav"
86        aria-expanded="false"
87        aria-label="Toggle navigation"
88      >
89        <span class="navbar-toggler-icon"></span>
90      </button>
91      <div class="collapse navbar-collapse" id="navbarNav">
92        <ul class="navbar-nav ml-auto">
93          <li class="nav-item mr-5">
94            <a class="nav-link" href="{% url 'logout' %}">Home</a>
95          </li>
96          <li class="nav-item mr-5">
97            <a class="nav-link" href="#">Location</a>
98          </li>
99          <li class="nav-item mr-5">
100            <a class="nav-link" href="#">Document Upload</a>
101          </li>
102          <li class="nav-item">
103            <form method="post" action="{% url 'logout' %}" class="form-inline">
104              {% csrf_token %}
105              <button type="submit" class="btn btn-danger">Logout</button>
106            </form>
107          </li>
108        </ul>
109      </div>
110    </nav>
111
112    <div class="container mt-5">
113      {% if success_message %}
114      <div class="alert alert-success" role="alert">
115        {{ success_message }}
116      </div>
117      {% endif %}
118
119      {% if error_message %}
120      <div class="alert alert-danger" role="alert">
121        {{ error_message }}
122      </div>
123      {% endif %}
124    </div>

```

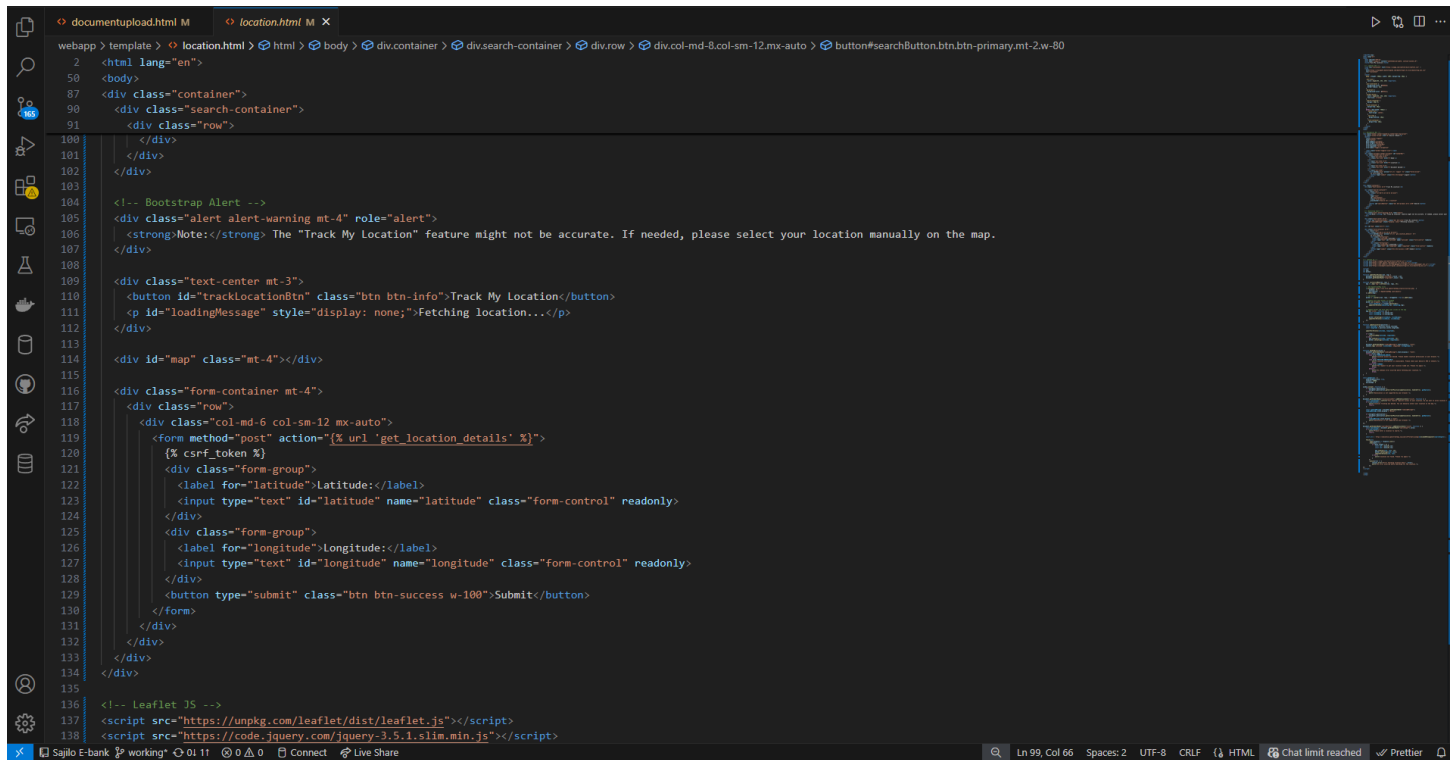
Figure 125 Document upload UI code

```

documentupload.html M  emailverification.html M X
webapp > template > emailverification.html > html > head > style > .verification-card
2  <html lang="en">
3  <head>
12  <style>
18  }
19  .verification-card {
20  max-width: 400px;
21  margin: 80px auto;
22  border-radius: .5rem;
23  box-shadow: 0 4px 12px rgba(0,0,0,0.05);
24  }
25  </style>
26  </head>
27  <body>
28  <div class="container">
29  <div class="card verification-card">
30  <div class="card-body text-center">
31  <h3 class="card-title mb-4">Verify Your Email</h3>
32  <p class="lead">
33  We've sent a verification link to
34  <strong>{{ email }}</strong>.
35  </p>
36  <p class="mb-4">
37  Click the button below once you've received the email.
38  </p>
39  <form method="POST">
40  {{ csrf_token }}
41  <button type="submit" class="btn btn-primary btn-lg btn-block">
42  Verify Email
43  <i class="ml-2 fa fa-envelope-open"></i>
44  </button>
45  </form>
46  </div>
47  </div>
48  </div>
49
50  <!-- Optional JavaScript; choose one of the two! -->
51  <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
52  <script
53  | src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"
54  ></script>
55  <script
56  | src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"
57  ></script>

```

Figure 126 Email Verification UI Code



```
webapp > template > location.html > html > body > div.container > div.search-container > div.row > div.col-md-8.col-sm-12.mx-auto > button#searchButton.btn.btn-primary.mt-2.w-80
2 <html lang="en">
50 </body>
87 <div class="container">
90 <div class="search-container">
91 <div class="row">
100 | </div>
101 </div>
102 </div>
103
104 <!-- Bootstrap Alert -->
105 <div class="alert alert-warning mt-4" role="alert">
106 <strong>Note:</strong> The "Track My Location" feature might not be accurate. If needed, please select your location manually on the map.
107 </div>
108
109 <div class="text-center mt-3">
110 <button id="trackLocationBtn" class="btn btn-info">Track My Location</button>
111 <p id="loadingMessage" style="display: none;">Fetching location...</p>
112 </div>
113
114 <div id="map" class="mt-4"></div>
115
116 <div class="form-container mt-4">
117 <div class="row">
118 <div class="col-md-6 col-sm-12 mx-auto">
119 <form method="post" action="{% url 'get_location_details' %}">
120 <div class="form-group">
121 <input type="text" id="latitude" name="latitude" class="form-control" readonly>
122 </div>
123 <div class="form-group">
124 <input type="text" id="longitude" name="longitude" class="form-control" readonly>
125 </div>
126 <button type="submit" class="btn btn-success w-100">Submit</button>
127 </form>
128 </div>
129 </div>
130 </div>
131 </div>
132 </div>
133 </div>
134 </div>
135
136 <!-- Leaflet JS -->
137 <script src="https://unpkg.com/leaflet/dist/leaflet.js"></script>
138 <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
```

Figure 127 location tracking UI Code

```

2 <html lang="en">
3 <head>
4   <style>
5     @media (max-width: 768px) {
6       .container {
7         padding: 10px;
8       }
9     }
10  </style>
11 </head>
12 <body>
13   <div class="container">
14     <div class="row main">
15       <div class="col-md-6 form-section">
16         <h2 class="text-center">Welcome to Sajilo Ebank!</h2><br>
17         <h3 class="text-center">Login</h3>
18         <form method="POST">
19           { csrf_token %}
20           <div class="mb-3">
21             <label for="email" class="form-label">Email:</label>
22             <input type="text" id="email" name="email" class="form-control" placeholder="Enter your email" required>
23           </div>
24           <div class="mb-3">
25             <label for="password" class="form-label">Password:</label>
26             <input type="password" id="password" name="password" class="form-control" placeholder="Enter your password" required>
27           </div>
28           <button type="submit" class="btn btn-primary w-100">Submit</button>
29         </form>
30         <p class="text-center mt-3">Not registered yet ? <a href="{% url 'register' %}" class="text-decoration-none">Create an account</a></p>
31         <p class="text-center mt-3">Forgot your password ? <a href="{% url 'send-password-reset' %}" class="text-decoration-none">Reset password</a></p>
32         {% if error %}
33         <div class="alert alert-danger alert-dismissible fade show" role="alert">
34           {{ error }}
35           <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
36         </div>
37         {% endif %}
38       </div>
39       <div class="col-md-6 image-section">
40         <h2>lets get started!</h2>
41       </div>
42     </div>
43   </div>
44   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
45 </body>

```

Figure 128 Login page UI code

Figure 129 Password reset UI code

```

2 <html lang="en">
3 <head>
4 <style>
122 </style>
123 </head>
124 <body>
125 <div class="main-container">
126 <div class="image-section">
127 <h2>Welcome to Sajilo Ebank!</h2>
128 <p>Fast, secure, and easy banking services.</p>
129 </div>
130
131 <div class="registerform-section">
132 <h1>Create an Account</h1>
133 <form method="post">
134 <%= csrf_token %>
135 <label for="email">Enter your email</label>
136 <input type="text" id="email" name="email" placeholder="Enter your email" value="{% email|default:'' %}" required>
137
138 <label for="phone">Enter your phone number</label>
139 <input type="text" id="phone" name="phone" placeholder="Enter your phone number" value="{% phone|default:'' %}" required>
140
141 <label for="password">Enter your password</label>
142 <input type="password" id="password" name="password" placeholder="Enter your password" required>
143
144 <label for="repassword">Re-enter password</label>
145 <input type="password" id="re-password" name="re-password" placeholder="Re-enter your password" required>
146
147 <button type="submit">Sign Up</button>
148
149 <p>Already have an account? <a href="{% url 'login' %}">Login now</a></p>
150 <%= if error %>
151 <div class="alert alert-danger alert-dismissible fade show" role="alert">
152 <%= error %>
153
154 </div>
155 <%= endif %>
156
157 <%= if message %>
158 <div class="alert alert-success alert-dismissible fade show" role="alert">
159 <%= message %>
160
161 </div>

```

Figure 130 register page UI Code

```

webapp > template > reset.html > html > body.bg-light > div.container > div.row.justify-content-center.mt-5 > div.col-md-6.col-lg-5.bg-light > div.card.shadow > div.card-header.bg-purple.text-white.text-center
2
<html lang="en">
18
<body class="bg-light">
19
  <div class="container">
20
    <div class="row justify-content-center mt-5">
21
      <div class="col-md-6 col-lg-5 bg-light">
22
        <div class="card shadow">
23
          <div class="card-header bg-purple text-white text-center">
24
            <h4>Password Reset</h4>
25
          </div>
26
          <div class="card-body p-4">
27
            <p class="text-muted mb-4">Enter your email address and we'll send you a link to reset your password.</p>
28
            <form method="POST">
29
              {%- csrf_token %}
30
              <div class="form-floating mb-3 h-75">
31
                <input type="email" class="form-control" id="email" name="email" placeholder="name@example.com" required>
32
                <label for="email">Email address</label>
33
              </div>
34
              <div class="d-grid gap-2 w-100 p-3">
35
                <button type="submit" class="btn btn-primary w-60">
36
                  <i class="bi bi-envelope me-2"></i>
37
                  Reset Link
38
                </button>
39
              </div>
40
            </form>
41
          </div>
42
          <div class="card-footer text-center py-3">
43
            <a href="{% url 'login' %}" class="text-decoration-none" style="color: #6f42c1;">
44
              <i class="bi bi-arrow-left-circle me-1"></i>
45
              Back to login
46
            </a>
47
          </div>
48
        </div>
49
      </div>
50
    </div>
51
    <div class="text-center mt-4">
52
      <div class="py-2 px-3 bg-light border rounded-3 d-inline-block shadow-sm" style="border-color: #6f42c1 !important;">
53
        <i class="bi bi-info-circle text-purple me-2"></i>
54
        <small class="text-muted" style="color: #6f42c1 !important;">If you don't receive an email, please check your <span class="text-purple fw-semibold">spam folder</span>.</small>
55
      </div>
56
    </div>
57
  </div>
58
</body>
59
</html>
60

```

Figure 131 Reset password link UI code

```

webapp > template > dashboard.html > html > body > div.container.dashboard > div.card.shadow-sm > div.card-body > div.table-responsive > table.table.table-bordered.table-hover > tbody > tr > td
2   <html lang="en">
55  <body>
63   <div class="container dashboard">
107   <div class="card shadow-sm">
108     <div class="card-header bg-purple text-white">
109       <h5 class="mb-0">
110         <i class="bi bi-people-fill me-2"></i>User Details
111       </h5>
112     </div>
113     <div class="card-body">
114       <div class="table-responsive">
115         <table class="table table-bordered table-hover">
116           <thead>
117             <tr>
118               <th>No.</th>
119               <th>User Id</th>
120               <th>Email</th>
121               <th>Phone number</th>
122               <th>Account creation date</th>
123               <th>Email Verified Status</th>
124               <th>Document Verified Status</th>
125               <th>Last Login</th>
126               <th>Login Count</th>
127             </tr>
128           </thead>
129           <tbody>
130             {% for user in users %}
131             <tr>
132               <td>{{ forloop.counter }}</td>
133               <td>{{ user.user_id }}</td>
134               <td>{{ user.email }}</td>
135               <td>{{ user.phone_number }}</td>
136               <td>{{ user.account_created_date }}</td>
137               <td>{{ user.is_email_verified }}</td>
138               <td>{{ user.is_document_valid }}</td>
139               <td>{{ user.last_login }}</td>
140               <td>{{ user.login_count }}</td>
141             </tr>
142             {% empty %}
143             <tr>
144               <td colspan="9" class="text-center text-muted">No users found.</td>
145             </tr>
146             {% endfor %}

```

Figure 132 Dashboard UI code


```

2 <html lang="en">
48 <body>
50 <nav class="navbar navbar-expand-lg navbar-dark bg-purple">
51 <div class="container-fluid">
64 <div class="collapse navbar-collapse" id="navbarNav">
80 </form>
81 </li>
82 </ul>
83 </div>
84 </div>
85 </nav>
86
87 <div class="container mt-5">
88 <div class="row justify-content-center">
89 <div class="col-lg-8 col-md-10 col-sm-12">
90 <h2 class="mb-4 text-center text-lg-start">Citizenship Information Form</h2>
91 <form class="needs-validation" novalidate>
92 <div class="mb-3">
93 <label for="citizenshipNumber" class="form-label">Citizenship Number</label>
94 <input type="text" class="form-control" id="citizenshipNumber" placeholder="Enter your citizenship number" required>
95 </div>
96
97 <div class="mb-3">
98 <label for="fullName" class="form-label">Full Name</label>
99 <input type="text" class="form-control" id="fullName" placeholder="Enter your full name" required>
100 </div>
101
102 <div class="mb-3">
103 <label class="form-label">Gender</label>
104 <select class="form-select" id="gender" required>
105 <option value="" selected disabled>Select your gender</option>
106 <option value="Male">Male</option>
107 <option value="Female">Female</option>
108 <option value="Other">Other</option>
109 </select>
110 </div>
111
112 <div class="mb-3">
113 <label for="dob" class="form-label">Date of Birth</label>
114 <input type="date" class="form-control" id="dob" required>
115 </div>
116
117 <div class="mb-3">
118 <label for="placeOfBirth" class="form-label">Place of Birth</label>

```

Figure 133 Citizenship detail form UI code

```

webapp > template > < location.html > < html > < body > < div.container > < div.search-container > < div.row > < div.col-md-8.col-sm-12.mx-auto > < button#searchButton.btn.btn-primary.mt-2.w-80
2 <html lang="en">
50 <body>
87 <div class="container">
90 <div class="search-container">
91 <div class="row">
93 <input
94 type="text"
95 id="searchInput"
96 class="form-control"
97 placeholder="Search for a location"
98 />
99 <button id="searchButton" class="btn btn-primary mt-2 w-80">Search</button>
100 </div>
101 </div>
102 </div>
103
104 <!-- Bootstrap Alert -->
105 <div class="alert alert-warning mt-4" role="alert">
106 <strong>Note:</strong> The "Track My Location" feature might not be accurate. If needed, please select your location manually on the map.
107 </div>
108
109 <div class="text-center mt-3">
110 <button id="trackLocationBtn" class="btn btn-info">Track My Location</button>
111 <p id="loadingMessage" style="display: none;">Fetching location...</p>
112 </div>
113
114 <div id="map" class="mt-4"></div>
115
116 <div class="form-container mt-4">
117 <div class="row">
118 <div class="col-md-6 col-sm-12 mx-auto">
119 <form method="post" action="{% url 'get_location_details' %}">
120 {%- csrf.token %}
121 <div class="form-group">
122 <label for="latitude">Latitude:</label>
123 <input type="text" id="latitude" name="latitude" class="form-control" readonly>
124 </div>
125 <div class="form-group">
126 <label for="longitude">Longitude:</label>
127 <input type="text" id="longitude" name="longitude" class="form-control" readonly>
128 </div>
129 <button type="submit" class="btn btn-success w-100">Submit</button>

```

Figure 134 Location UI code

```

webapp > template > < kyc.html > < html > < head > < style > < .webcam-container
2 <html lang="en">
3 <head>
9 <style>
16 .webcam-container {
18 background-color: #ffffff;
19 }
20 </style>
21 </head>
22 <body>
23
24 <div class="container py-5">
25 <h1 class="text-center header mb-4">📹 Live Webcam Feed</h1>
26
27 <div class="text-center">
28 <div class="p-4">
29 
30 </div>
31 </div>
32 </div>
33 </div>
34
35
36 <!-- Optional Bootstrap JS -->
37 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
38 </body>
39 </html>
40

```

A space-separated list of the classes of the element. Classes allows CSS and JavaScript to select and access specific elements via the **class selectors** or functions like the method `Document.getElementsByClassName()`.

[MDN Reference](#)

Figure 135 Video capture UI code

8.3.2 SAMPLE CODE OF THE AUTOMATION SCRIPTS

```
9
10 image = cv2.imread(image_path)
11
12 def grayscale(image):
13     """Converts an image to grayscale.
14     """
15     return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
16 grayscale_image = grayscale(image)
17
18 def denoise(grayscale_image):
19     """Reduces noise in the image using a median blur filter.
20     """
21     return cv2.medianBlur(grayscale_image, 5)
22
23 image_denoise = denoise(grayscale_image)
24
25
26 def sharpen(image):
27     """Sharpens the image using a Laplacian filter.
28     """
29     kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
30     return cv2.filter2D(image, -1, kernel)
31
32 sharpen_image = sharpen(image_denoise)
33 text = pytesseract.image_to_string(grayscale_image)
34 print(text)
35
36
37
```

Figure 136 Text extraction from image code

```
detect_face.py > ...
1  from mtcnn import MTCNN
2  import cv2
3
4  #MTCNN face detector
5  detector = MTCNN()
6
7  # Loading the images of document
8  image = cv2.imread('images/44.jpg')
9
10 # Detect faces
11 faces = detector.detect_faces(image)
12 print(faces)
13 # Extract and save faces
14 for i, face in enumerate(faces):
15     x, y, w, h = face['box']
16     face_image = image[y:y+h+3, x:x+w+3]
17     cv2.imwrite(f'extracted_face_{i}.jpg', face_image)
```

Figure 137 Face detection code

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE

Using cuda device
GPU: Tesla P100-PCIE-16GB
Available memory: 17.06 GB

# Define paths
train_metadata_path = "/kaggle/input/ocr-donut-data/data/train/metadata.jsonl"
val_metadata_path = "/kaggle/input/ocr-donut-data/data/val/metadata.jsonl"
test_metadata_path = "/kaggle/input/ocr-donut-data/data/test/metadata.jsonl"

def resize_image(image, max_size=(384, 384)):
    """Resize image while maintaining aspect ratio"""
    width, height = image.size
    if width > max_size[0] or height > max_size[1]:
        image.thumbnail(max_size, Image.LANCZOS)
    return image

model_name = "naver-clova-ix/donut-base"
processor = DonutProcessor.from_pretrained(model_name)
```

Figure 138 Ocr donut fine tuning note book

```
ocr_test.py > ...
1 from donut import DonutModel
2 from PIL import Image
3 import torch
4
5 # Change the path here:
6 model = DonutModel.from_pretrained("E:/Sajilo E-bank/20250413_044859/20250413_044859")
7 if torch.cuda.is_available():
8     model.half()
9     device = torch.device("cuda")
10    model.to(device)
11 else:
12    model.encoder.to(torch.bfloat16)
13
14 model.eval()
15
16 image = Image.open(r"E:\Sajilo E-bank\images\33.jpg").convert("RGB") # Change here
17 with torch.no_grad():
18     output = model.inference(image=image, prompt="<s_data>")
19
20 print(output)
```

Figure 139 OCR donut testing code

```

data_processing_OCR.py > create_sets
7 def create_sets(df, train=0.7, val=0.2, test=0.1):
10     """
11
12     if not abs(train + val + test - 1.0) < 1e-6:
13         raise ValueError("train + val + test must equal 1.")
14
15     # Define folders
16     folders = {
17         "train": "data/train",
18         "val": "data/val",
19         "test": "data/test"
20     }
21
22     # Create directories
23     for folder in folders.values():
24         os.makedirs(folder, exist_ok=True)
25
26     # Shuffle the DataFrame
27     samples = df.sample(frac=1.0).reset_index(drop=True)
28
29     # Compute split indices
30     n = len(samples)
31     n_train = int(train * n)
32     n_val = int(val * n)
33
34     for idx, row in tqdm(samples.iterrows(), total=n):
35         # Build the nested data structure
36         data = {
37             "citizenship_certificate_no": row["citizenship_certificate_no"],
38             "full_name": row["full_name"],
39             "sex": row["sex"],
40             "date_of_birth": {

```

Figure 140 Ocr donut data processing code

```

0 def detect_and_save_faces(image_path, user_id):
1     # Initialize the MTCNN detector
2     detector = MTCNN()
3
4     # Load the image
5     image = cv2.imread(image_path)
6
7     # Detect faces
8     faces = detector.detect_faces(image)
9     face_urls = []
10
11     # Extract and save faces
12     for i, face in enumerate(faces):
13         x, y, w, h = face['box']
14         face_image = image[y:y+h+3, x:x+w+3]
15         timestamp = datetime.now().strftime('%Y%m%d%H%M%S') # Format: YYYYMMDDHHMMSS
16         face_filename = f"{user_id}_face_{timestamp}_{i}.jpg"
17         face_path = os.path.join(settings.MEDIA_ROOT, 'uploads', 'faces', face_filename)
18         os.makedirs(os.path.dirname(face_path), exist_ok=True)
19         cv2.imwrite(face_path, face_image)
20         face_url = f"{settings.MEDIA_URL}uploads/faces/{face_filename}"
21         face_urls.append(face_url)
22
23     return face_urls

```

Figure 141 detect face and crop the face code

```

def detect_front_side(confidence_scores):
    frontside_class_id = 4 # Assuming 'frontside' is the 5th class (index 4)
    frontside_confidence = [conf for cls, conf in confidence_scores if cls == frontside_class_id]
    other_classes_confidence = [conf for cls, conf in confidence_scores if cls != frontside_class_id]

    if frontside_confidence and max(frontside_confidence) > 0.95:
        if all(conf > 0.80 for conf in other_classes_confidence):
            return True
    return False

def detect_back_side(confidence_scores):
    backside_class_id = 1 # 'backside' is the 2nd class (index 1)
    fingerprint_class_id = 3 # 'fingerprint' is the 4th class (index 3)

    backside_confidence = [conf for cls, conf in confidence_scores if cls == backside_class_id]
    fingerprint_confidence = [conf for cls, conf in confidence_scores if cls == fingerprint_class_id]

    if backside_confidence and max(backside_confidence) > 0.90:
        if fingerprint_confidence and max(fingerprint_confidence) > 0.85:
            return True
    return False

```

Figure 142 Back side and front side validation code

```

def yolo_model_test(model_path, image_path, save_path, is_back_side=False):
    # Loading YOLO model
    model = YOLO(model_path)
    results = model.predict(image_path)
    if isinstance(results, list):
        results = results[0]
    if not os.path.exists(save_path):
        os.makedirs(save_path)
    # Save the processed image
    result_image_path = os.path.join(save_path, os.path.basename(image_path))
    results.save(result_image_path)

    # Confidence scores of the predicted labeled attributes
    confidence_scores = []
    is_valid_document = False

    if hasattr(results, 'boxes') and results.boxes:
        for box in results.boxes:
            class_id = box.cls.item() # Class ID of the detected object
            confidence = box.conf.item() # Confidence score of the detection
            confidence_scores.append((class_id, confidence))

    # Use separate functions for front-side and back-side detection
    if not is_back_side:
        is_valid_document = detect_front_side(confidence_scores)
    else:
        is_valid_document = detect_back_side(confidence_scores)

```

Figure 143 yolo model test code

8.4 APPENDIX D: DESIGNS

8.4.1 GANTT CHART

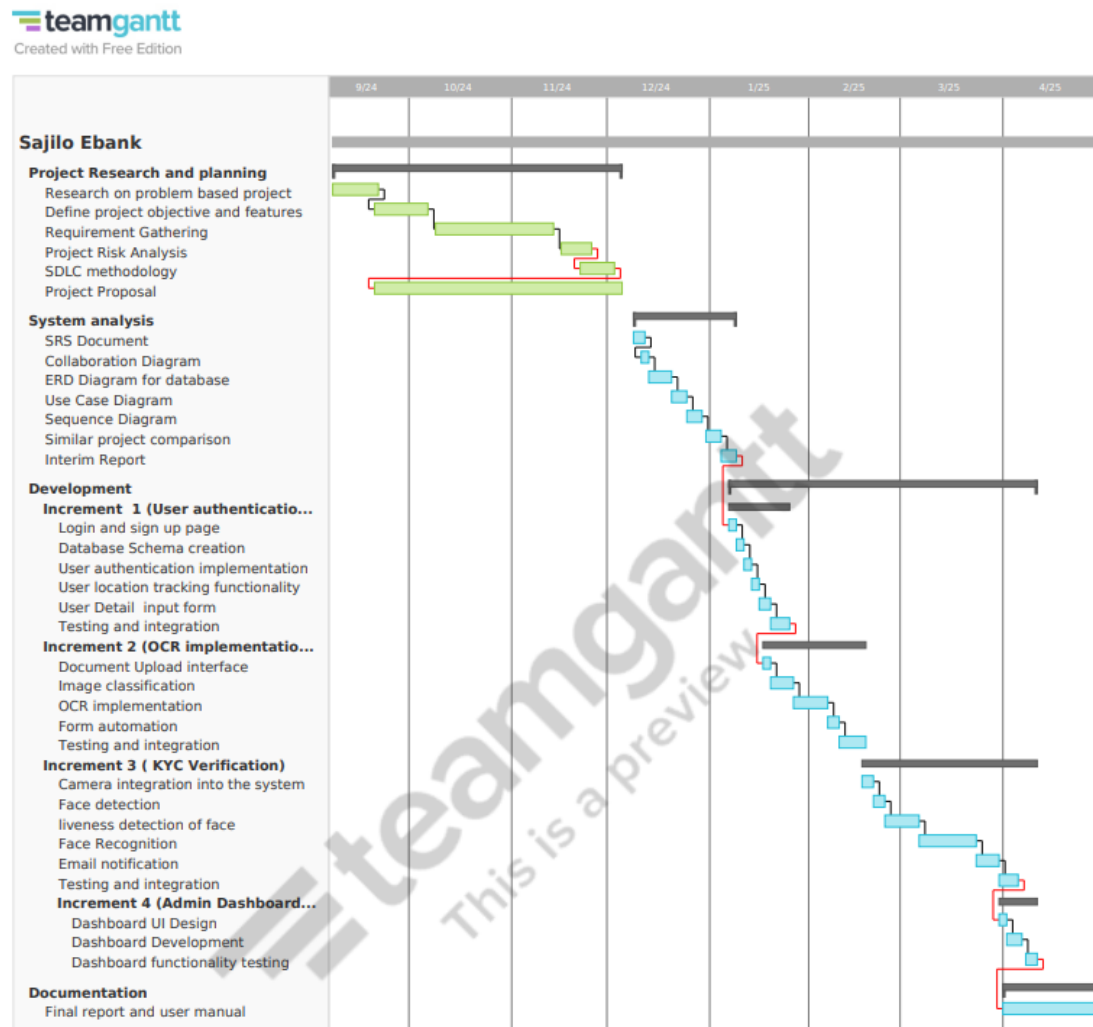


Figure 144 Gantt chart

8.4.2 WORK BREAK DOWN STRUCTURE

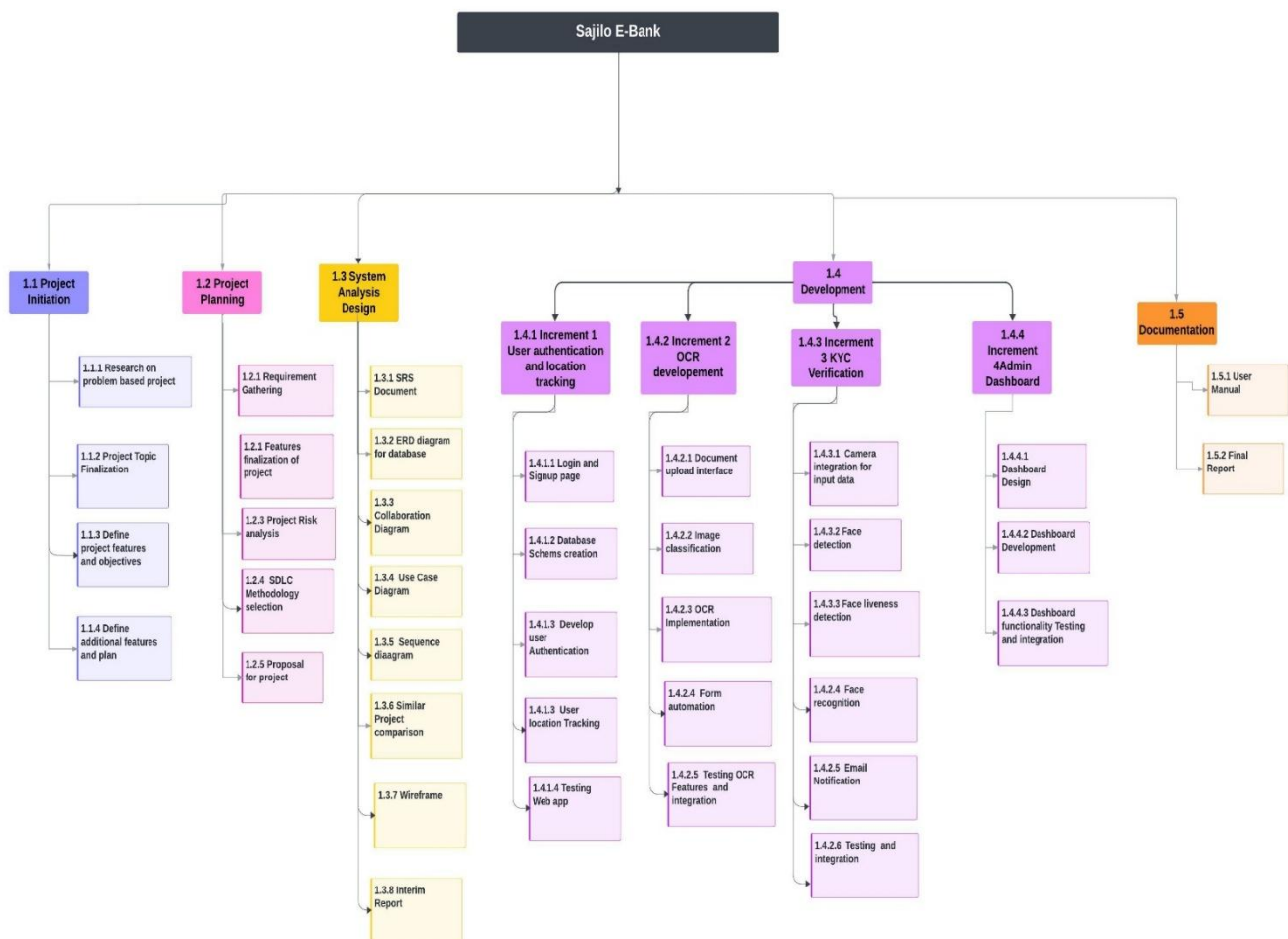


Figure 145 Work Breakdown Structure

8.4.3 MILESTONE CHART

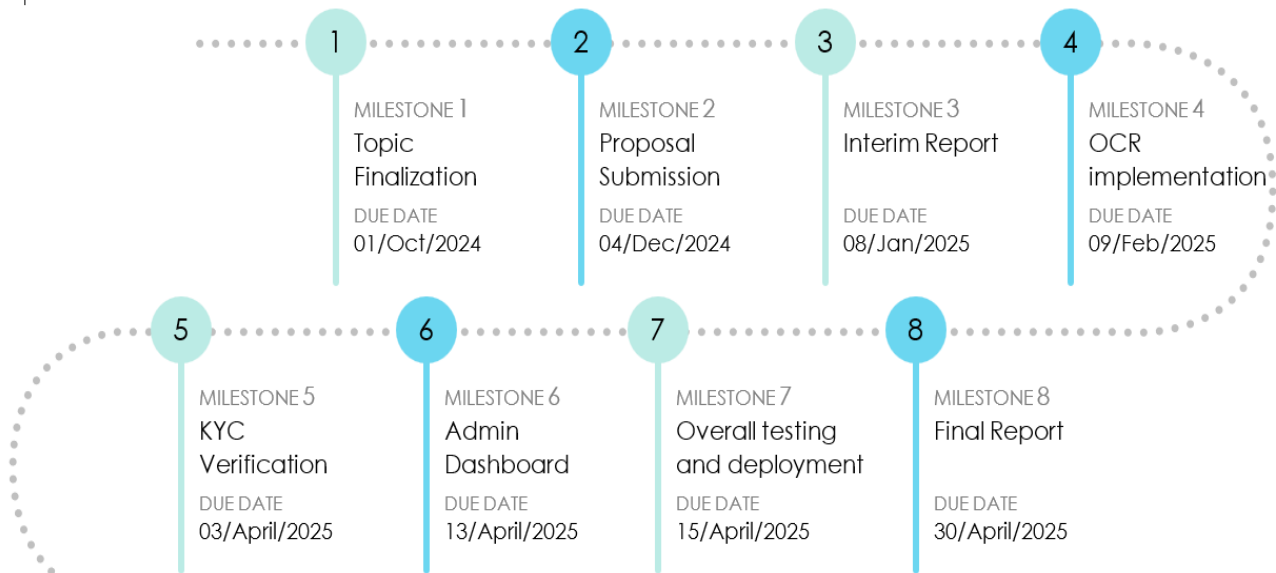


Figure 146 Milestone chart

8.4.4 SYSTEM ARCHITECTURE

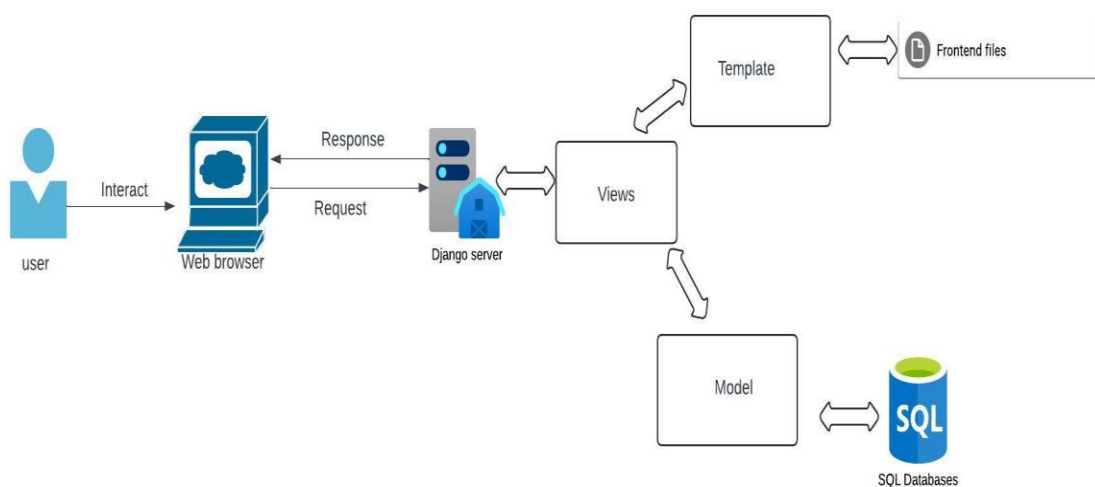
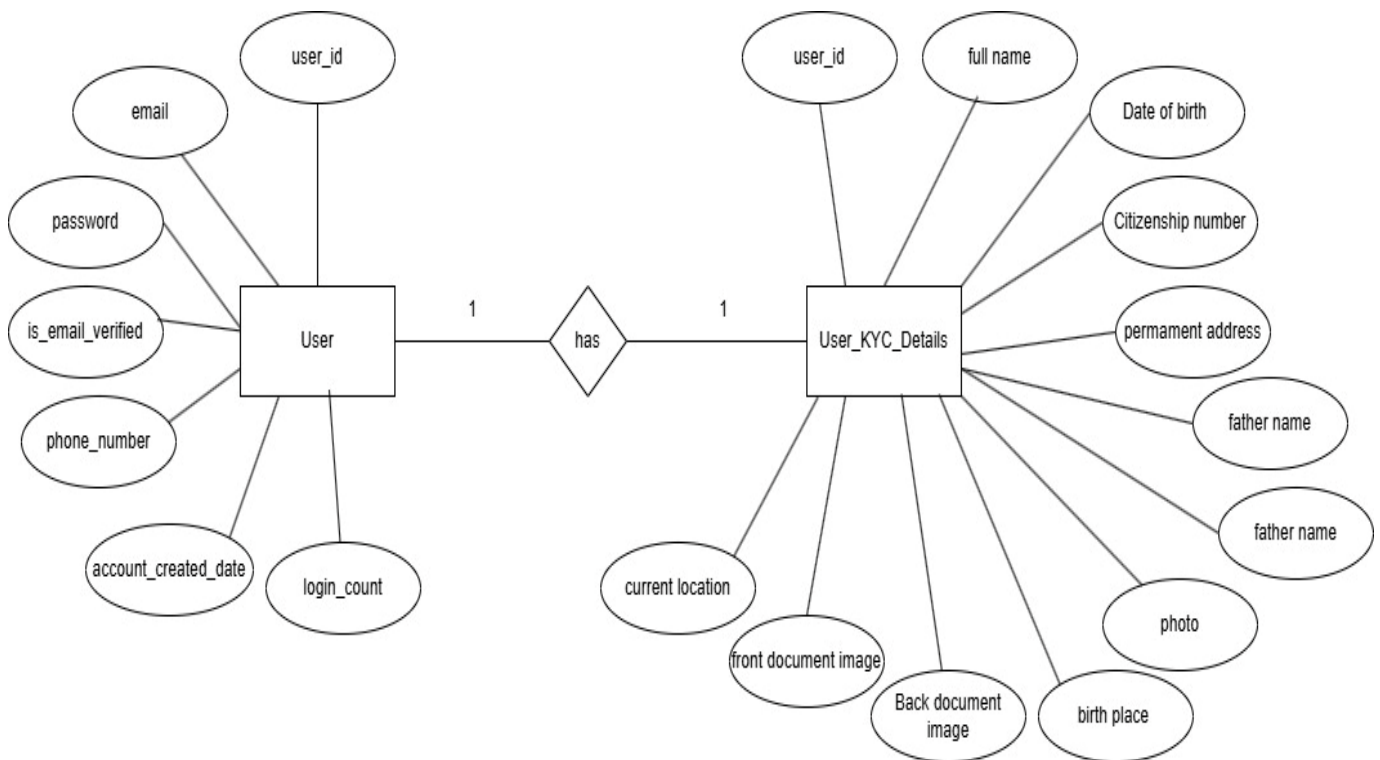


Figure 147 System architecture design

8.4.5 ERD

*Figure 148 Initial ERD diagram*

8.4.6 USE CASE DIAGRAM

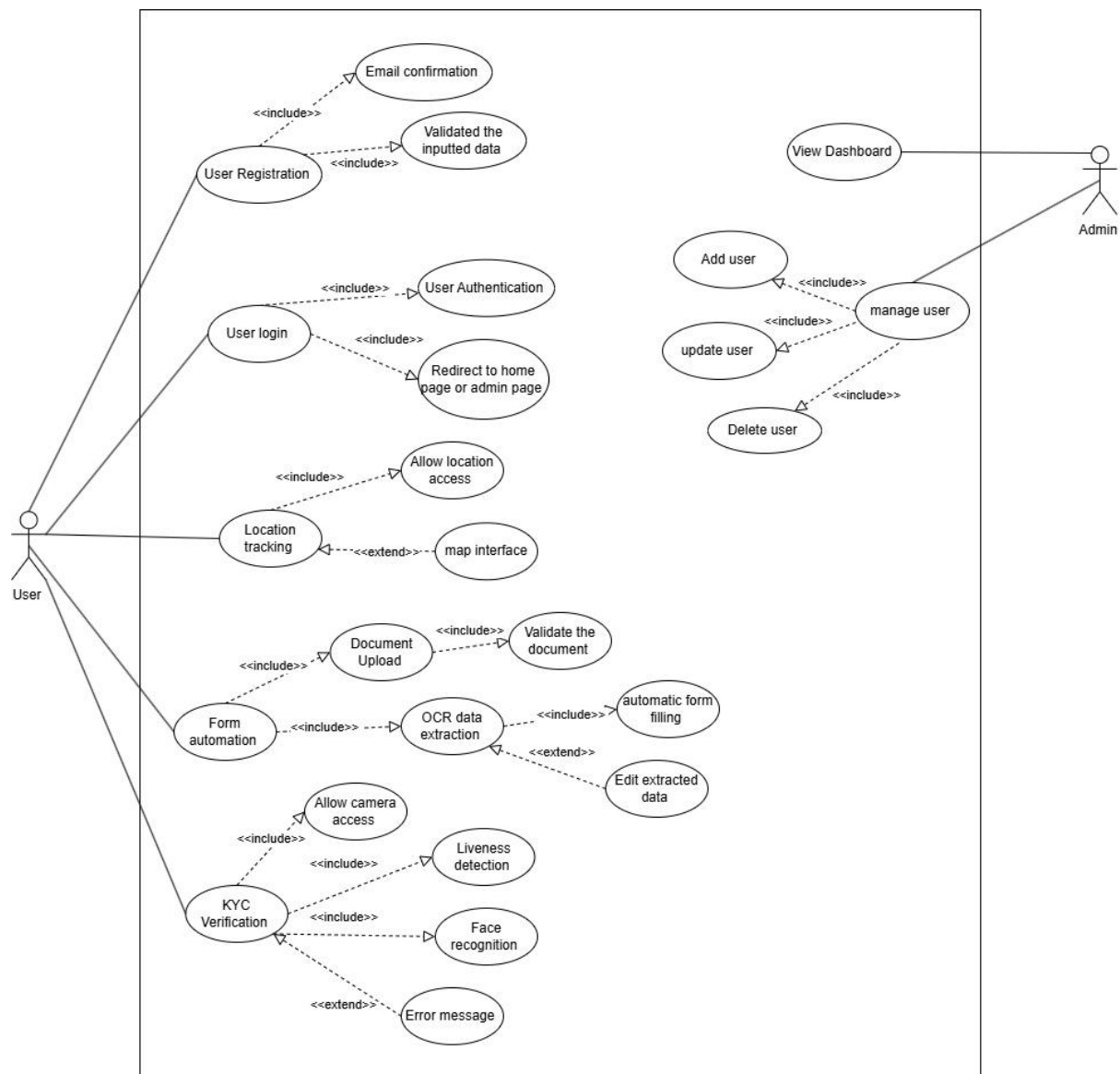


Figure 149 Use case diagram

8.4.7 SEQUENCE DIAGRAM

1) Sequence Diagram of User Registration

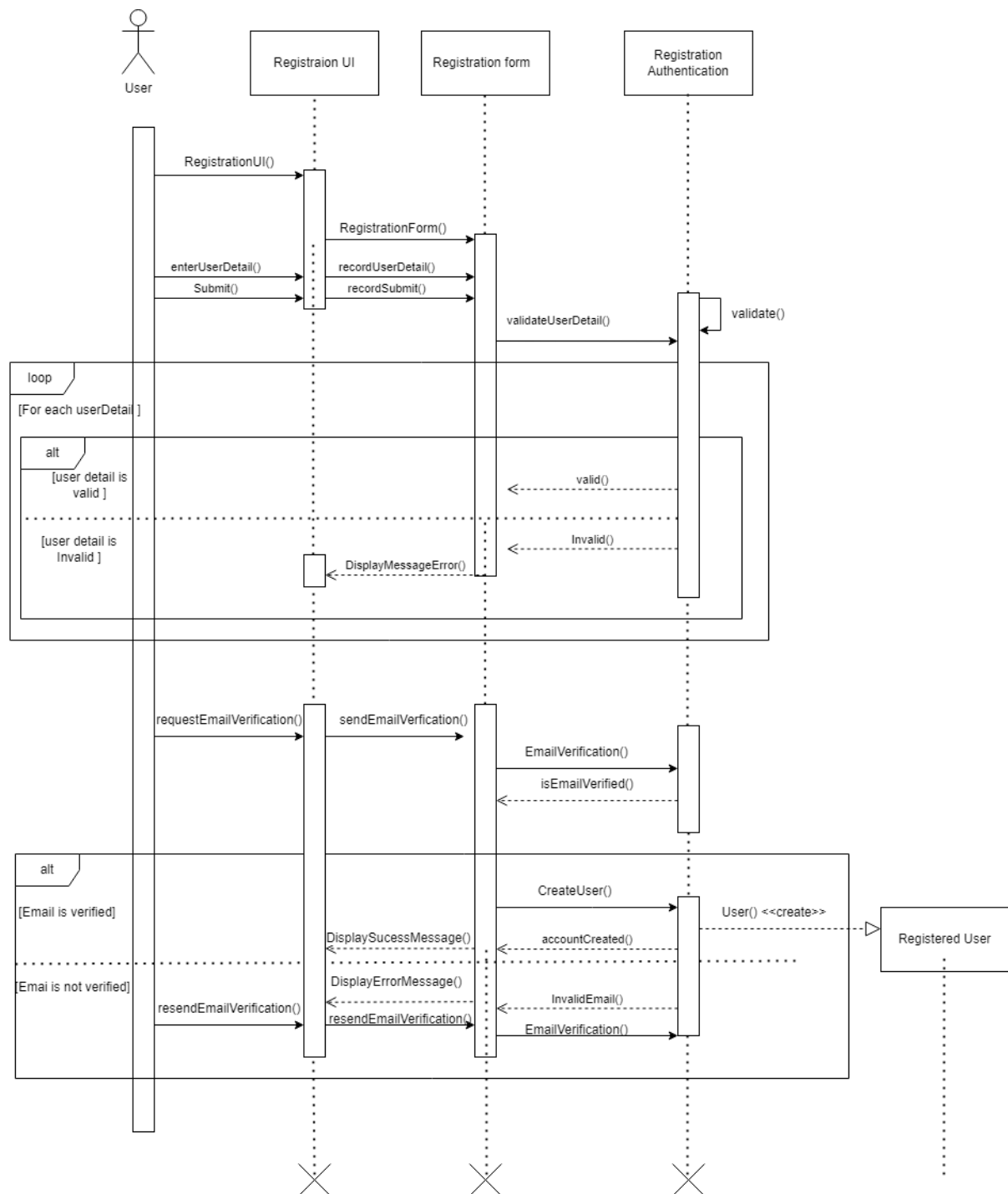


Figure 150 Sequence Diagram for User Registration

2) Sequence Diagram of User login

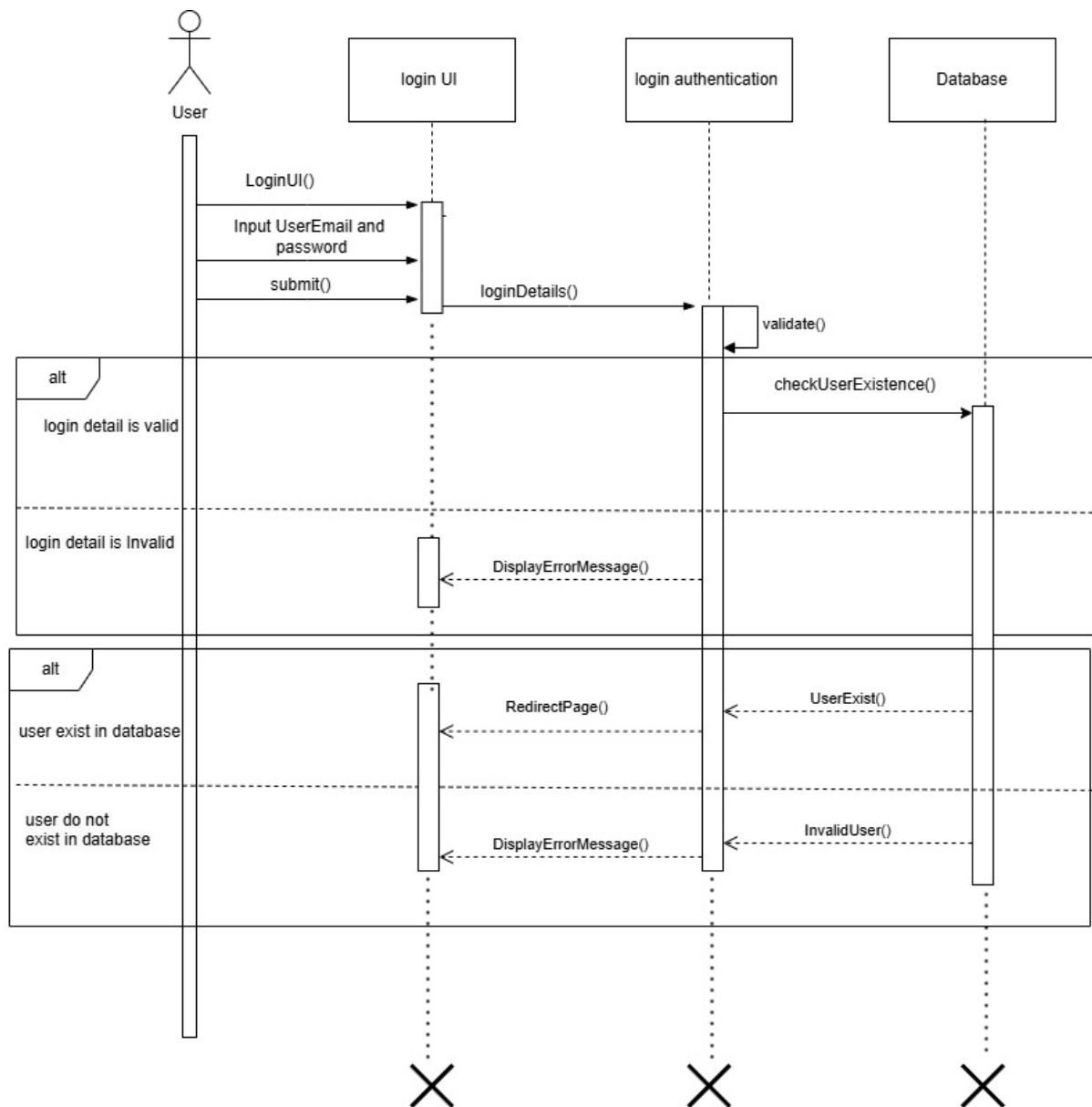


Figure 151 Sequence Diagram of User Login

3) Sequence Diagram of OCR Form Automation

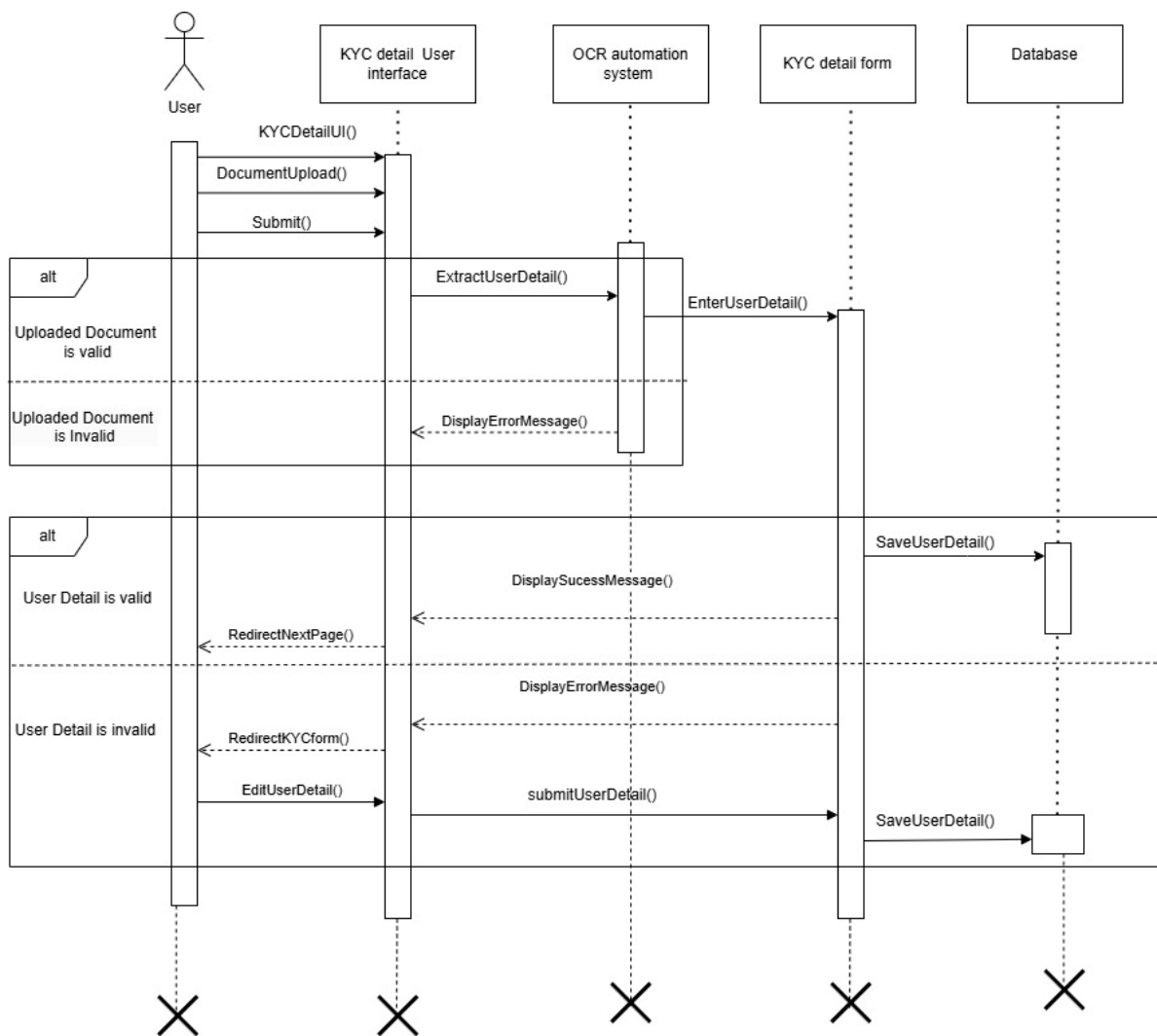


Figure 152 Sequence diagram of OCR form Automation

4) Sequence diagram of User location tracking

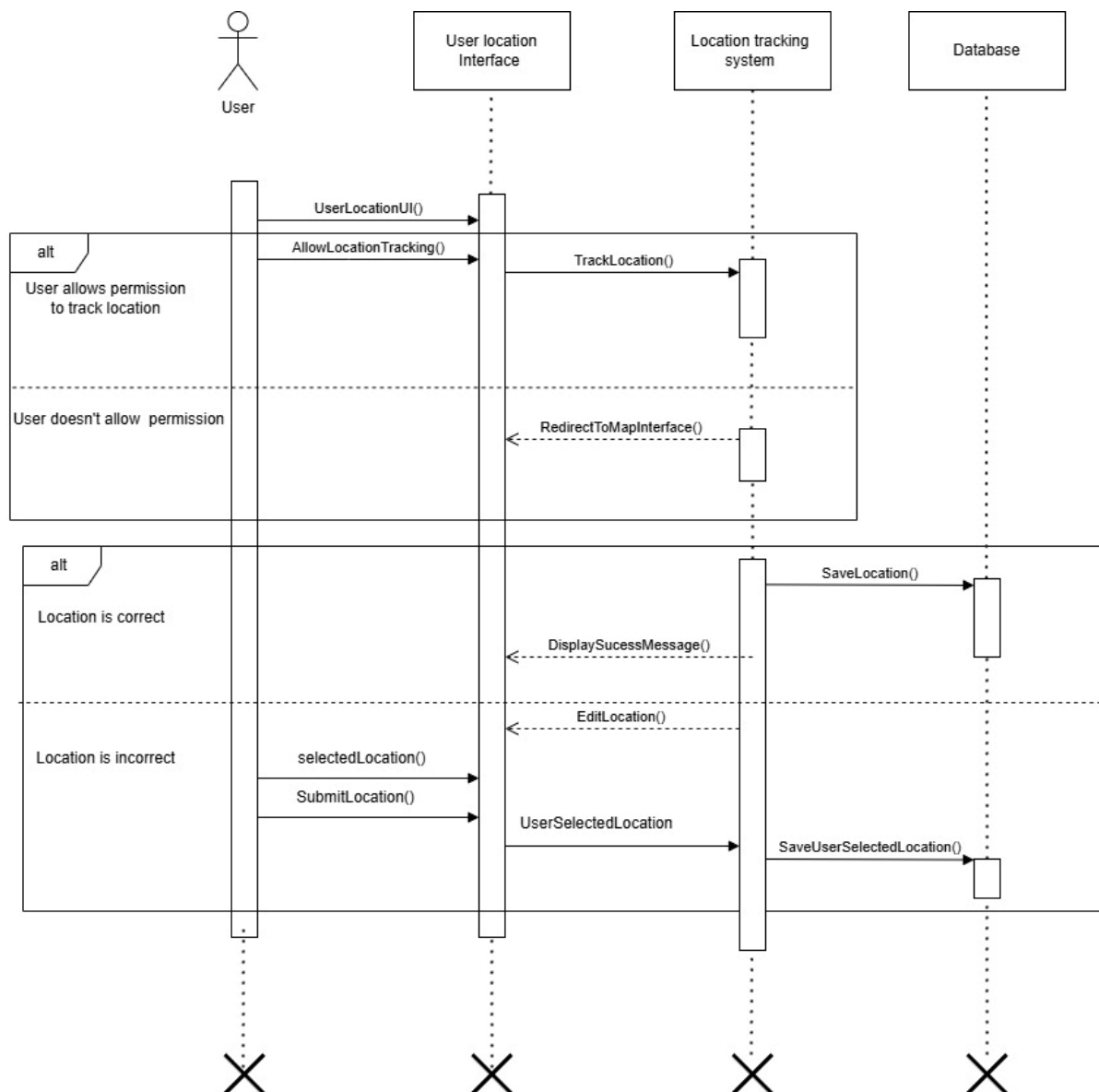


Figure 153 Sequence Diagram of User location Tracking

5) Sequence diagram of KYC verification through Face recognition.

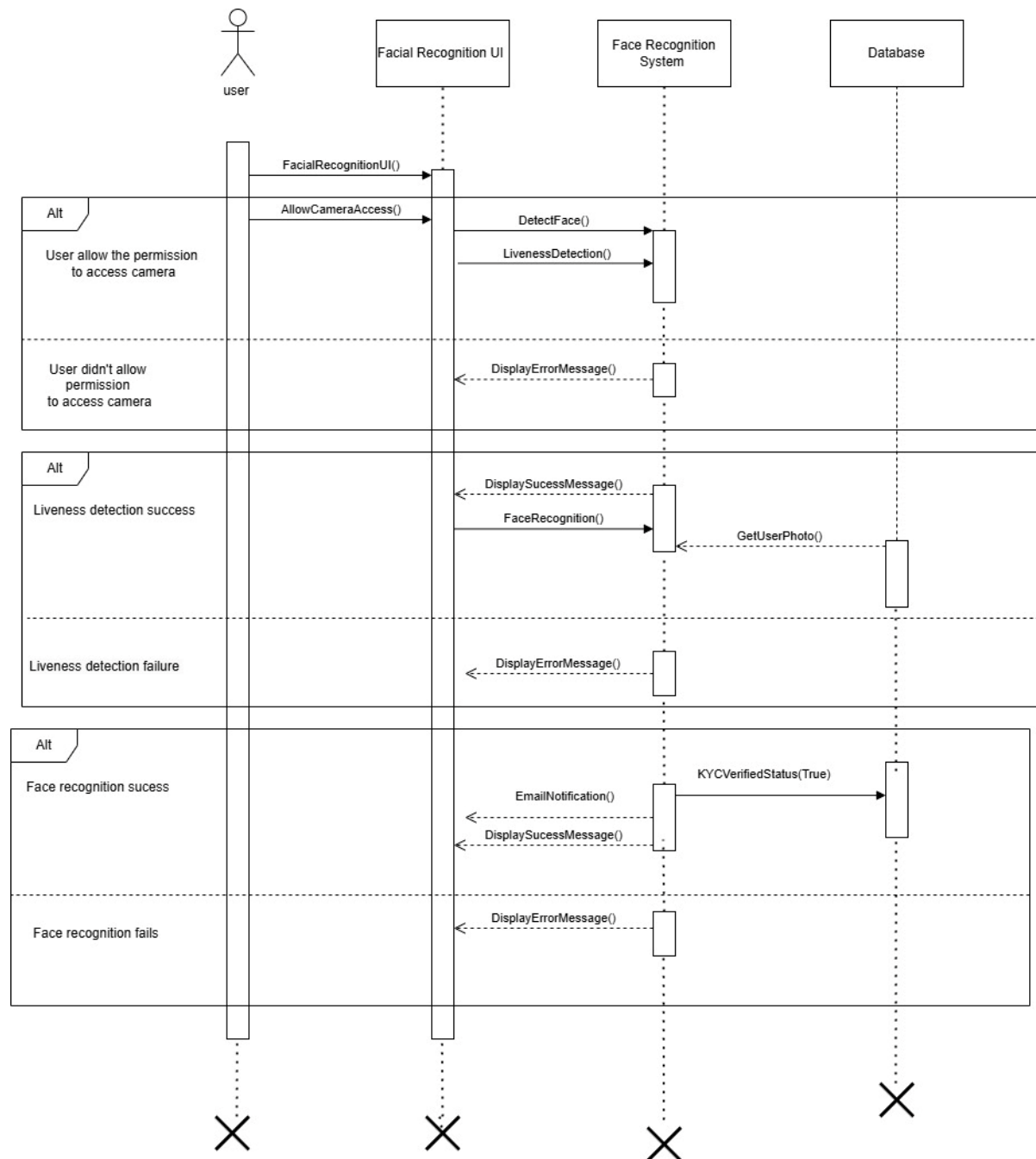


Figure 154 Sequence diagram of KYC Verification through facial recognition

8.4.8 COLLABORATION DIAGRAM

1) Collaboration Diagram of User Registration

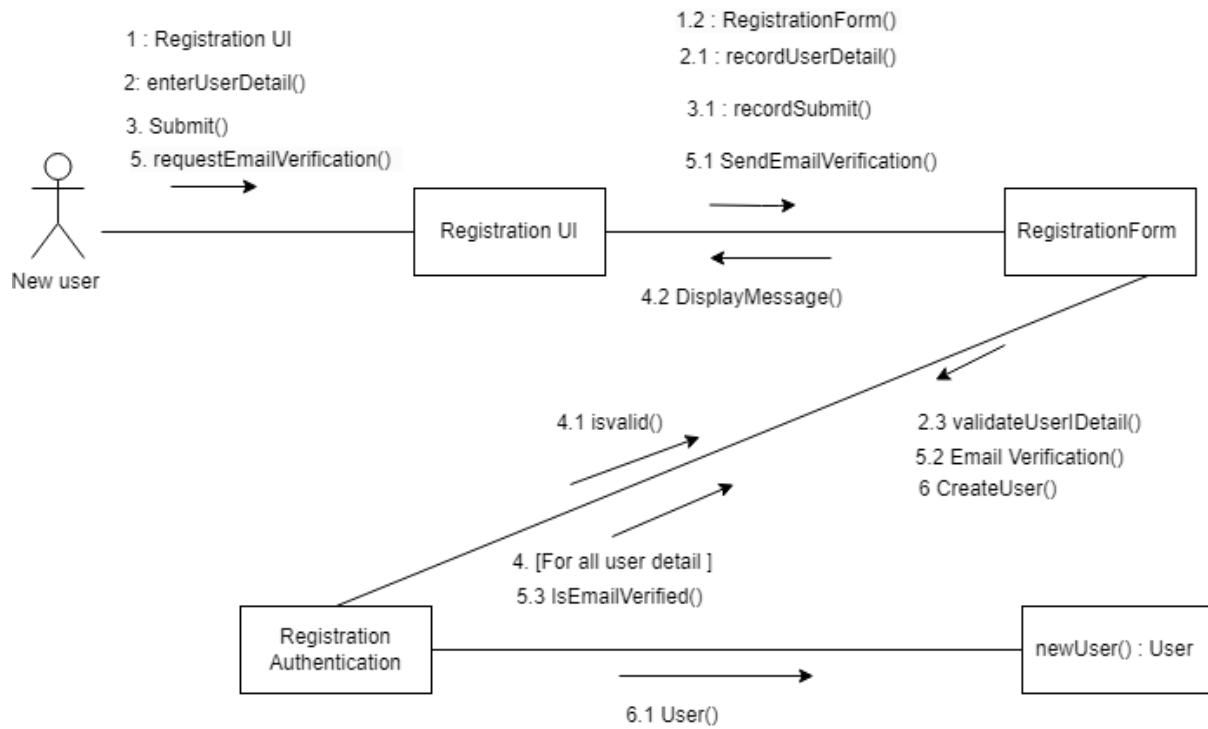


Figure 155 Collaboration Diagram of User Registration

2) Collaboration Diagram of User Login

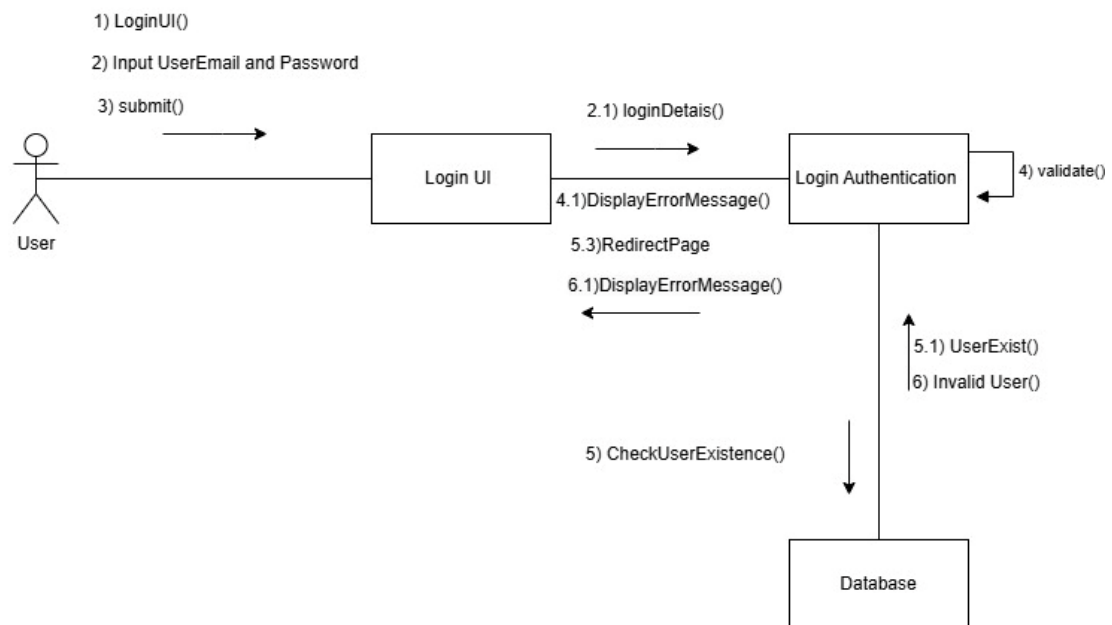


Figure 156 Collaboration Diagram of User Login

3) Collaboration diagram of OCR form automation

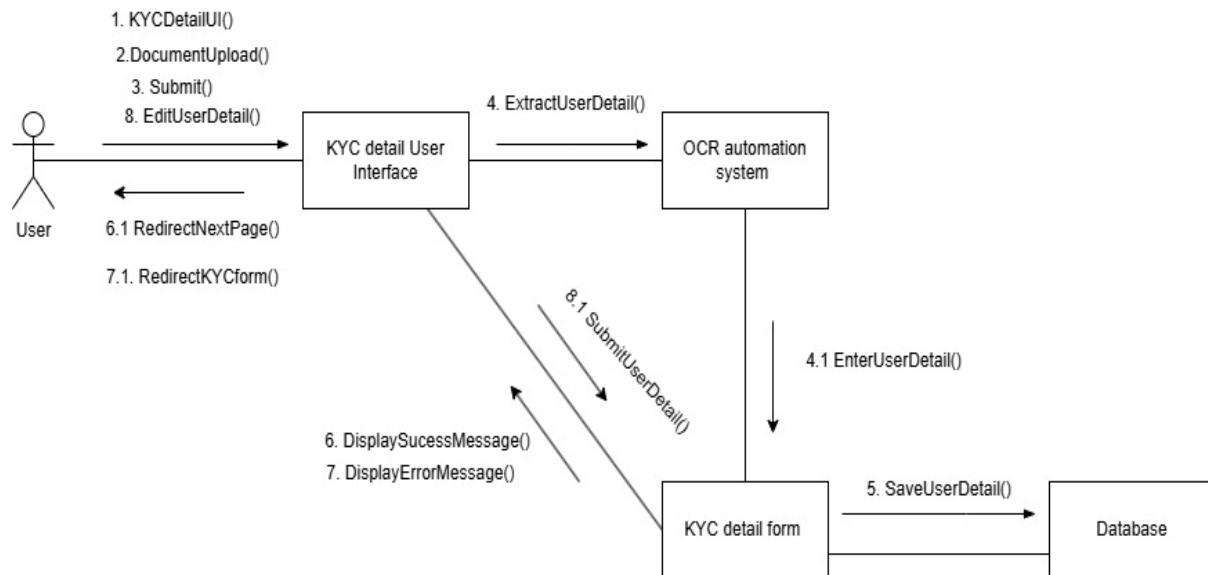


Figure 157 Collaboration Diagram of OCR form automation

4) Collaboration diagram of User location tracking

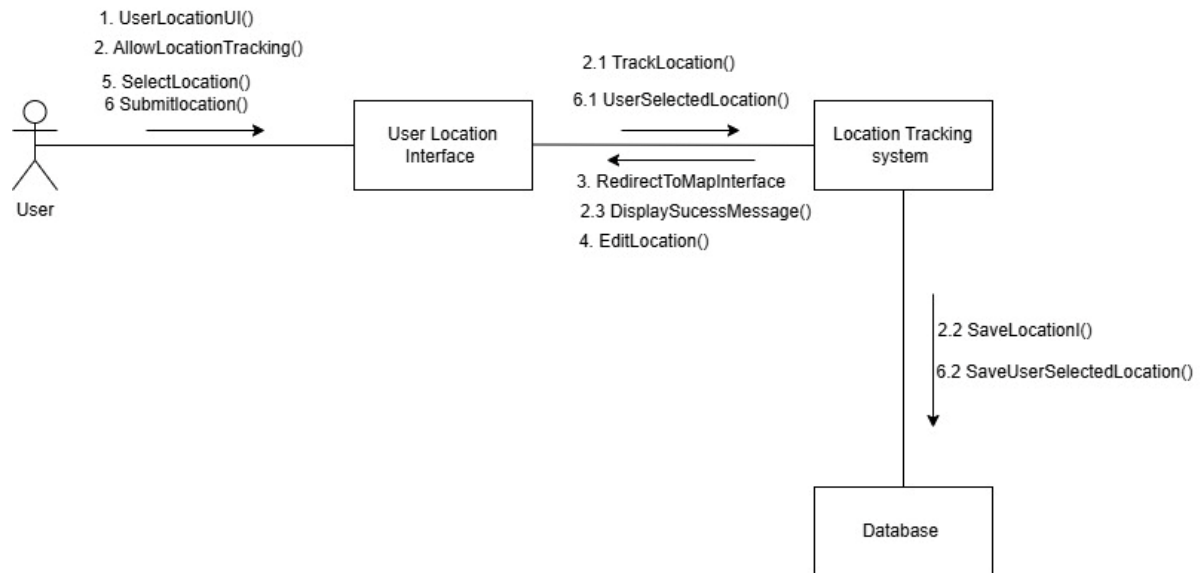


Figure 158 Collaboration Diagram of User location tracking

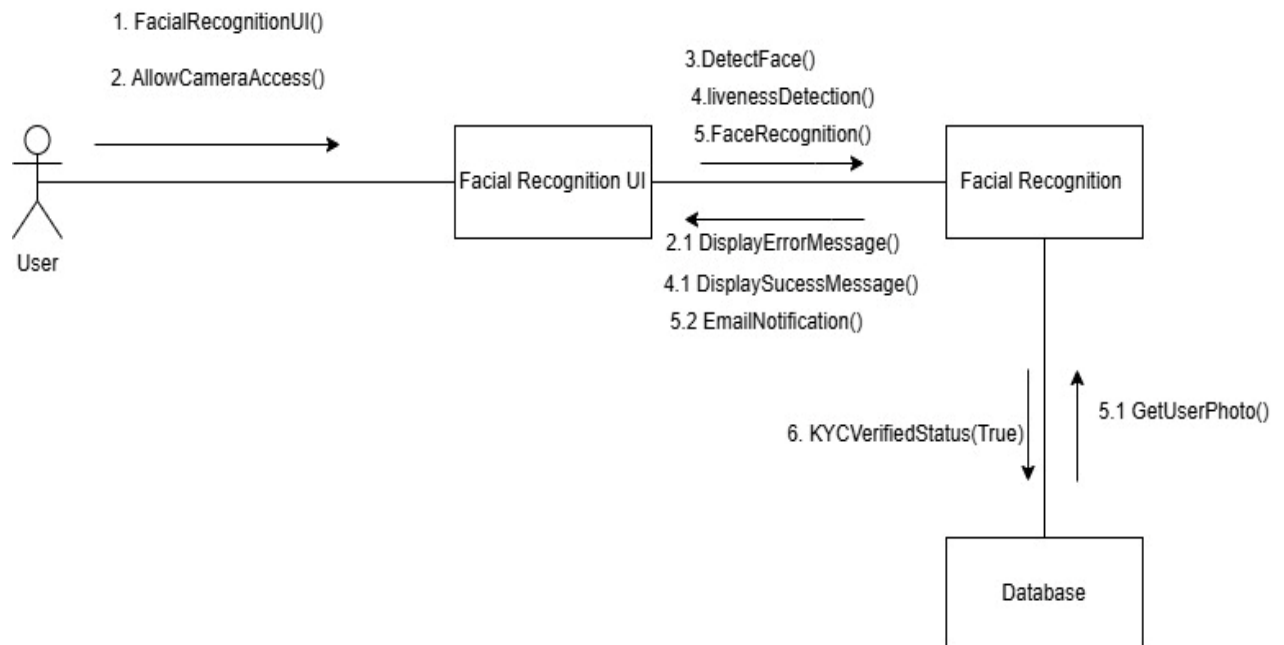
5) Collaboration diagram of KYC verification through facial recognition

Figure 159 Collaboration diagram of KYC verification through facial recognition

8.4.9 FLOWCHART

1) Flow chart of user login

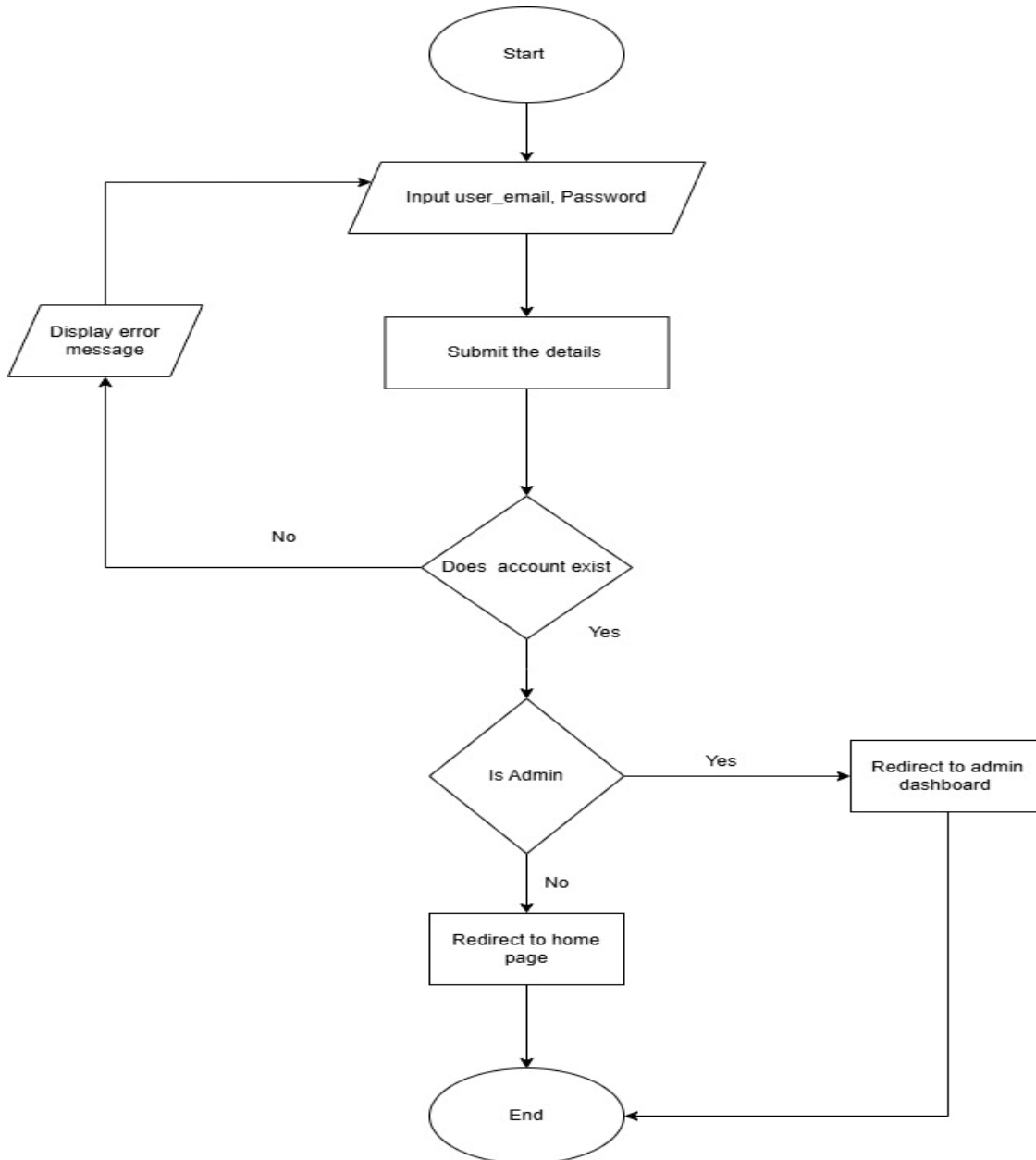
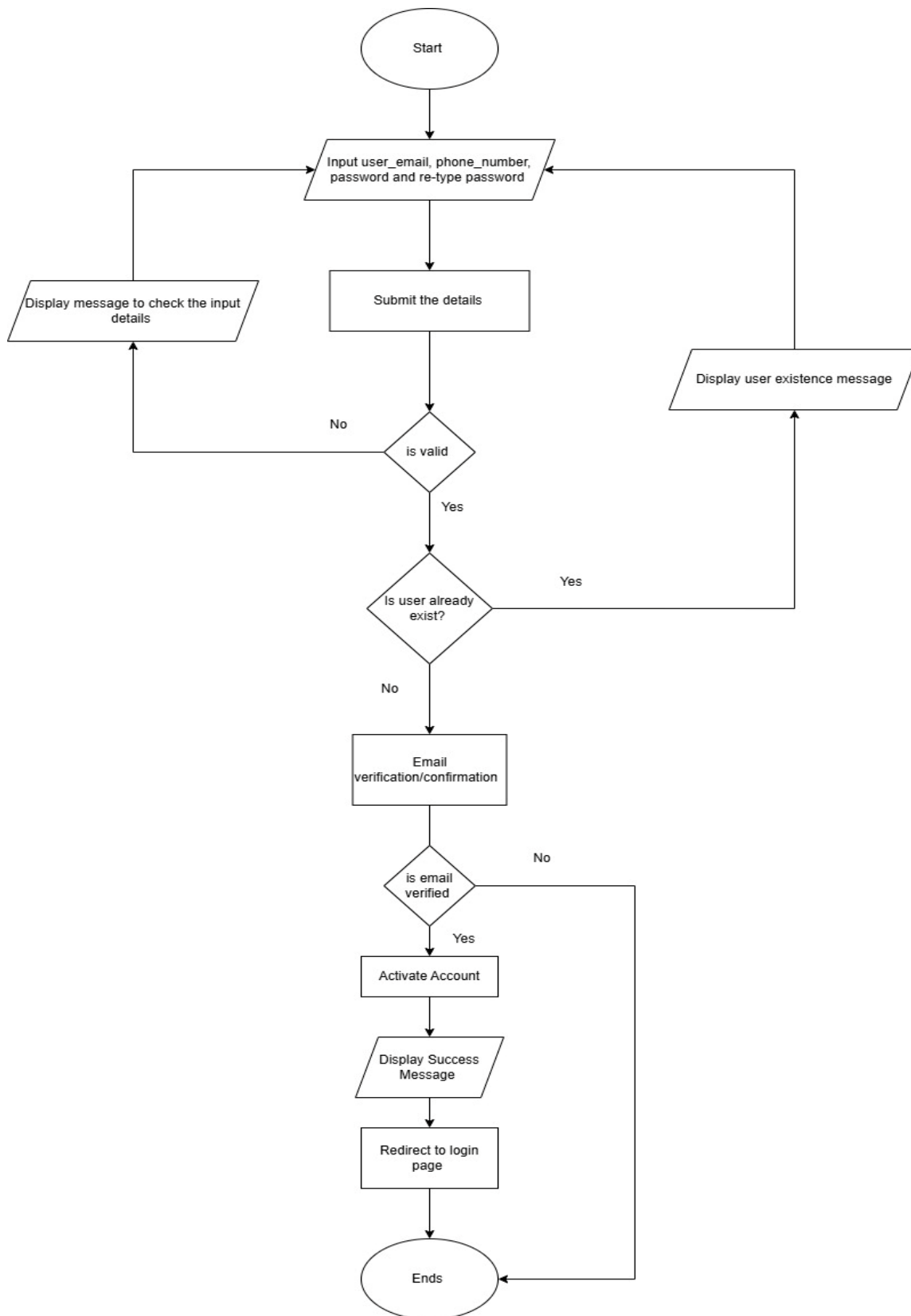


Figure 160 Flow Chart of user login

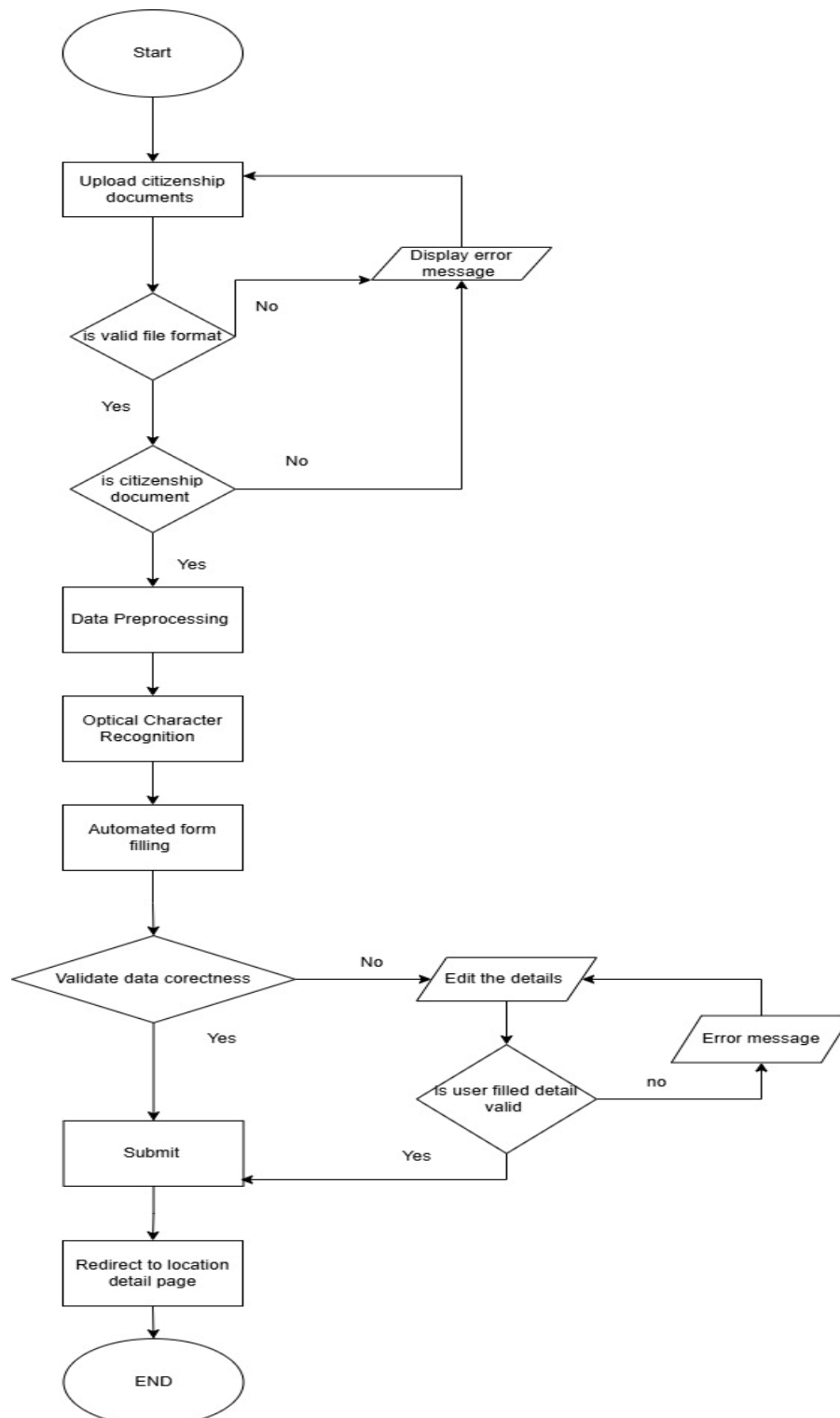
2) Flow chart of user registration



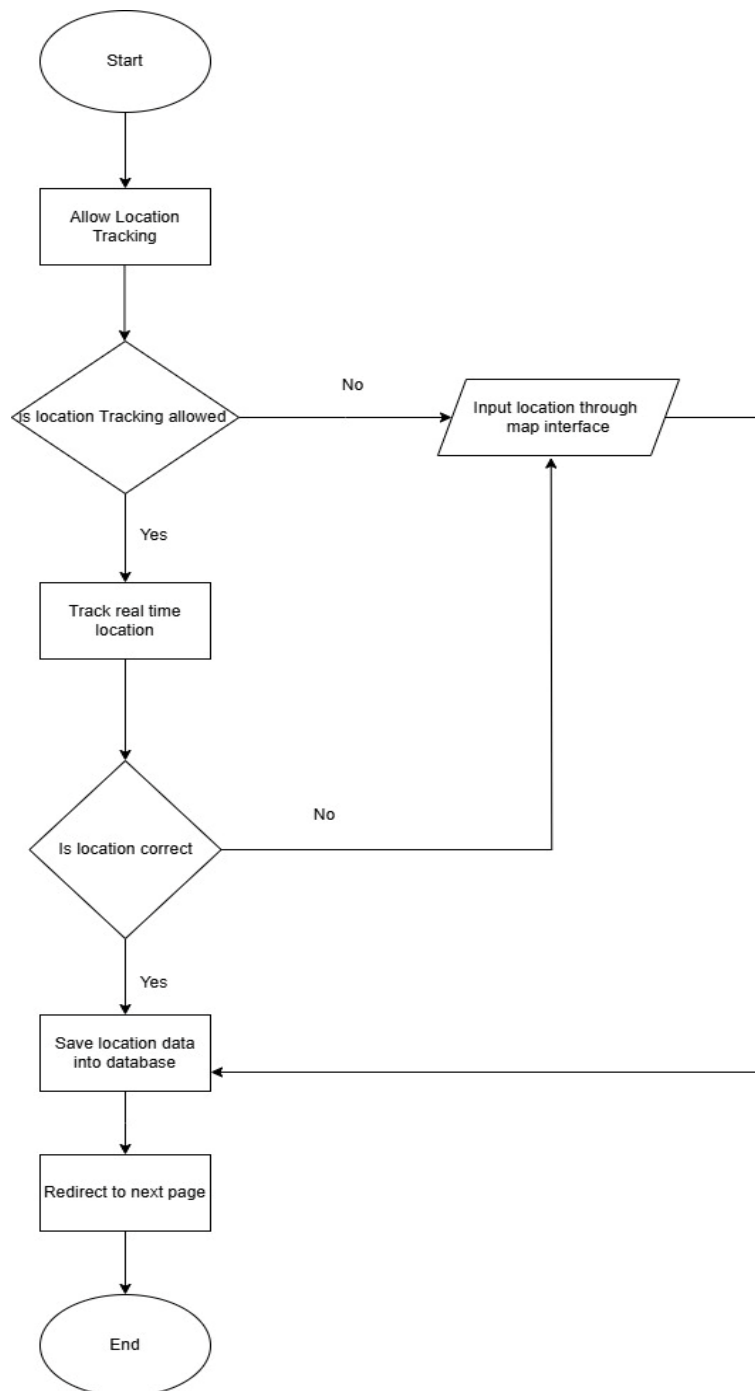
4.

Figure 161 Flow Chart of User Registration

3) Flow chart of form automation using OCR

*Figure 162 Flow Chart for Form Automation Using OCR*

4) Flow chart of User location tracking

*Figure 163 Flow Chart of User location Tracking*

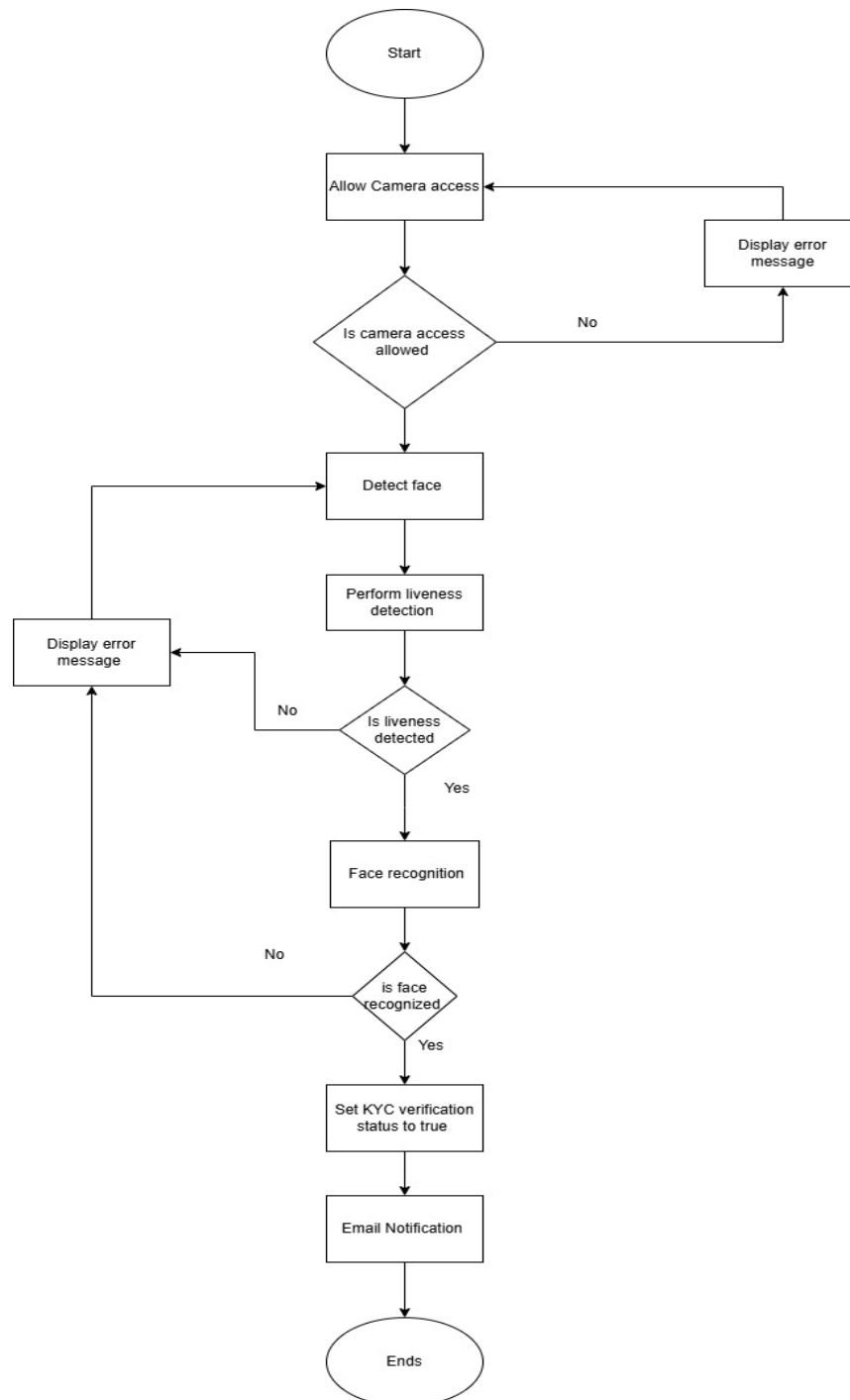
5) Flow chart of KYC Verification through facial recognition

Figure 164 Flow chart of KYC verification through face recognition

8.4.10 ACTIVITY DIAGRAM

1) Activity Diagram of user registration

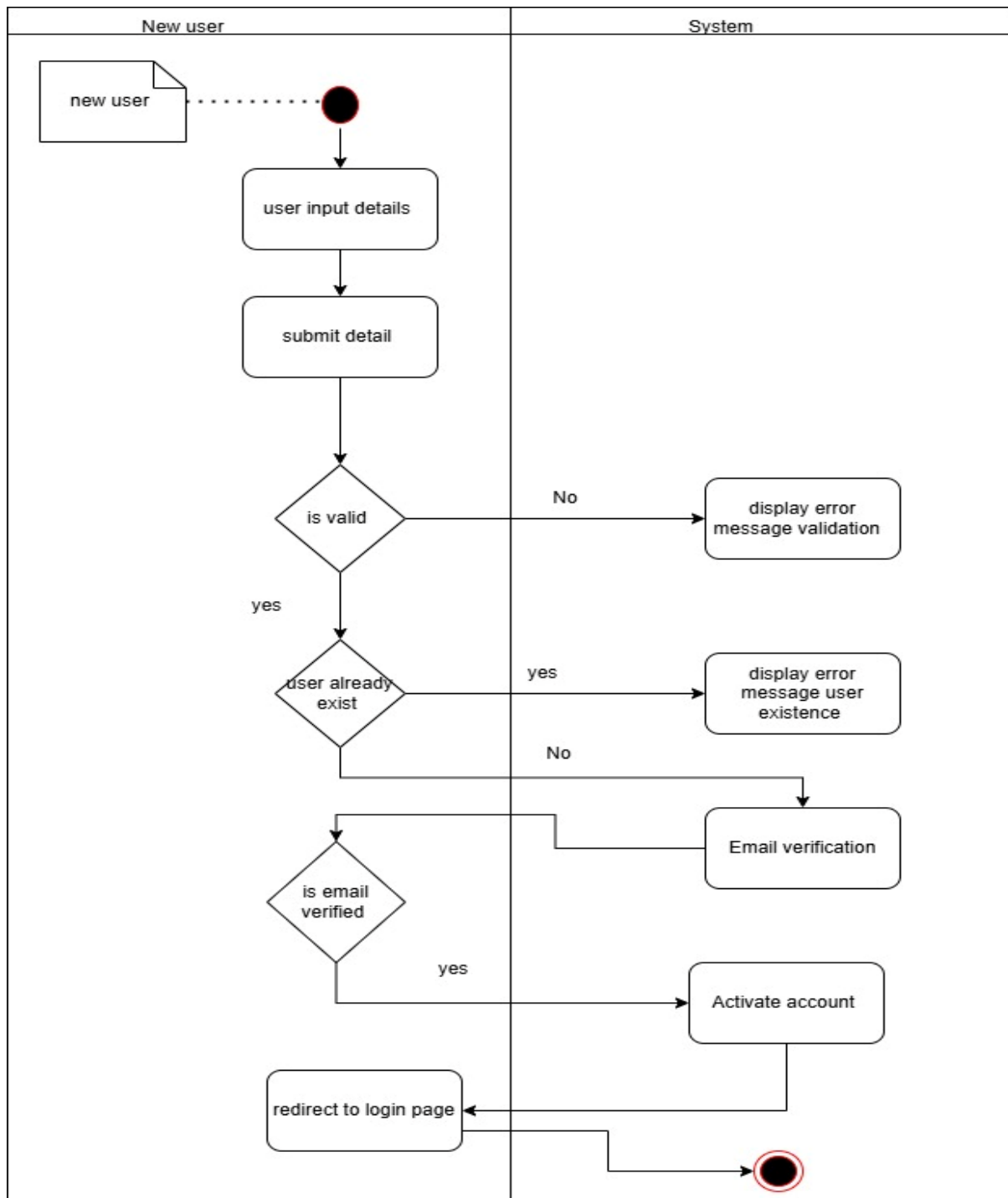


Figure 165 Activity Diagram of user registration

2) Activity Diagram of user login

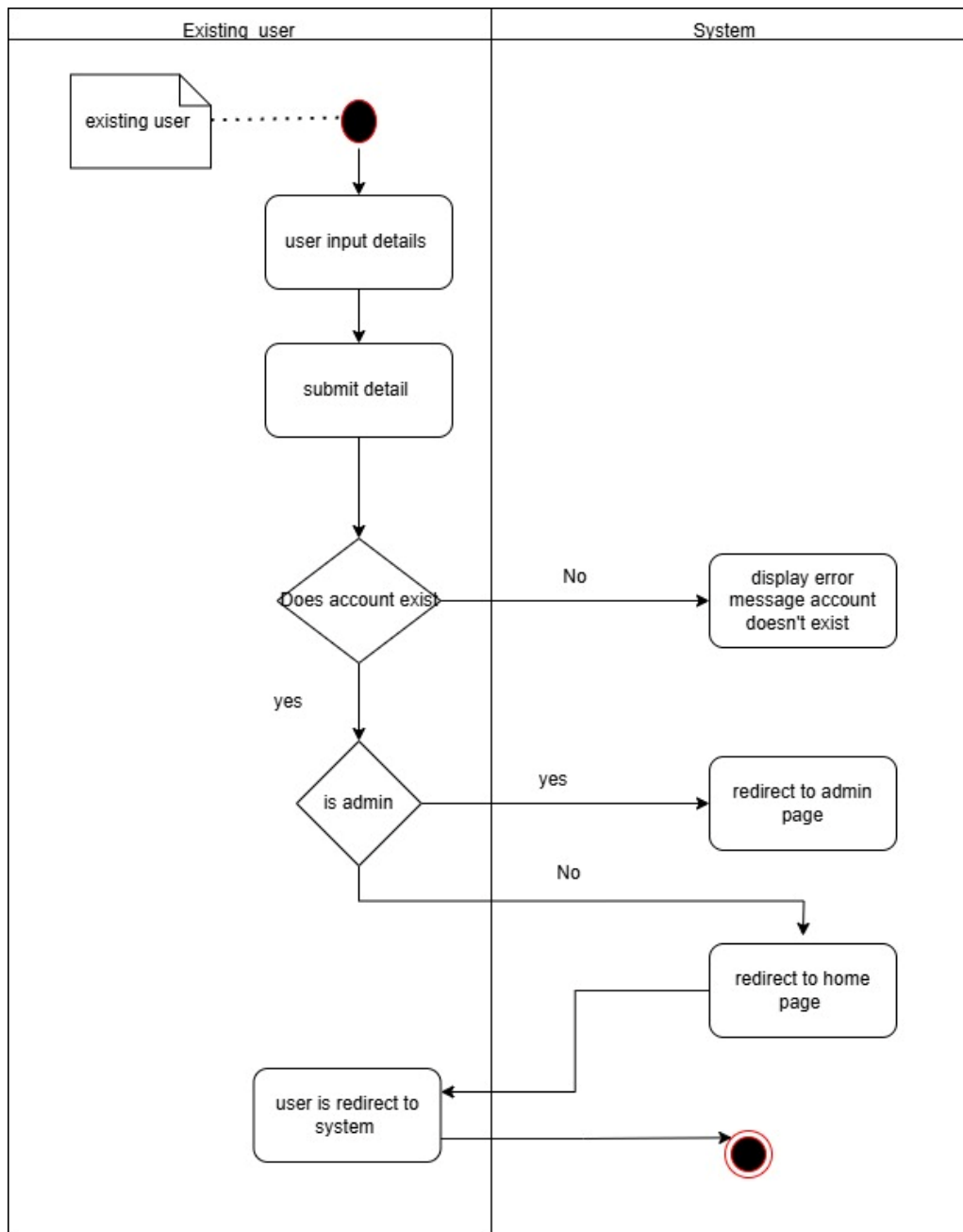


Figure 166 Activity Diagram of user login

3) Activity Diagram of user location tracking

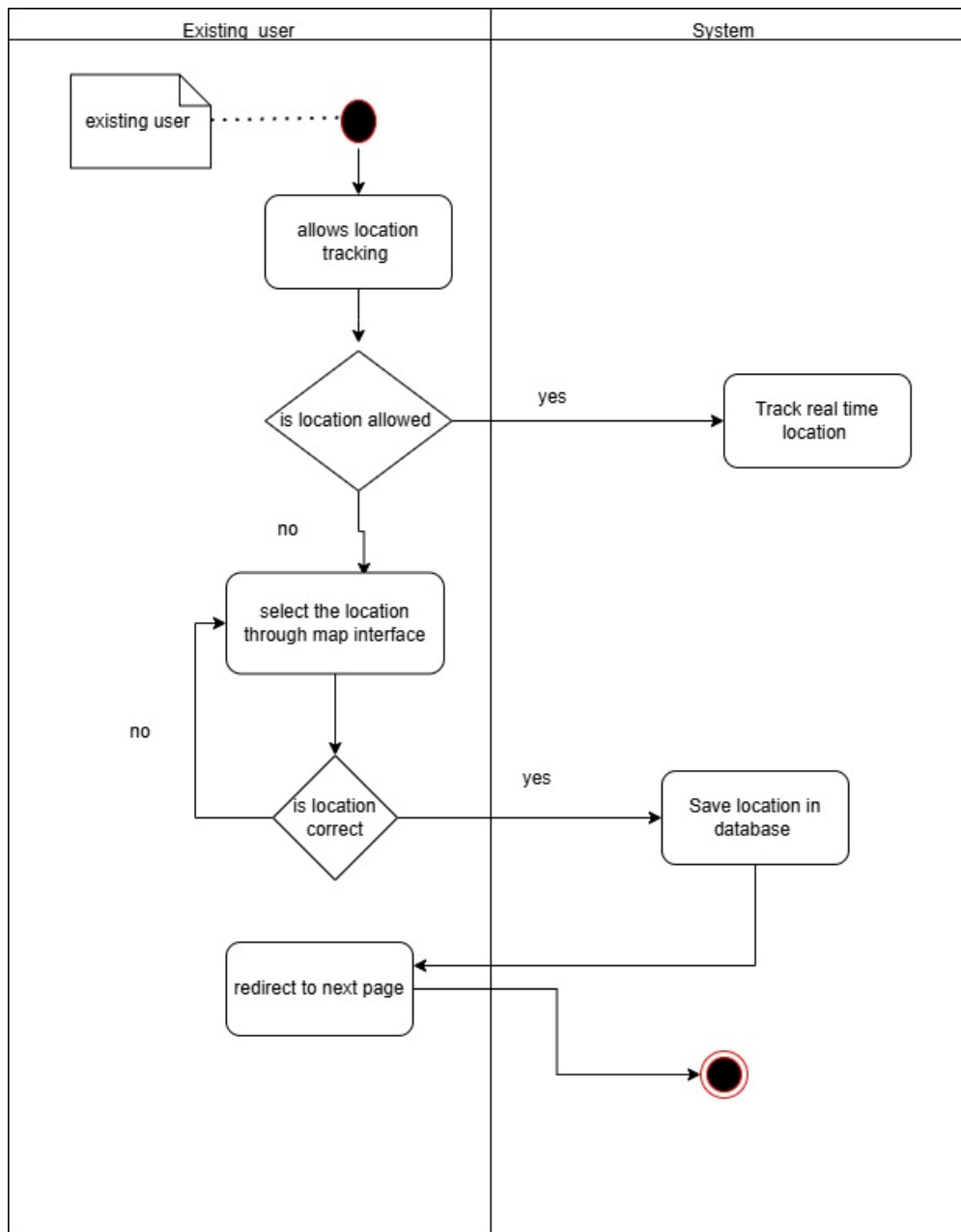


Figure 167 Activity Diagram of user location tracking

4) Activity Diagram of form automation through OCR

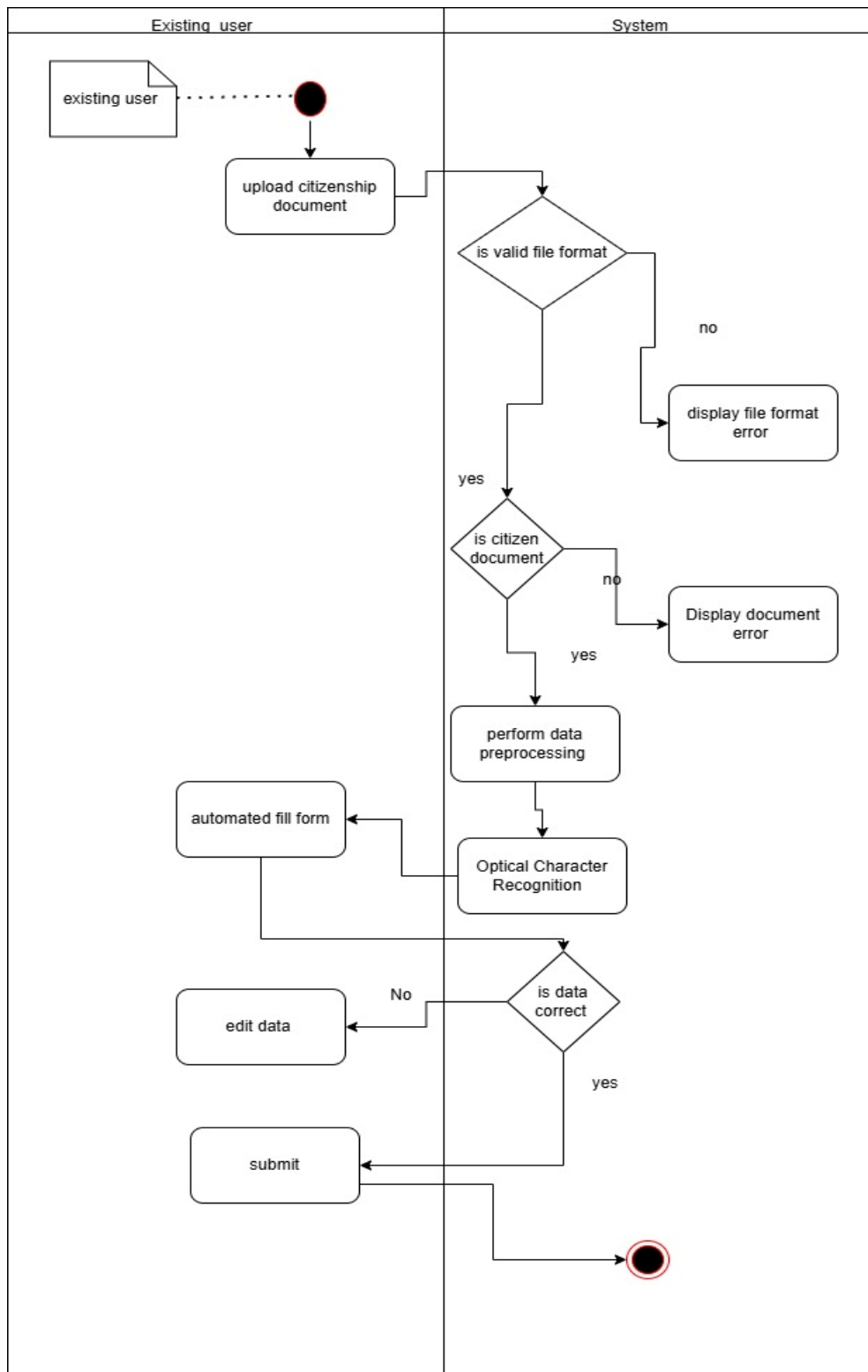


Figure 168 Activity Diagram of form automation through ocr

5) Activity Diagram of face recognition

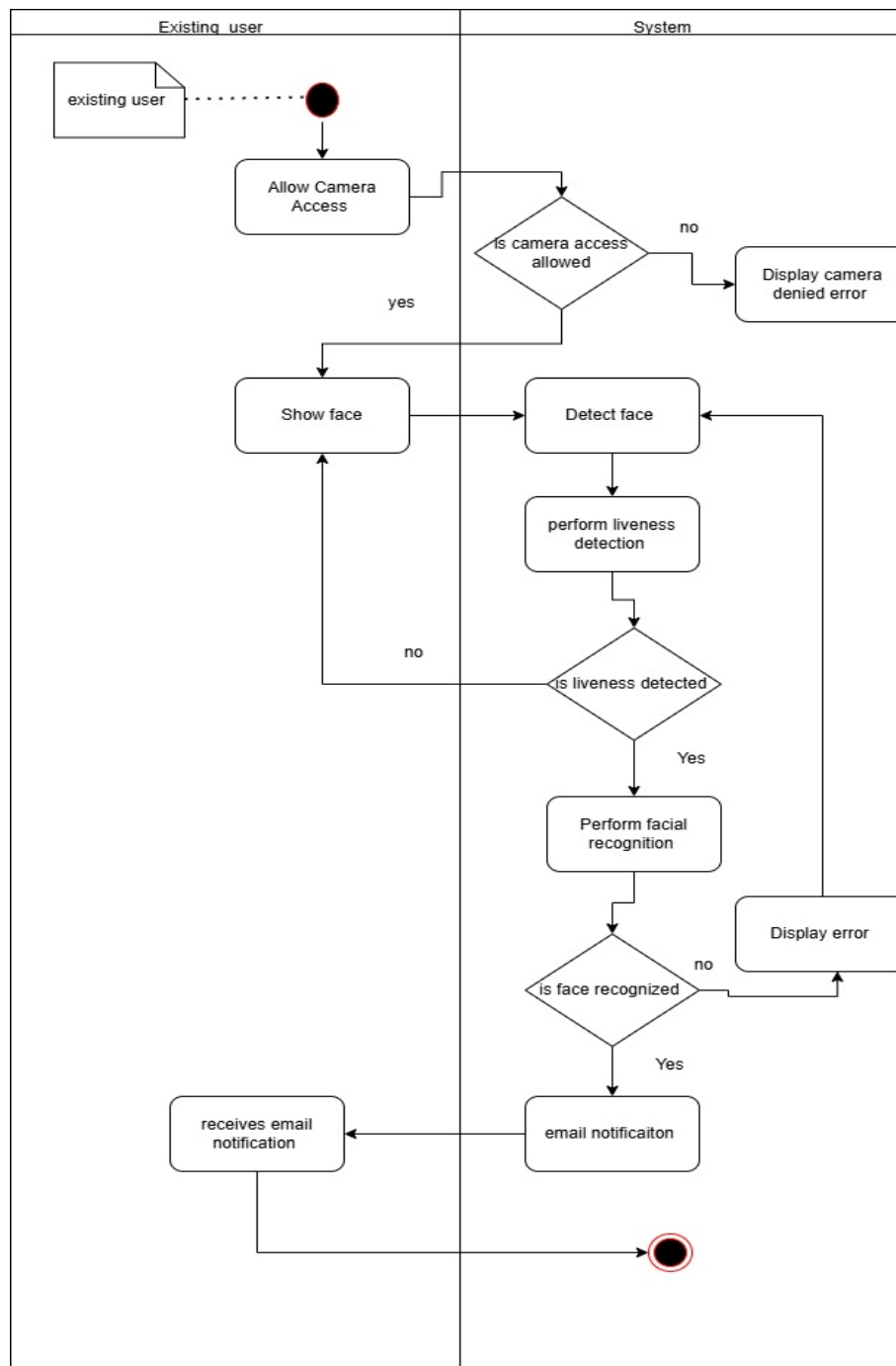
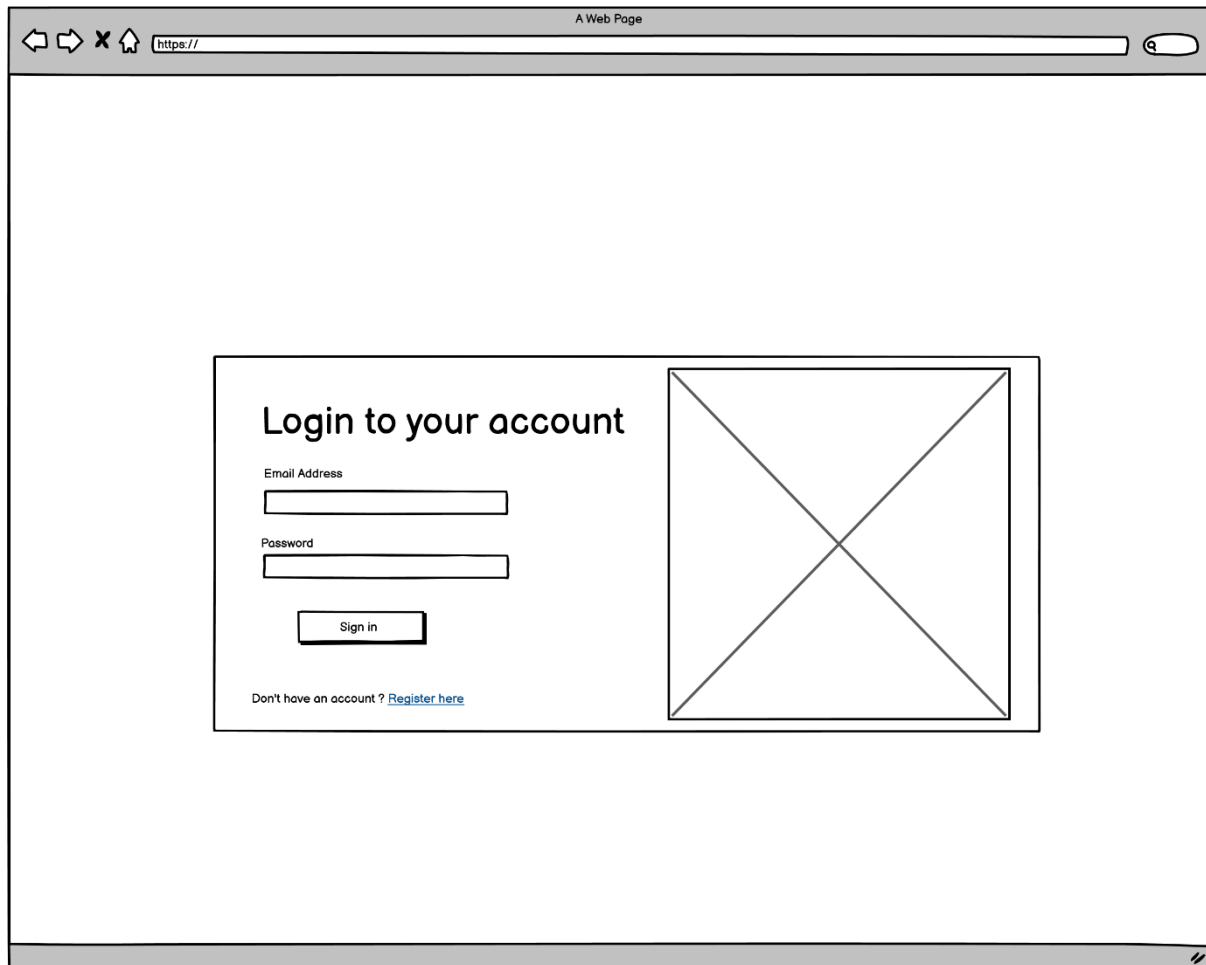


Figure 169 Activity Diagram of face recognition

8.4.11 WIREFRAME

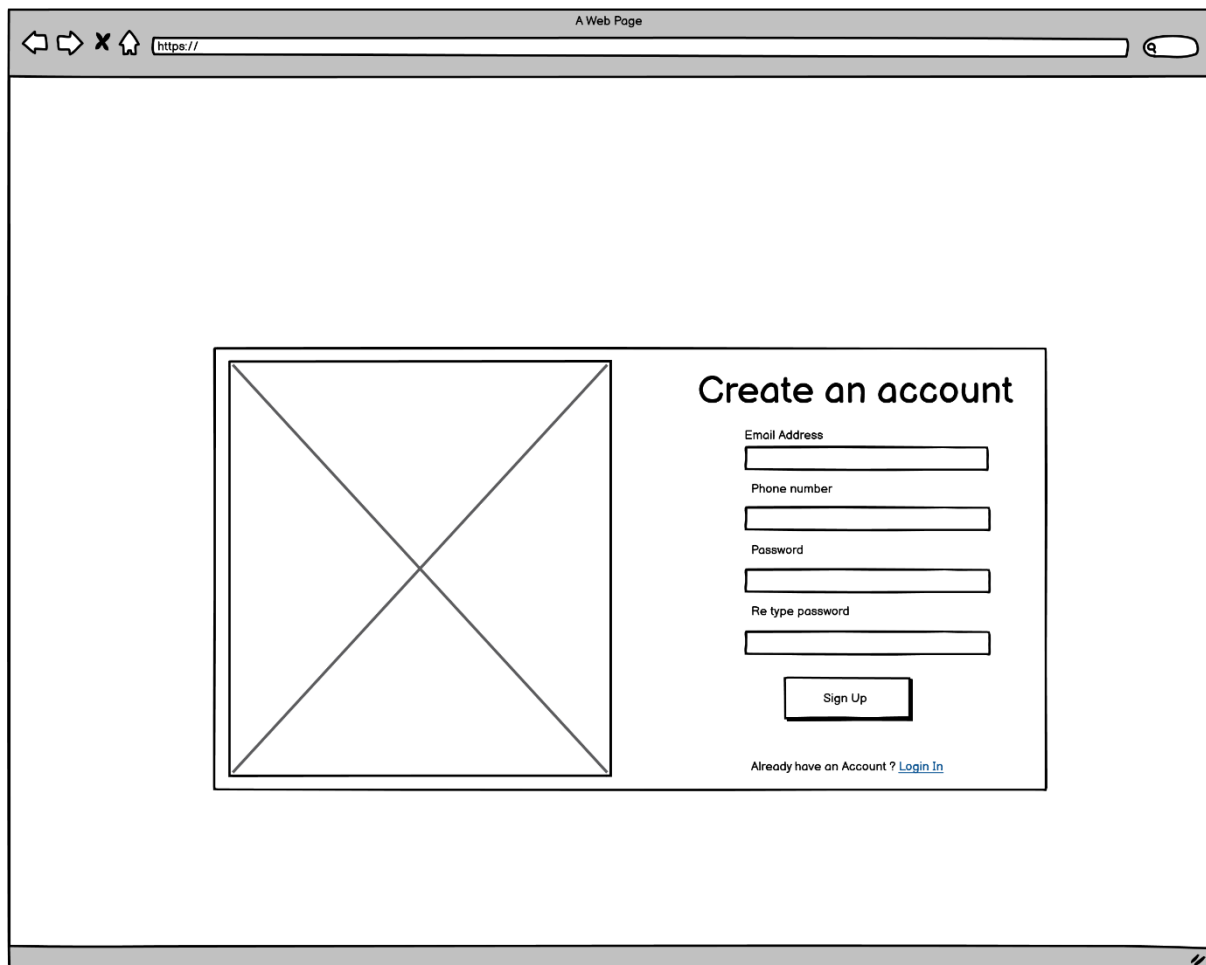
1) Login page



The wireframe shows a web browser window titled "A Web Page" with a URL bar containing "https://". The main content area features a login form titled "Login to your account". The form includes two input fields labeled "Email Address" and "Password", a "Sign in" button, and a link "Don't have an account ? [Register here](#)". To the right of the form is a large square placeholder with a diagonal cross, indicating a missing image or logo.

Figure 170 Wireframe of login page

2) Sign up page

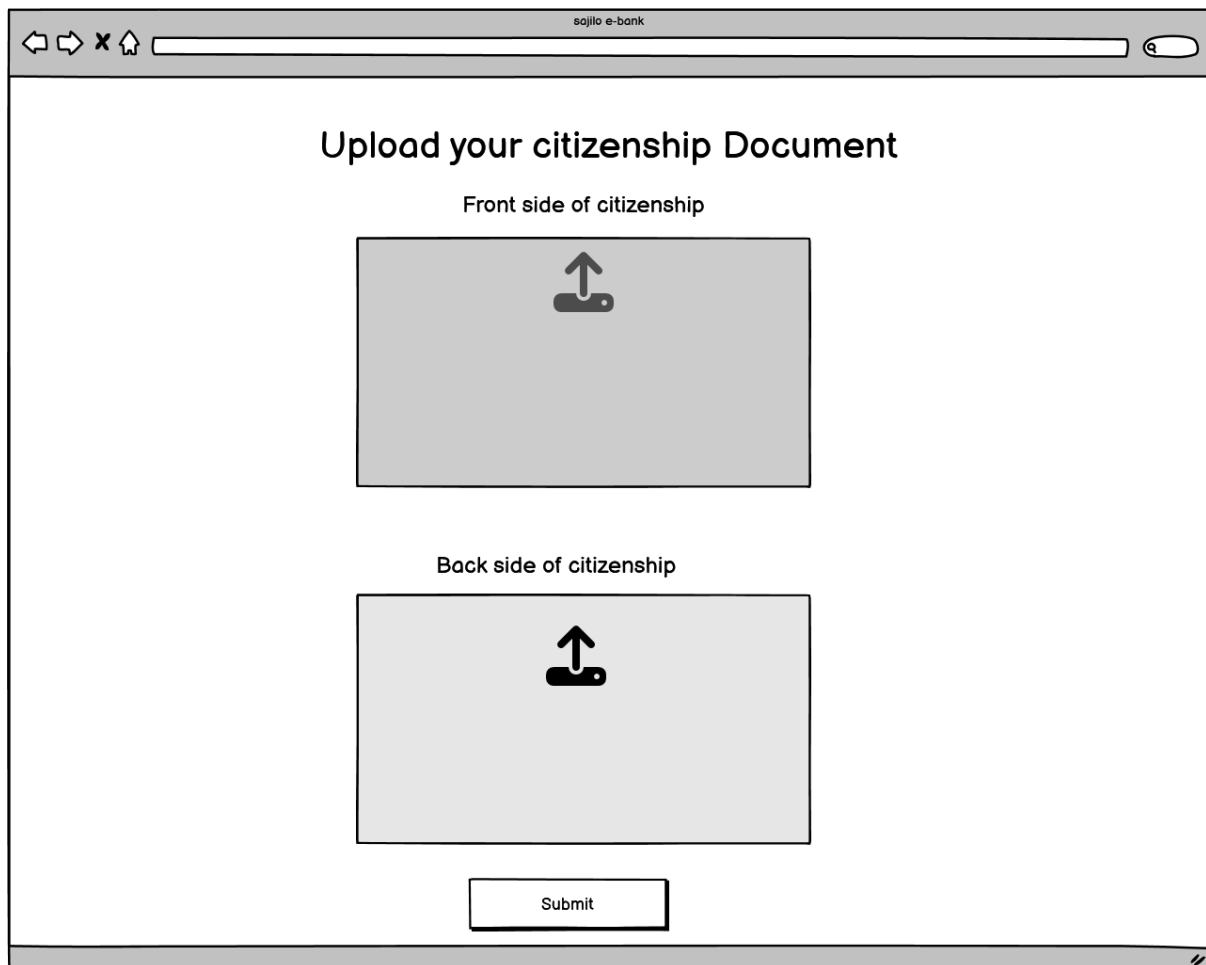


The wireframe shows a web browser window titled "A Web Page" with a URL bar containing "https://". The main content area features a sign-up form titled "Create an account". To the left of the form is a large square placeholder with a diagonal 'X'. The form includes the following elements:

- Create an account** (Section Header)
- Email Address** (Label) with a text input field.
- Phone number** (Label) with a text input field.
- Password** (Label) with a text input field.
- Re type password** (Label) with a text input field.
- Sign Up** (Button).
- Already have an Account ? [Login In](#)** (Text with a link).

Figure 171 Wireframe of signup page

3) Document Upload page



The wireframe shows a web browser window with the address bar displaying 'sajilo e-bank'. The main content area has a title 'Upload your citizenship Document'. Below the title, there are two sections: 'Front side of citizenship' and 'Back side of citizenship'. Each section contains a large rectangular box with a central upload icon (an upward arrow on a document). At the bottom of the page, there is a 'Submit' button.

Upload your citizenship Document

Front side of citizenship

Back side of citizenship

Submit

Figure 172 Wireframe of Document upload page

4) User location page

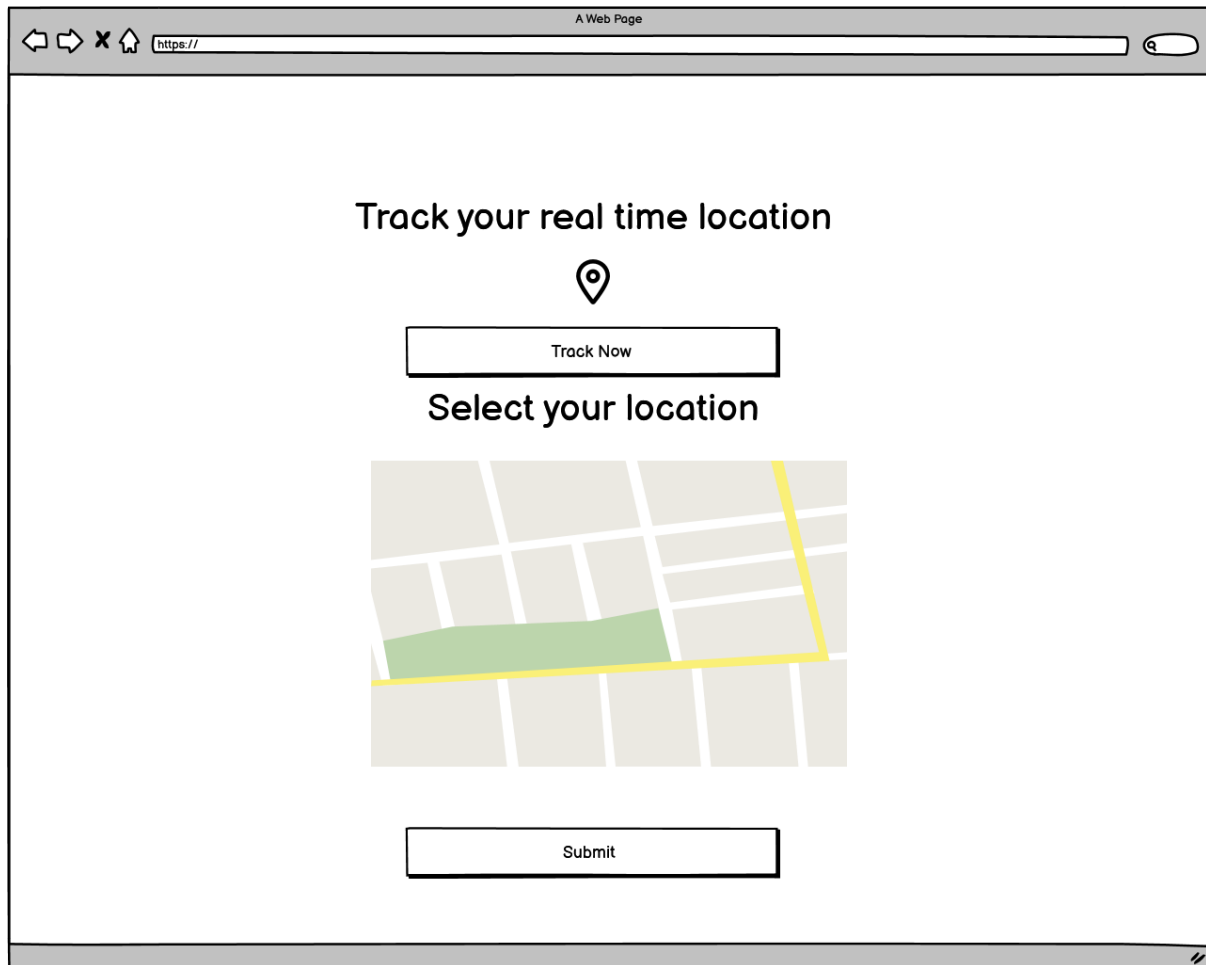


Figure 173 Wireframe of User Location Tracking page

5) Wireframe of personal detail

The wireframe shows a web browser window with the title 'sajilo e-bank'. The main content area is titled 'Personal Details'. It contains two columns of form fields. The left column has four fields: 'Full Name', 'Citizenship Number', 'Birth Place', and 'Mother Name'. The right column has three fields: 'Date of birth' (with a calendar icon), 'Gender', 'Permanent Address', and 'Father Name'. A 'Submit' button is located at the bottom right of the form area.

Personal Details	
Full Name 	Date of birth / /
Citizenship Number 	Gender
Birth Place 	Permanent Address
Mother Name 	Father Name
Submit	

Figure 174 Wireframe of personal detail form

6) Wireframe of face recognition for kyc verification

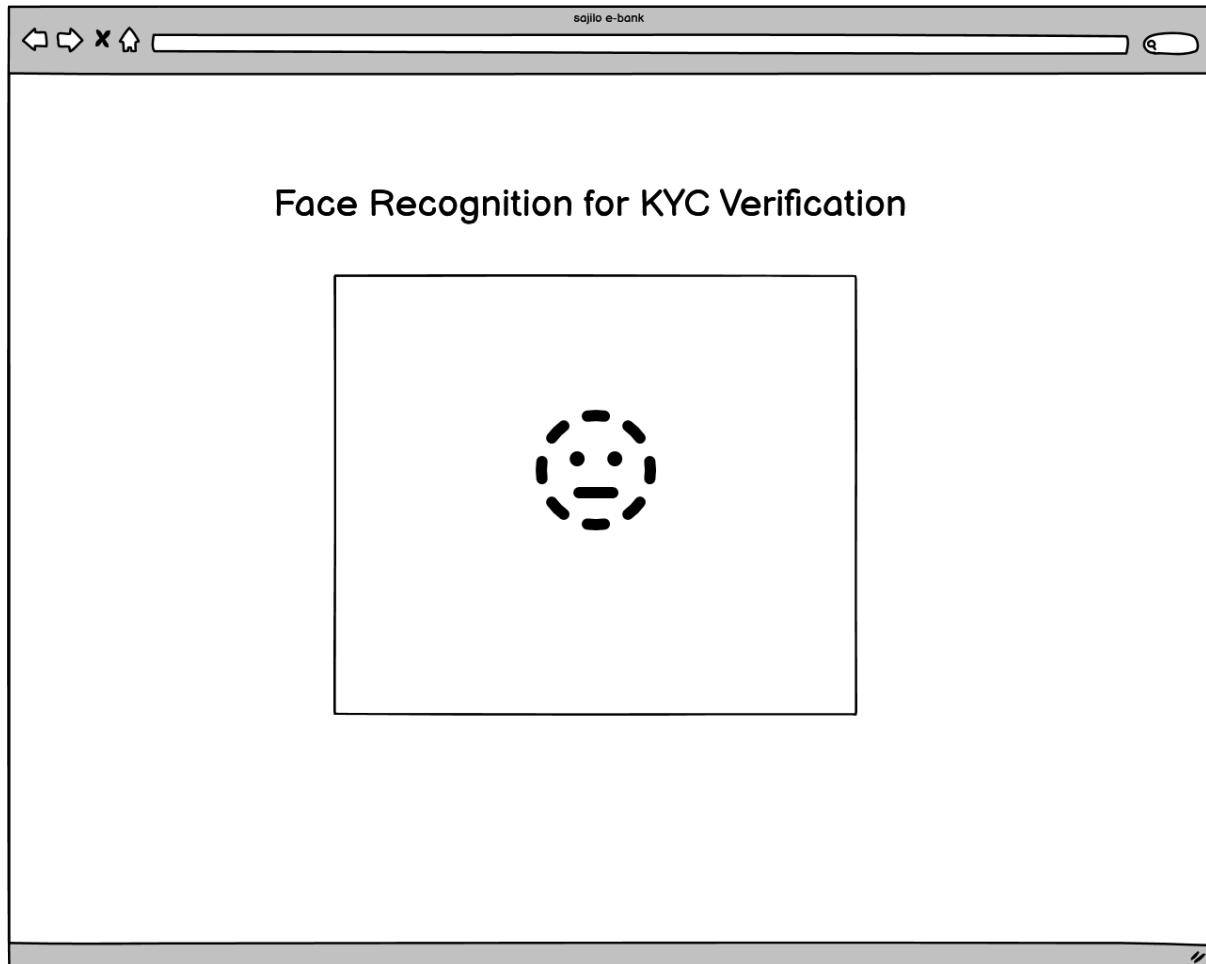


Figure 175 Wireframe of face recognition for kyc verification

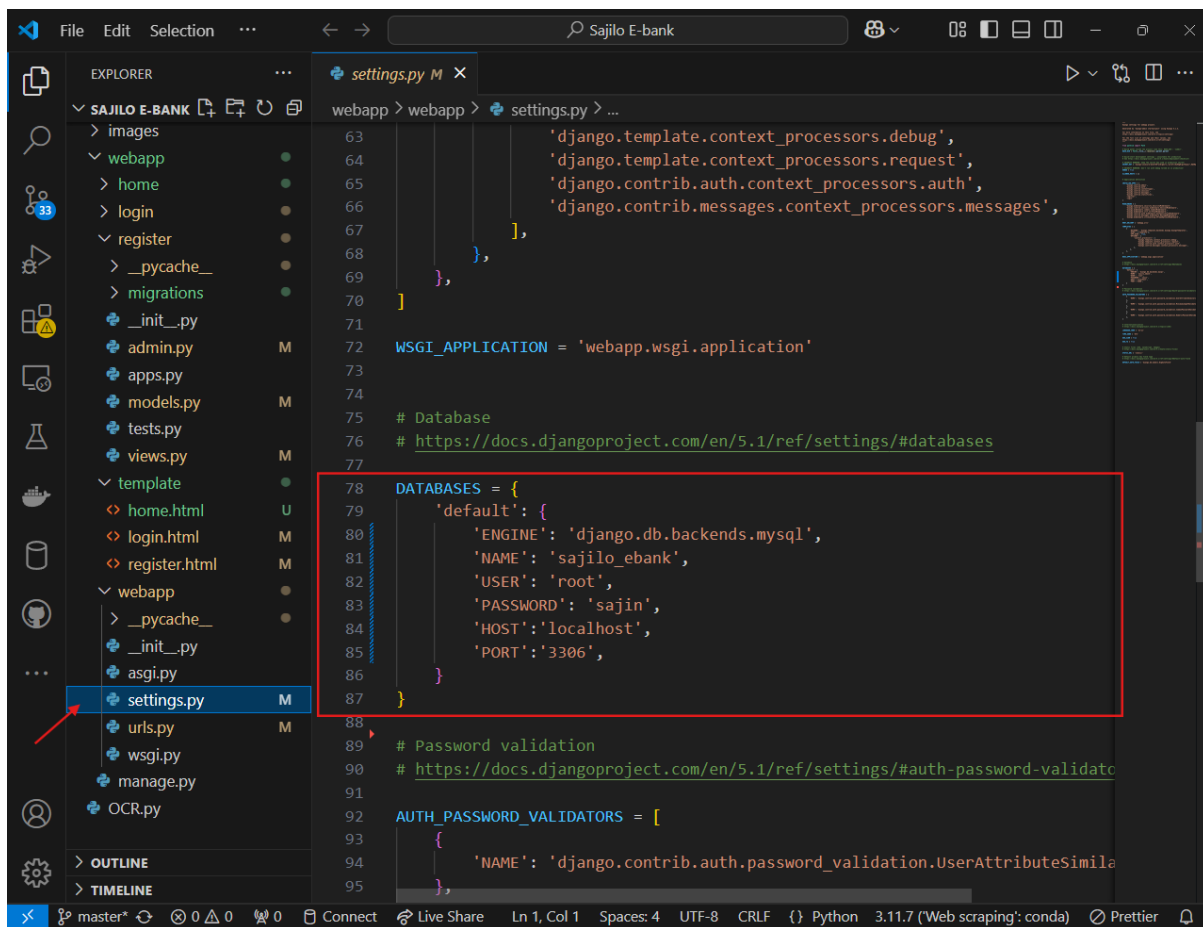


Figure 176 Django connected with MySQL databas

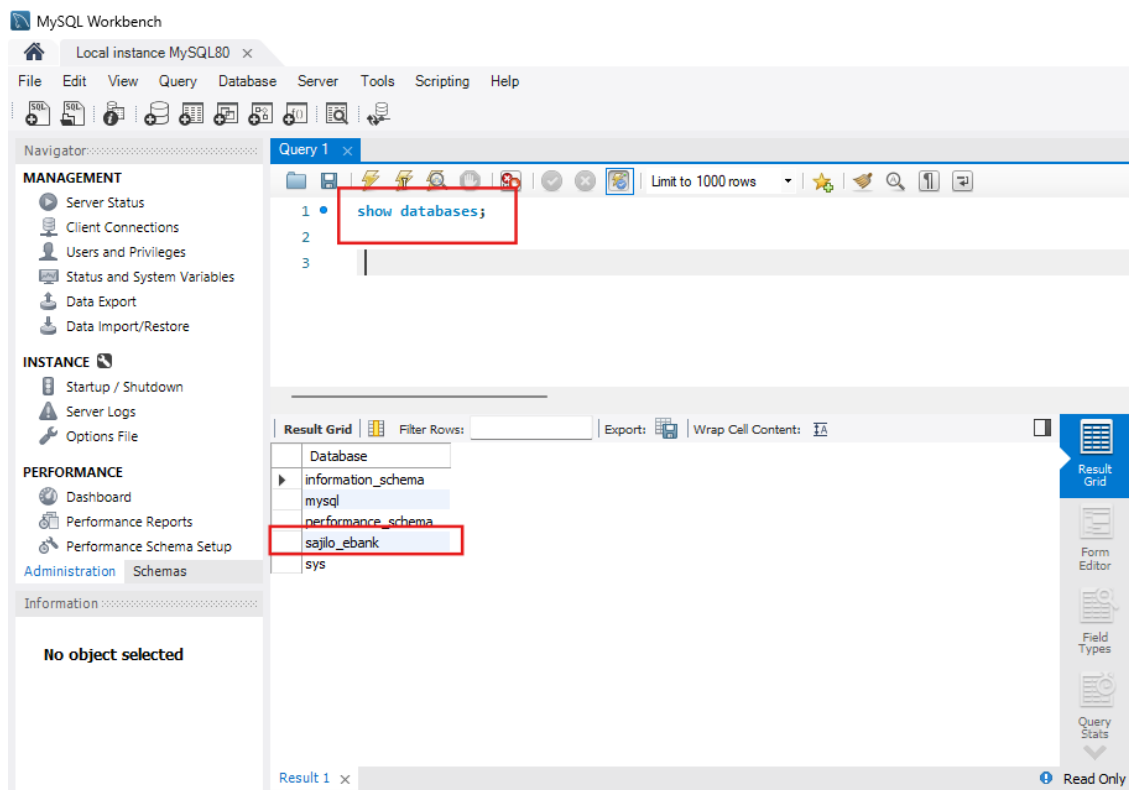


Figure 177 Database of the project

8.5 APPENDIX E: SCREENSHOT OF THE SYSTEM

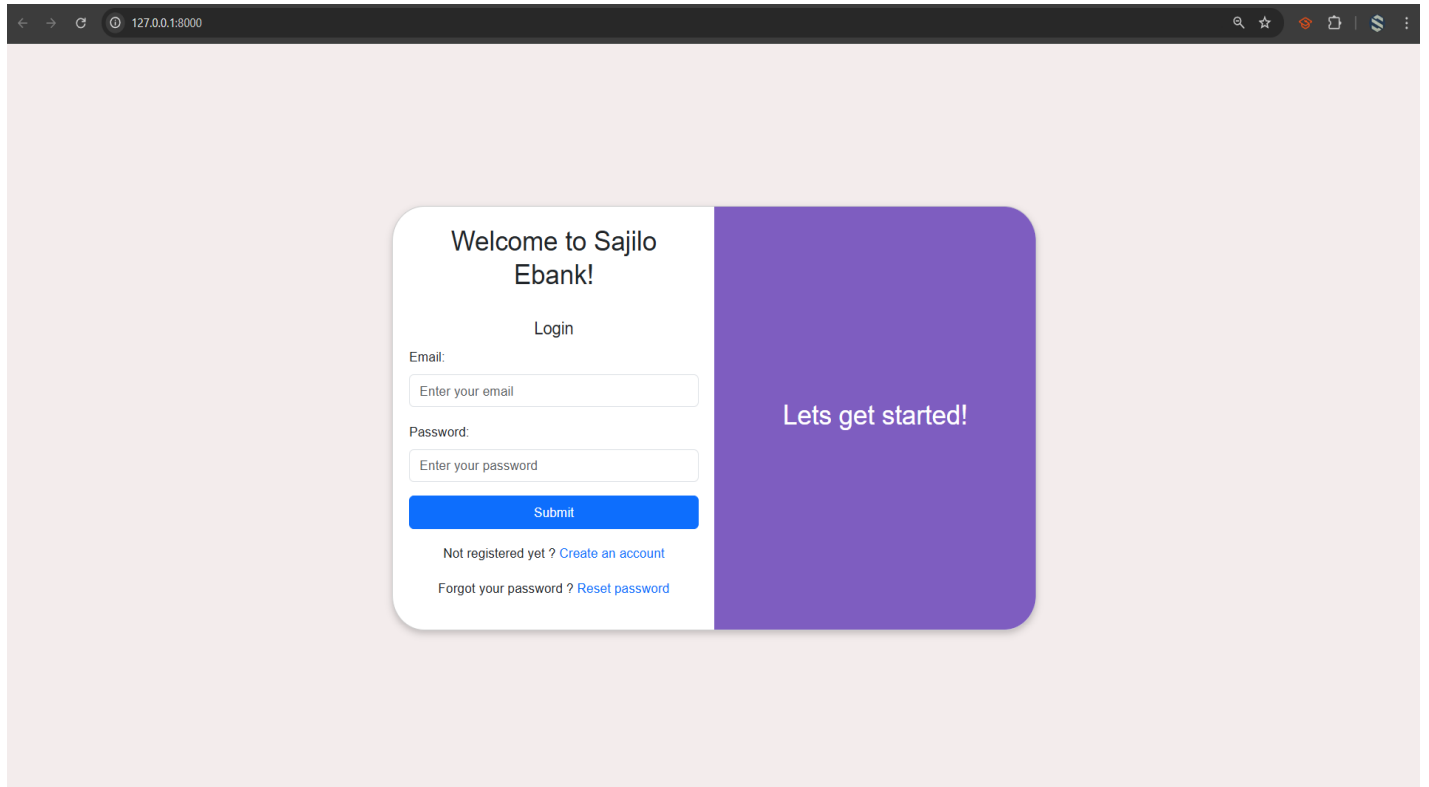
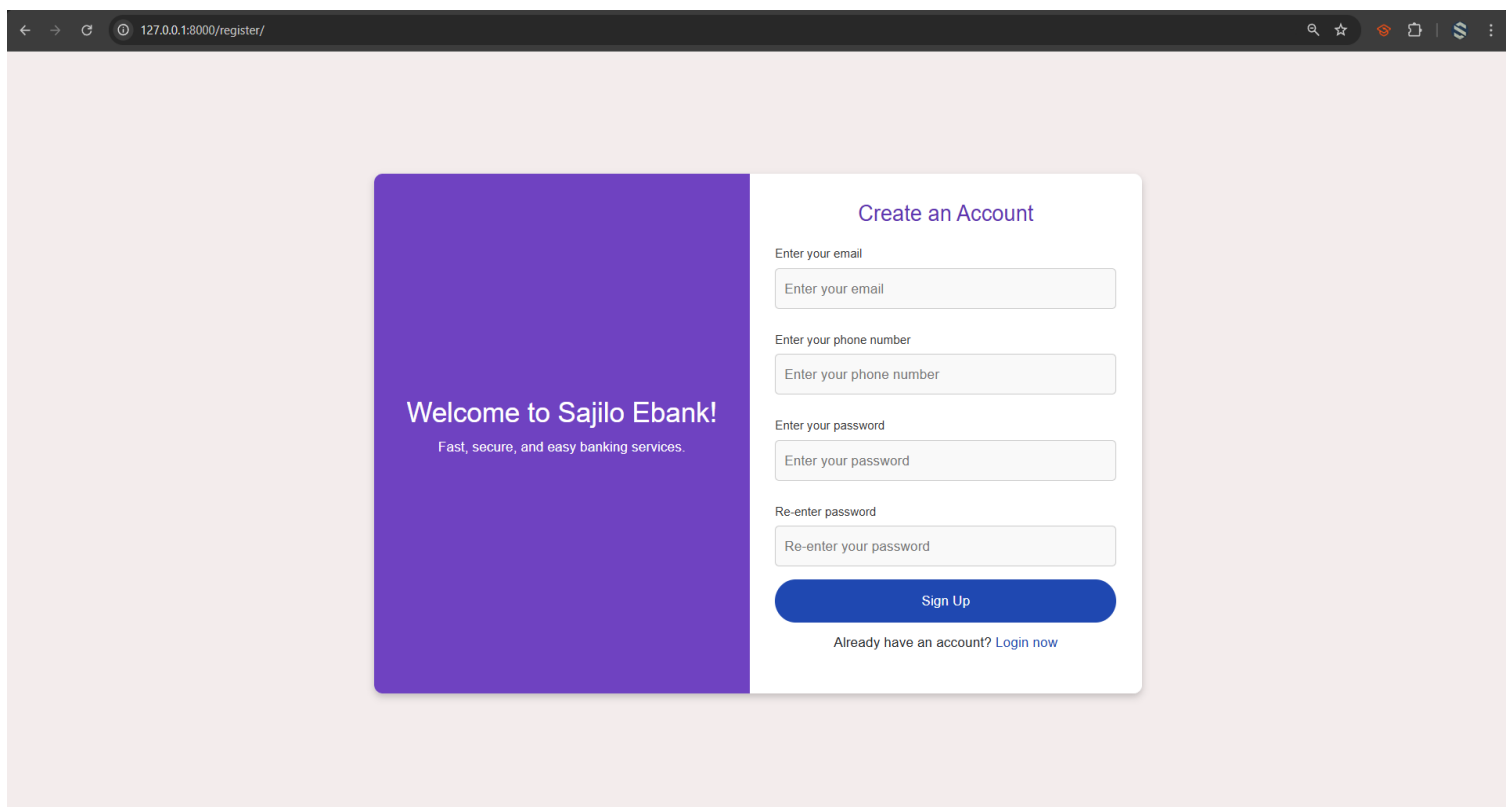


Figure 178 Login page



127.0.0.1:8000/register/

Welcome to Sajilo Ebank!
Fast, secure, and easy banking services.

Create an Account

Enter your email

Enter your phone number

Enter your password

Re-enter password

[Sign Up](#)

Already have an account? [Login now](#)

Figure 179 Signup page

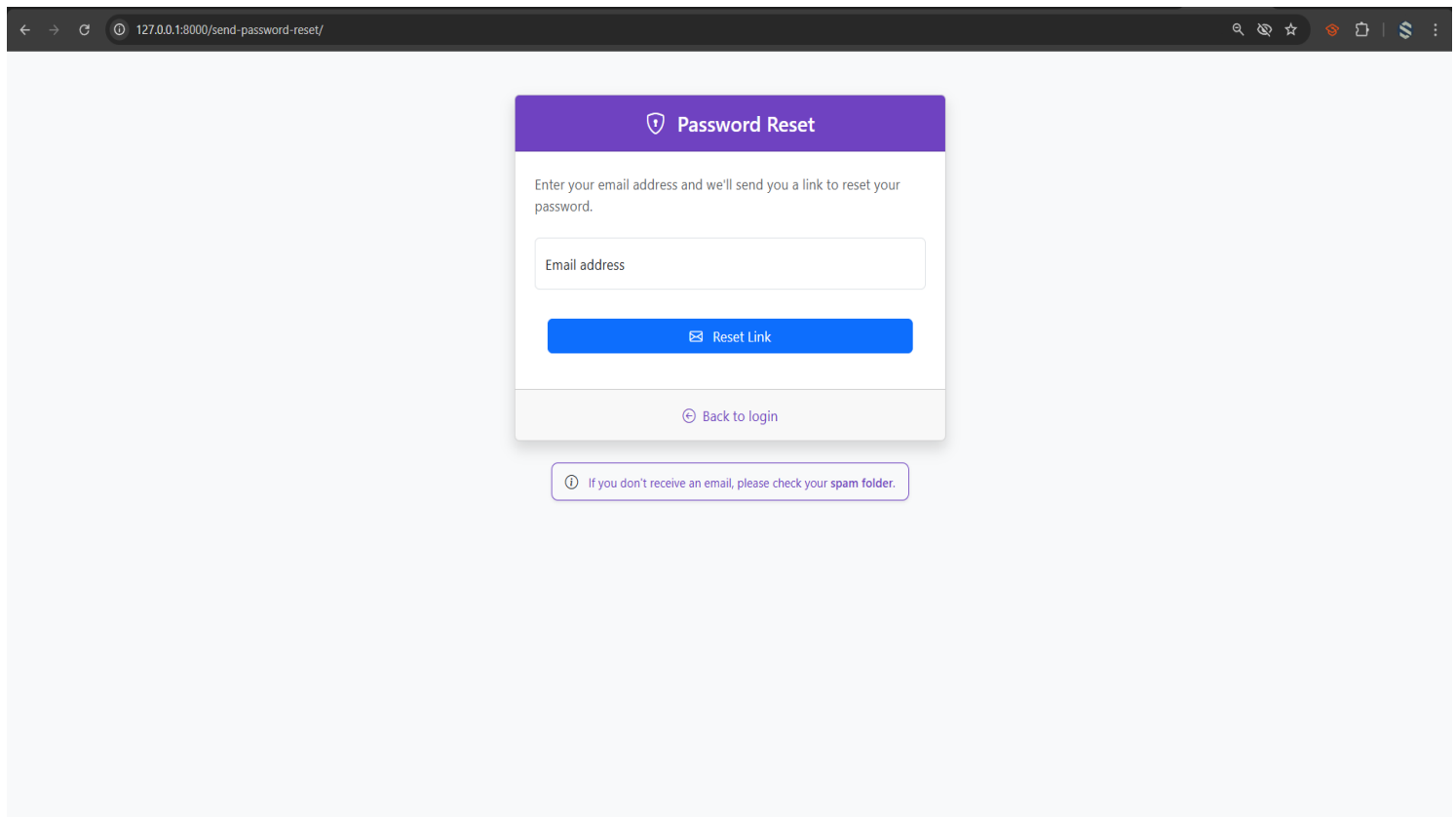


Figure 180 Reset password email sent page

Sajilo E-Bank

HomeLocationDocument UploadLogout

Upload Your Documents

Front Side

Back Side

Upload

Figure 181 Document Upload

Sajilo E-Bank

HomeLocationDocument UploadLogout

Track My Location

Search for a location

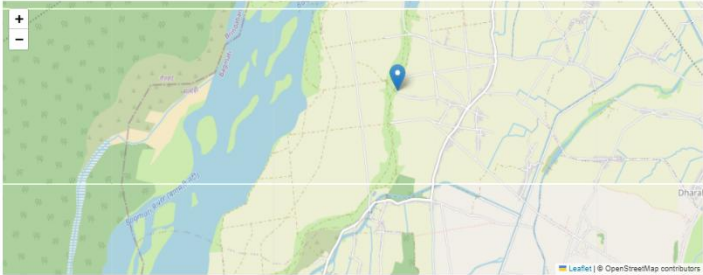
Search

Note: The "Track My Location" feature might not be accurate. If needed, please select your location manually on the map.

Track My Location

+

-



Latitude:

27.0889759

Longitude:

85.4684971

Submit

Figure 182 Location Tracking

22067177 Sajin Raj Amatya

148

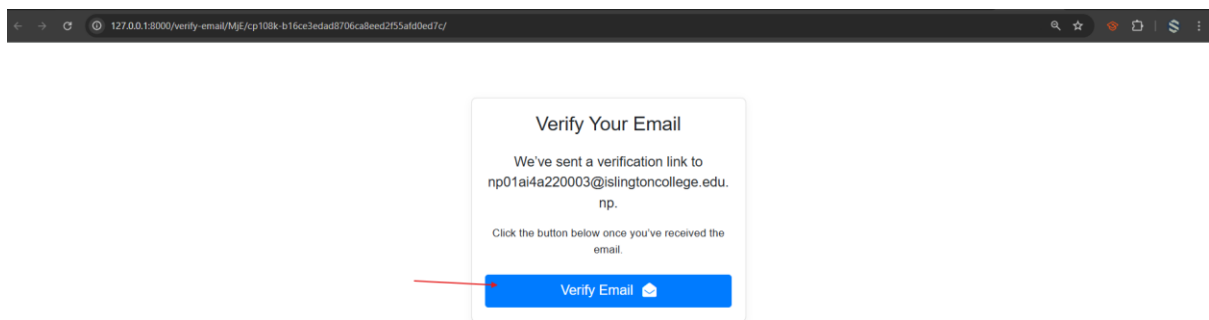


Figure 183 Email verification

127.0.0.1:8000/reset-password/21:1u9hKX:qtYOiEz2Pr4qiwLPryON1PIbfc8UzeqdkTDSdBhRa3g/

Reset Your Password

Choose a strong, unique password.

New Password

Confirm Password

Reset Password

Figure 184 Reset password form

Sajilo E-Bank

HomeLocationDocument UploadLogout

Citizenship Information Form

Citizenship Number

Enter your citizenship number

Full Name

Enter your full name

Gender

Select your gender

Date of Birth

mm/dd/yyyy

Place of Birth

Enter place of birth

Permanent Address

Enter your permanent address

Reset

Submit

Figure 185 Citizenship information form

Dashboard

Total Number of Users

3

Total Login Count

177

Total Verified Emails

2

User Details

No.	User Id	Email	Phone number	Account creation date	Email Verified Status	Document Verified Status	Last Login	Login Count	Update	Delete
1	11	sajinamatya88@gmail.com	11	Feb. 19, 2025, 4:42 p.m.	True	True	Feb. 19, 2025, 4:42 p.m.	174		
2	12	ss	ss	Feb. 22, 2025, 5:09 a.m.	False	False	Feb. 22, 2025, 5:09 a.m.	0		
3	21	np01ai4a220003@islingtoncollege.edu.np	9823788703	April 29, 2025, 9:12 a.m.	True	True	April 29, 2025, 9:12 a.m.	3		

Figure 186 User Dashboard

8.6 APPENDIX F : LOG SHEET

8.6.1 log sheet 1

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 1	Date: 8 September 2024
Start Time: 10:00 AM	End Time: 12: 00 AM
Items Discussed: a) Problem-based Project idea for the Final Year Project. b) Tools and techniques to be used for Project. c) Importance of Final Year Project for completion of bachelor's degree. d) Mistakes made by previous students in their final year projects.	
Achievements: a) Identified a real-world problem for FYP.	
Problems: Choosing a specific project idea from various options.	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Explore different projects and finalize the suitable one.	



 Student Sign



 Internal Supervisor Sign



 External Supervisor Sign

Figure 187 log sheet 1

8.6.2 logsheet 2

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 2	Date: 15 September 2024
Start Time: 11:00 AM	End Time: 12: 00 AM
Items Discussed: a) Discussed the project Idea b) Validated whether the project idea is suitable or not. c) Explored and discussed the features of the project. d) Technology required to build the feature of the project	
Achievements: a) Finalized the project for FYP. b) Explored additional the features for the project.	
Problems: Confused regarding the tech stack for the project.	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Prepare a draft proposal and submit it before the deadline.	



Student Sign



Internal Supervisor Sign



External Supervisor Sign

Figure 188 log sheet 2

8.6.3 log sheet 3

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 3	Date: 22 September 2024
Start Time: 1:00 PM	End Time: 3: 00 PM
Items Discussed: a) Presented the Draft proposal topic to the internal supervisor b) Discussed the suitable SDLC model for the project c) Discussed the tools and technologies that will be used for the project	
Achievements: a) Completion of Draft proposal. b) Finalized the features of the project.	
Problems: a) Selection of right SDLC model for the project. b) Selection of tools and technologies.	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)	



 Student Sign



 Internal Supervisor Sign



 External Supervisor Sign

Figure 189 log sheet 3

8.6.4 logs sheet 4

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 4	Date: October 2, 2024
Start Time: 10 :00 AM	End Time: 11: 00 AM
Items Discussed: a) Presented the project idea to external Supervisor b) Discussed about the real-world problem solution of the project	
Achievements: a) Idea about the similar system related to my project b) Idea about OCR based python module (tesseract)	
Problems: a) Identification of a Unique feature for the project which is different from the similar system	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor)	


Student Sign


Internal Supervisor Sign


External Supervisor Sign

Figure 190 Log sheet 4

8.6.5 log sheet 5

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 5	Date: November 10, 2024
Start Time: 1 :00 PM	End Time: 3 : 00 PM
Items Discussed: a) Presented the project to features to internal and external Supervisor. b) Discussed the additional unique features that can be added to the project	
Achievements: a) Set up the Django project with login and register features. b) Demo of OCR with pytesseract c) Got an idea of about one-shot learning method for facial recognition.	
Problems: a) Can't figure out the unique features for the project	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Explore additional feature to add into the project which is unique from other similar system.	



Student Sign



Internal Supervisor Sign



External Supervisor Sign

Figure 191 log sheet 5

8.6.6 log sheet 6

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 6	Date: November 17, 2024
Start Time: 1 :00 PM	End Time: 3: 00 PM
Items Discussed: a) Overall features of the project b) Discuss the feasibility of the voice recognition feature in our system Achievements: a) Removed unnecessary features voice recognition from the project Problems: a) Can't figure out the unique features of the project Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Complete the proposal from chapter 1 to chapter 4 and explore additional features.	



 Student Sign



 Internal Supervisor Sign



 External Supervisor Sign

Figure 192 log sheet 6

8.6.7 log sheet 7

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 8	Date: 1 st December 2024
Start Time: 1 :00 PM	End Time: 3: 00 PM
Items Discussed: a) Work breaks down structure and Gantt chart for incremental model b) Format for Milestone chart Achievements: a) Completion of Work break down structure and Gantt chart of proposal Problems: a) Didn't understood the format of milestone chart Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Complete and submit the project proposal	



Student Sign



Internal Supervisor Sign



External Supervisor Sign

Figure 193 log sheet 7

8.6.8 log sheet 8

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 9	Date: December 8, 2024
Start Time: 1 :00 PM	End Time: 3: 00 PM
Items Discussed: a) Overall Structure and content of the interim report. b) Referencing and formatting techniques for the interim report. c) Use of cloud computing service for the project. Achievements: a) Completion of Project Proposal. Problems: Couldn't figure out the content for the appendix section of the interim report along with content to include in it . Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Complete the introduction and background portion of the interim report.	



Student Sign


Internal Supervisor Sign


External Supervisor Sign

Figure 194 log sheet 8

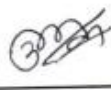
8.6.9 log sheet 9

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 11	Date: December 22, 2024
Start Time: 1:00 PM	End Time: 3: 00 PM
Items Discussed: a) Reviewed the Use case diagram and SRS Document. b) Necessary diagram of system analysis to include in the interim report.	
Achievements: a) Completion of Use case diagram. b) Completion of SRS Documentation of the project.	
Problems: Unable to understand the format of SRS Document and activity diagram.	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Complete the development portion along with system analysis design of interim report.	



Student Sign



Internal Supervisor Sign



External Supervisor Sign

Figure 195 log sheet 9

8.6.10 log sheet 10

FYP Logbook Entry Sheet	
Meeting No: 14	Date: January 12, 2025
Start Time: 1:00 PM	End Time: 3: 00 PM
Items Discussed: a) Proper referencing techniques for document citation. b) Microsoft word turn it in working mechanism. c) File naming conventions for the coursework submission.	
Achievements: a) Completion and submission of interim report.	
Problems: Couldn't figure out the way to exclude the bibliography, cited text from the turn it in report.	
Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Make a checklist of the task that will be completed within the semester break.	



Student Sign



Internal Supervisor Sign

External Supervisor Sign

Figure 196 log sheet 10

8.6.11 log sheet 11

FYP Logbook Entry Sheet

FYP Logbook Entry Sheet	
Meeting No: 15	Date: January 19, 2025
Start Time: 1:00 PM	End Time: 3: 00 PM
Items Discussed: a) Discussed about the Documentation structure and necessary artifacts that need to be included in the artifact folder. b) Discussed about the format of the appendices of the report. Achievements: Understood the format for collecting the project artifact. Tasks for Next Meeting: (Write down the task assigned to you by your supervisor) Complete the task mentioned in the checklist	



Student Sign

Internal Supervisor Sign



External Supervisor Sign

Figure 197 log sheet 11